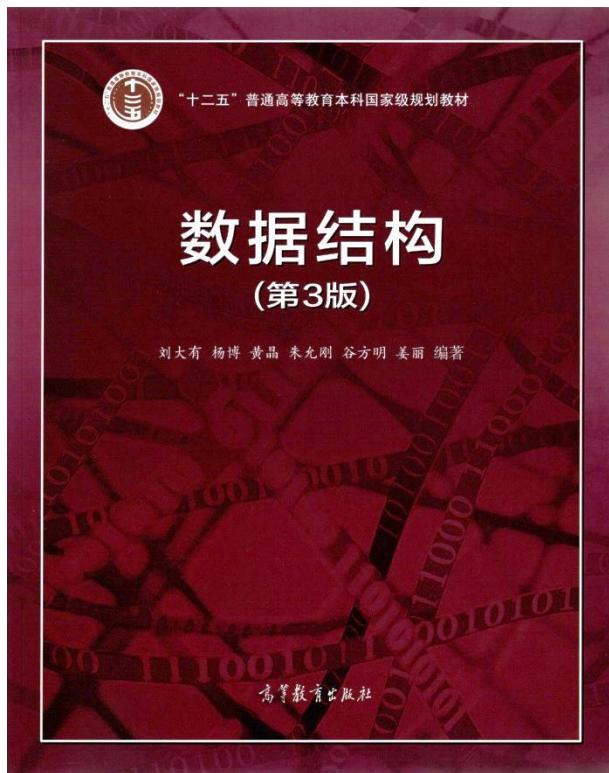




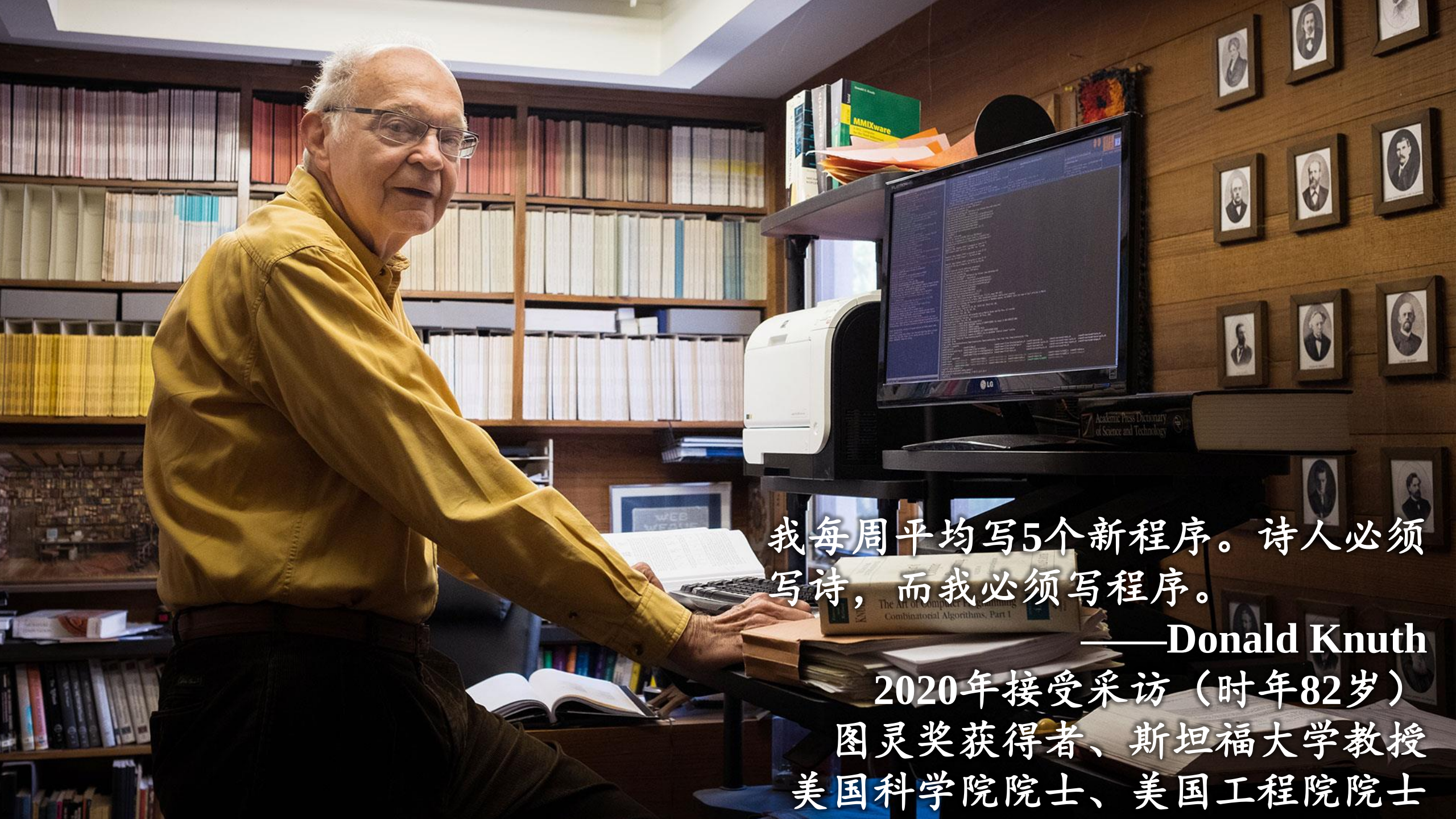
二叉树的存储和操作

- 二叉树的存储结构
- 二叉树的遍历
- 二叉树的其他基本操作
- 二叉树的序列化/反序列化
- 深度遍历的非递归算法



数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn



我每周平均写5个新程序。诗人必须写诗，而我必须写程序。

——Donald Knuth

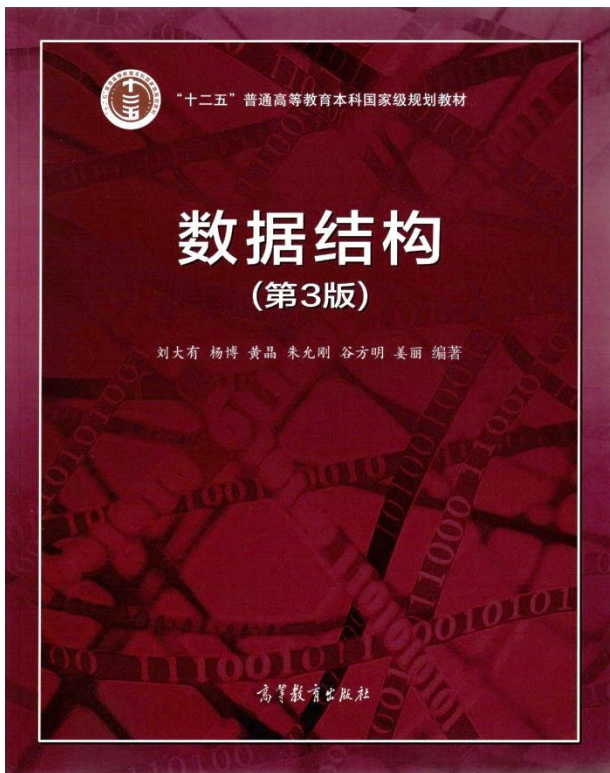
2020年接受采访（时年82岁）

图灵奖获得者、斯坦福大学教授
美国科学院院士、美国工程院院士



二叉树的存储和操作

- **二叉树的存储结构**
- 二叉树的遍历
- 二叉树的其他基本操作
- 二叉树的序列化/反序列化
- 深度遍历的非递归算法



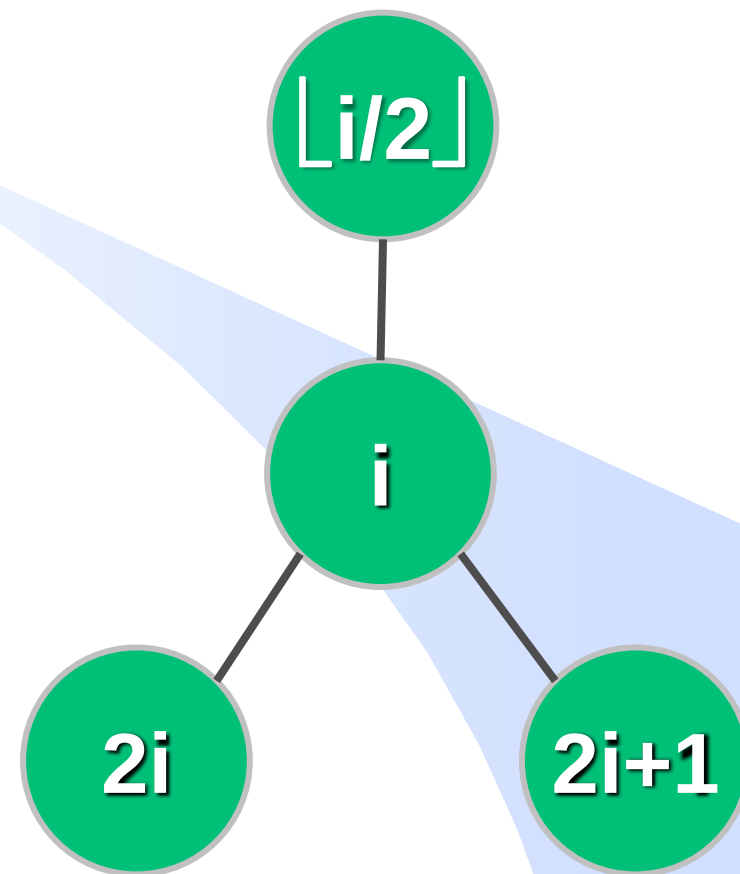
数据之法
结构之美
算法之道

二叉树的顺序存储

- 要存储一棵二叉树，必须存储其所有结点的**数据信息**、左孩子和右孩子**地址**，可用**顺序结构**存储，也可用**链接结构**存储。
- 二叉树的顺序存储是指将二叉树中所有结点存放在一块地址连续的存储空间中，同时**反映出二叉树中结点间的逻辑关系**。

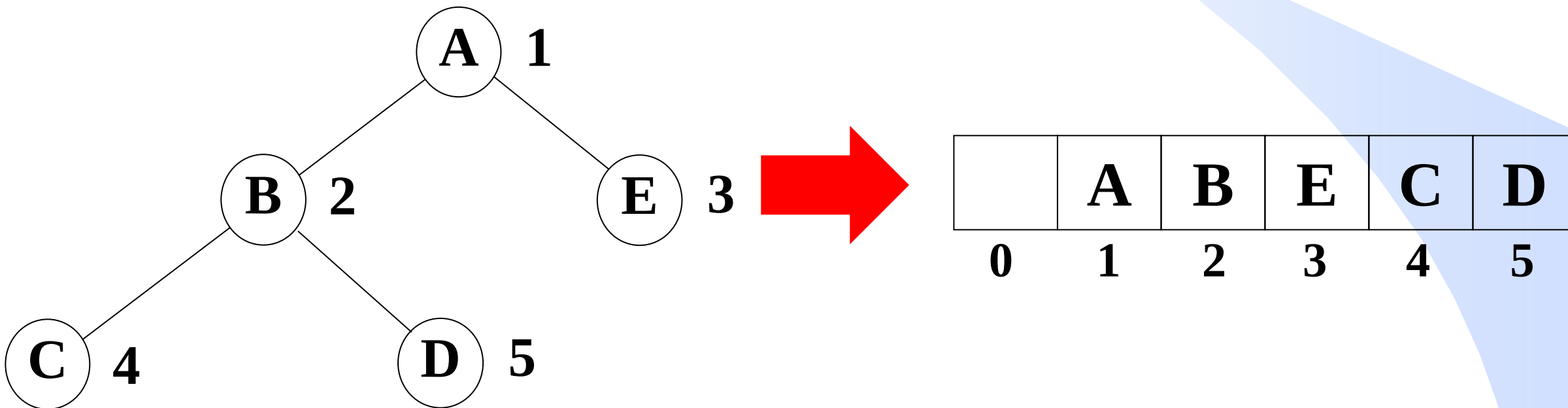
二叉树的顺序存储

- 回顾：对于完全二叉树，可按层次顺序对结点编号，结点的编号恰好反映了结点间的逻辑关系。
- 借鉴上述思想，利用一维数组 T 存储二叉树，把编号作为数组下标，根结点存放在 $T[1]$ 位置。
- 结点 $T[i]$ 的左孩子（若存在）存放在 $T[2i]$ 处，而 $T[i]$ 的右孩子（若存在）存放在 $T[2i+1]$ 处。



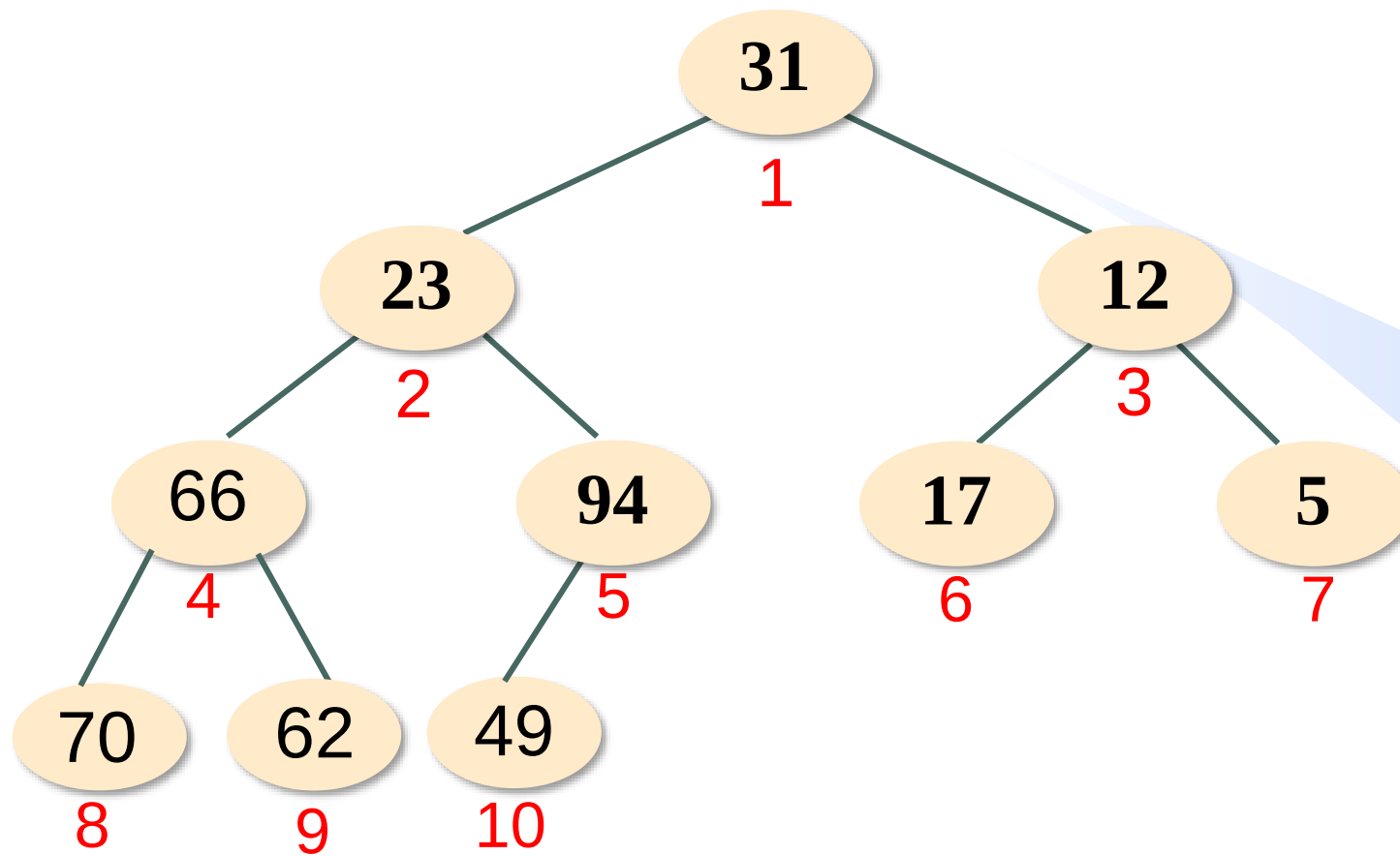
二叉树的顺序存储

若一个结点的下标是 i ，则其左孩子（若存在）存放在下标 $2i$ 处，右孩子（若存在）存放在 $2i+1$ 处



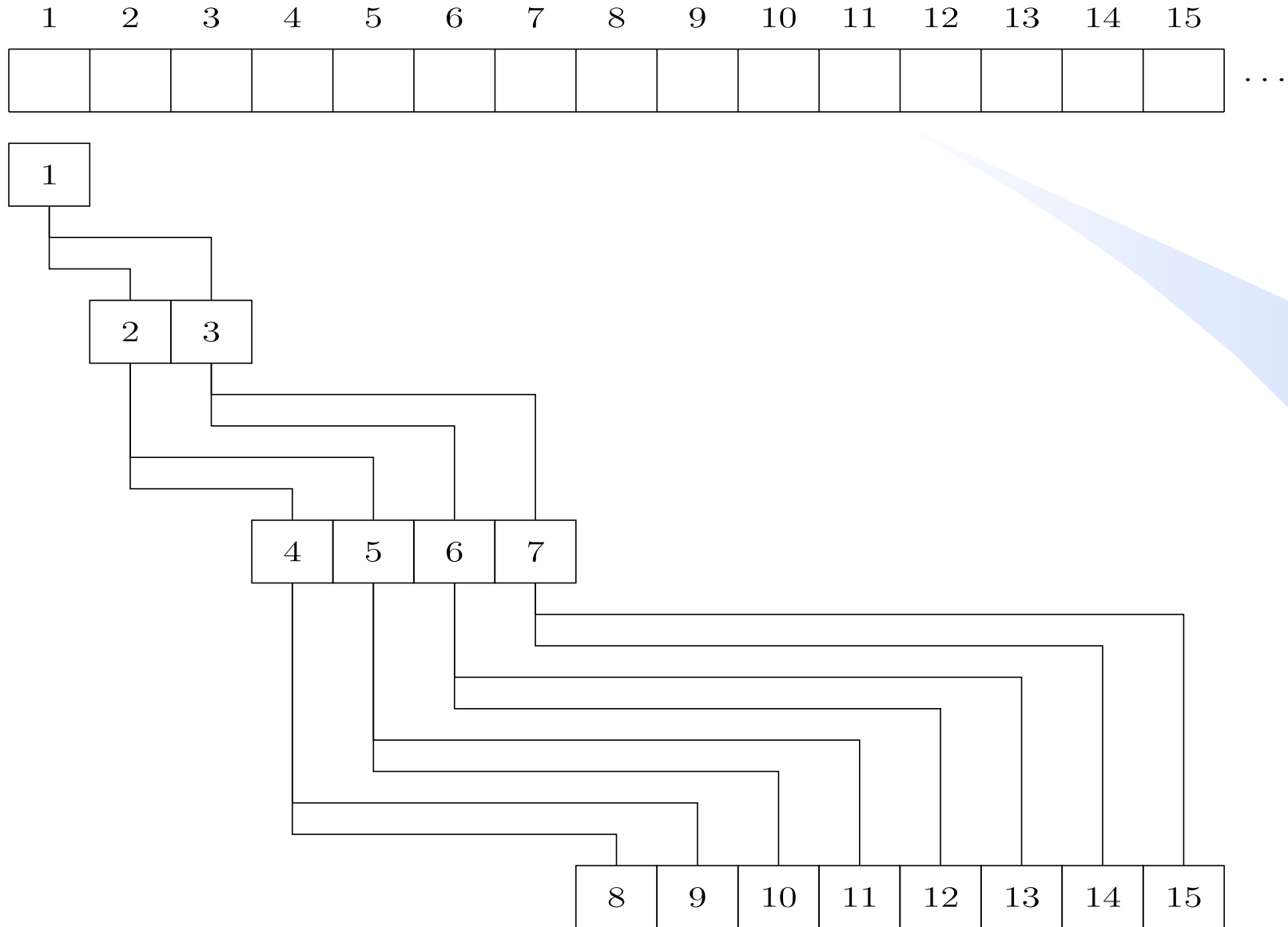
二叉树顺序存储结构

二叉树的顺序存储

 T

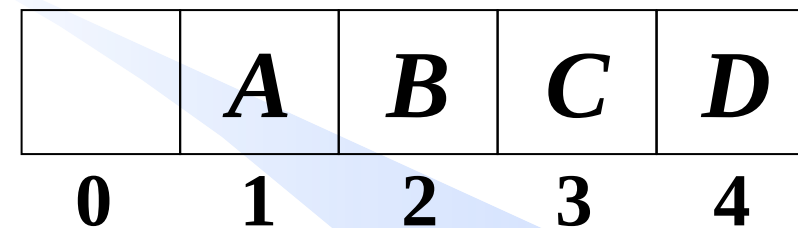
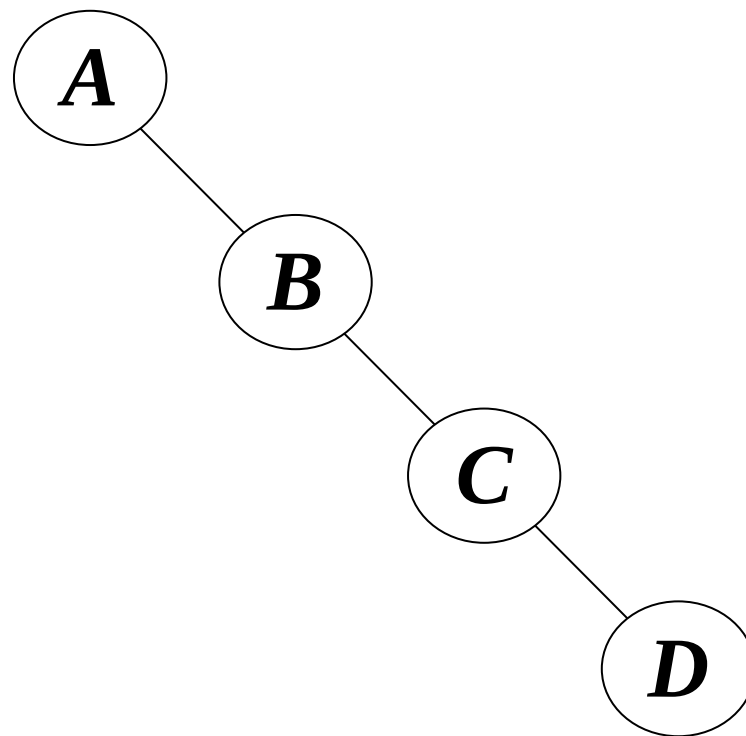
	31	23	12	66	94	17	5	70	62	49
0	1	2	3	4	5	6	7	8	9	10

二叉树的顺序存储

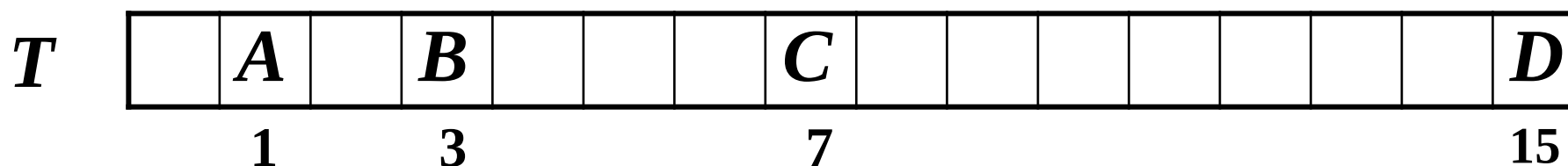


二叉树的顺序存储

➤ 适合于完全二叉树。但应用到非完全二叉树时，将造成空间浪费。



无法表达父子结点间的逻辑关系



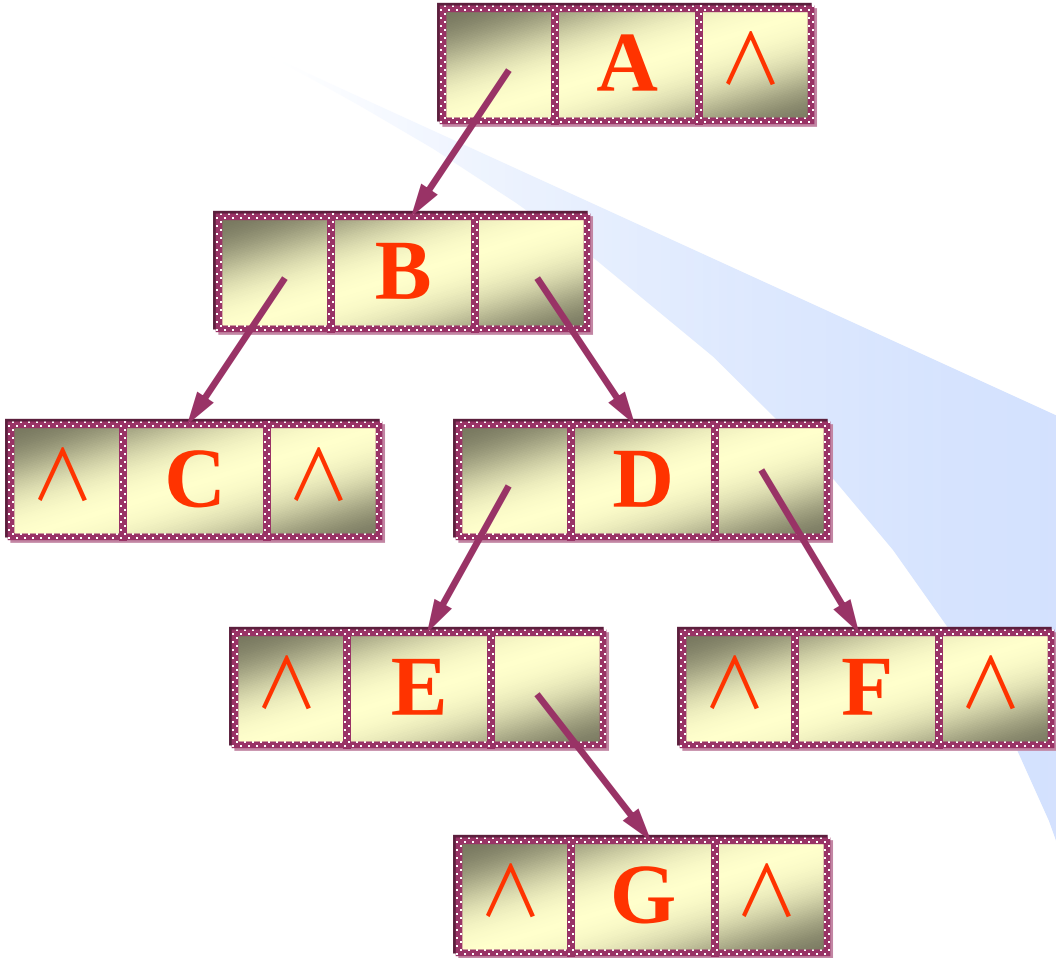
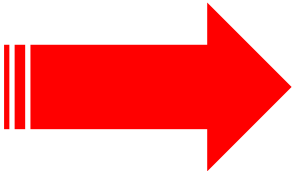
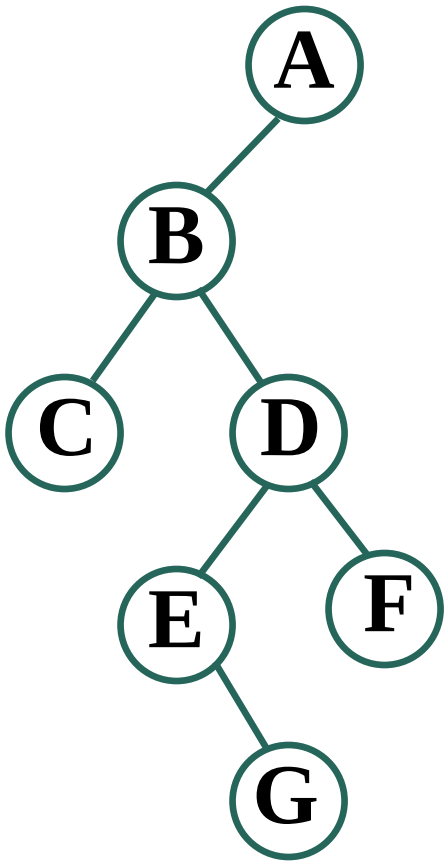
二叉树链接存储——二叉链表

- 各结点被随机存放在内存空间中，结点间的关系用指针表达。
- 二叉树的结点结构：二叉树结点应包含三个域——数据域 *data*、指针域 *left*（称为左指针）和指针域 *right*（称为右指针），其中左、右指针分别指向该结点的左、右子结点。



```
struct TreeNode{  
    int data;  
    TreeNode* left;  
    TreeNode* right;  
};
```

二叉树链接存储



二叉树链接存储

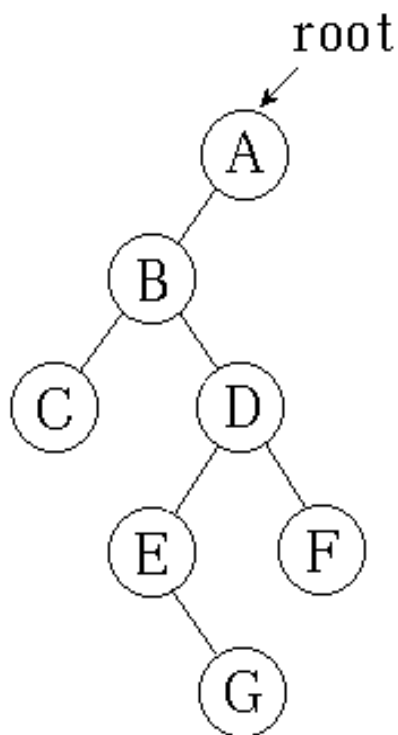
- 左子结点 = 左孩子
- 右子结点 = 右孩子
- **ADL**: *Left* (*t*)、*Data* (*t*)、*Right* (*t*)



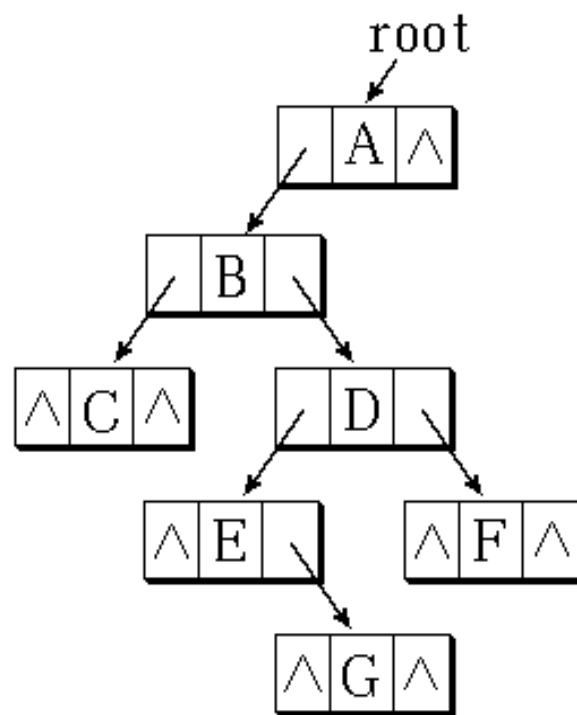
二叉树链接存储——三叉链表

<i>Left</i>	<i>Data</i>	<i>Parent</i>	<i>Right</i>
-------------	-------------	---------------	--------------

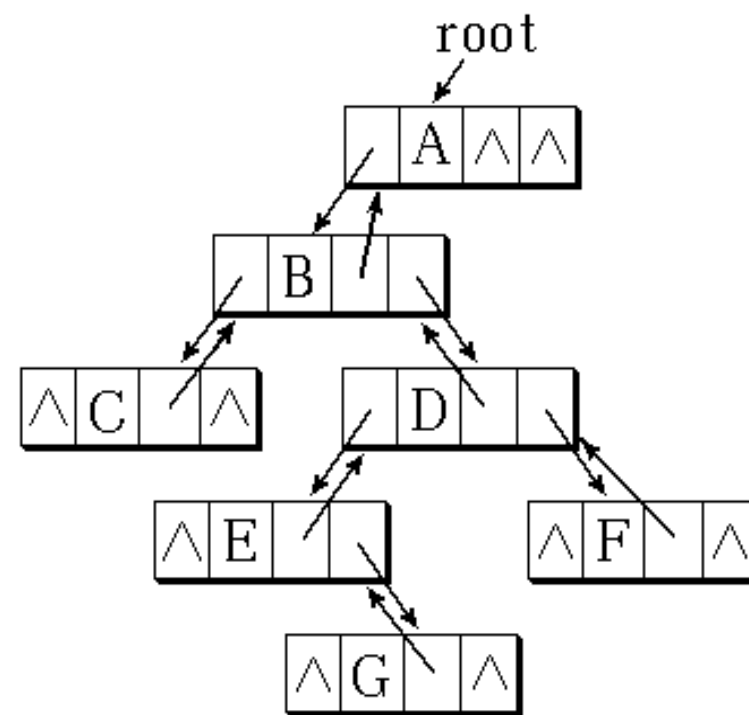
*Parent*指针指向父结点



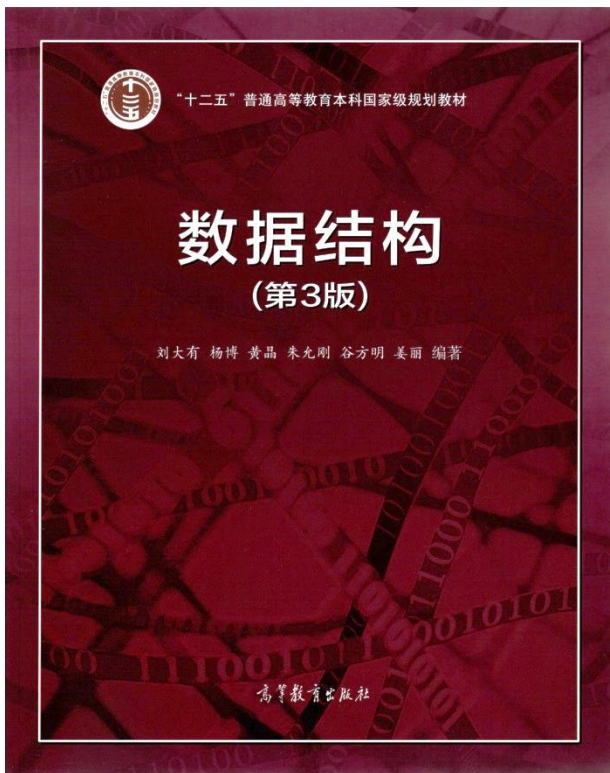
(a) 二叉树



(b) 二叉链表



(c) 三叉链表



二叉树的存储和操作

- 二叉树的存储结构
- **二叉树的遍历**
- 二叉树的其他基本操作
- 二叉树的序列化/反序列化
- 深度遍历的非递归算法

数据之法
结构之美
算法之道

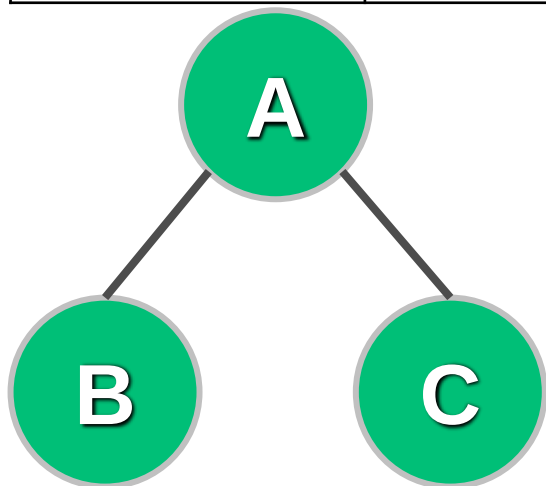
zhuyungang@jlu.edu.cn

二叉树的遍历

二叉树的遍历：按照一定**次序**访问二叉树中所有结点，并且每个结点仅被**访问一次**的过程。

当二叉树为空则什么都不做；否则遍历分三步进行：

遍历方法 步骤	先根遍历 (前序遍历)
步骤一	访问根结点
步骤二	先根遍历左子树
步骤三	先根遍历右子树

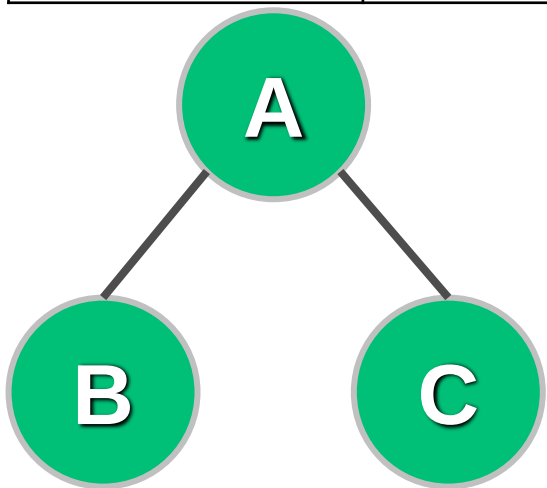


遍历方式

先根遍历：ABC

当二叉树为空则什么都不做；否则遍历分三步进行：

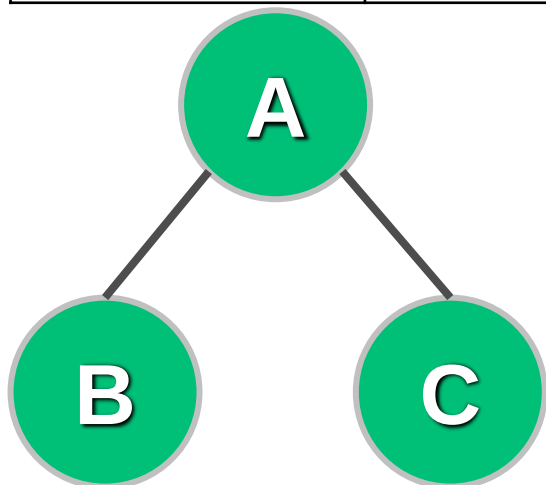
遍历方法 步骤	先根遍历 (前序遍历)	中根遍历 (中序遍历)
步骤一	访问根结点	中根遍历左子树
步骤二	先根遍历左子树	访问根结点
步骤三	先根遍历右子树	中根遍历右子树



遍历方式 { 先根遍历：ABC
中根遍历：BAC

当二叉树为空则什么都不做；否则遍历分三步进行：

遍历方法 步骤	先根遍历 (前序遍历)	中根遍历 (中序遍历)	后根遍历 (后序遍历)
步骤一	访问根结点	中根遍历左子树	后根遍历左子树
步骤二	先根遍历左子树	访问根结点	后根遍历右子树
步骤三	先根遍历右子树	中根遍历右子树	访问根结点



遍历方式 { 先根遍历：ABC
中根遍历：BAC
后根遍历：BCA

先根（中根、后根）遍历二叉树 T，得到 T 之结点的一个序列，称为 T 的先根（中根、后根）序列。

先根遍历 (Preorder Traversal, 前/先序遍历)

先根遍历二叉树算法的框架:

➤ 若二叉树为空, 则空操作;

➤ 否则

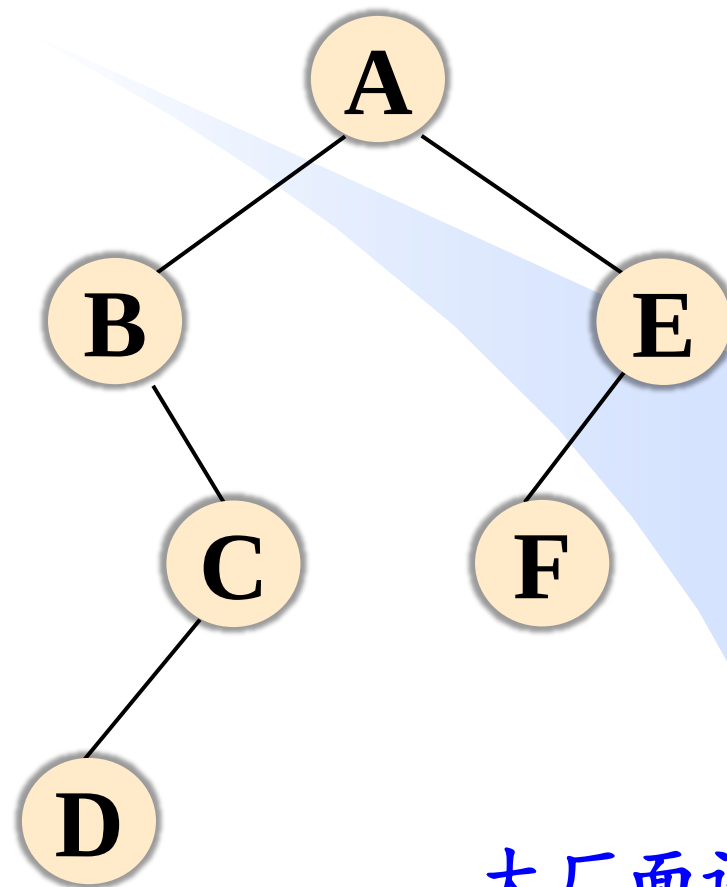
✓ 访问根结点;

✓ 先根遍历左子树;

✓ 先根遍历右子树。

遍历结果

A B C D E F



大厂面试题
[LeetCode144](https://leetcode.com/problems/binary-tree-preorder-traversal/)

二叉树递归先根遍历的算法

```
void Preorder(TreeNode* root){  
    if(root == NULL) return;  
    visit(root->data);  
    Preorder(root->left);  
    Preorder(root->right);  
}
```

时间复杂度 $O(n)$
空间复杂度 $O(h)$
 n 为二叉树结点数
 h 为二叉树高度

```
void visit(int data){  
    printf("%d ", data);  
}
```

二叉树递归先根遍历的算法

```
void Preorder(TreeNode* root){  
    if(root == NULL) return;  
    visit(root->data);  
    Preorder(root->left);  
    Preorder(root->right);  
}
```

分治法
分而治之

大事化小
小事化了

子问题1
访问根结点

子问题2
访问左子树的结点

子问题3
访问右子树的结点

大问题
访问整棵树的
所有结点

分治法

子问题相互独立
不重叠

自顶向下
递归实现

动态规划

子问题间往往具有重叠性，可
将子问题的解存入表中，以后
再遇到相同的子问题直接查表

自底向上
递推实现

中根遍历 (Inorder Traversal, 中序遍历)

中根遍历二叉树算法的框架:

➤ 若二叉树为空, 则空操作;

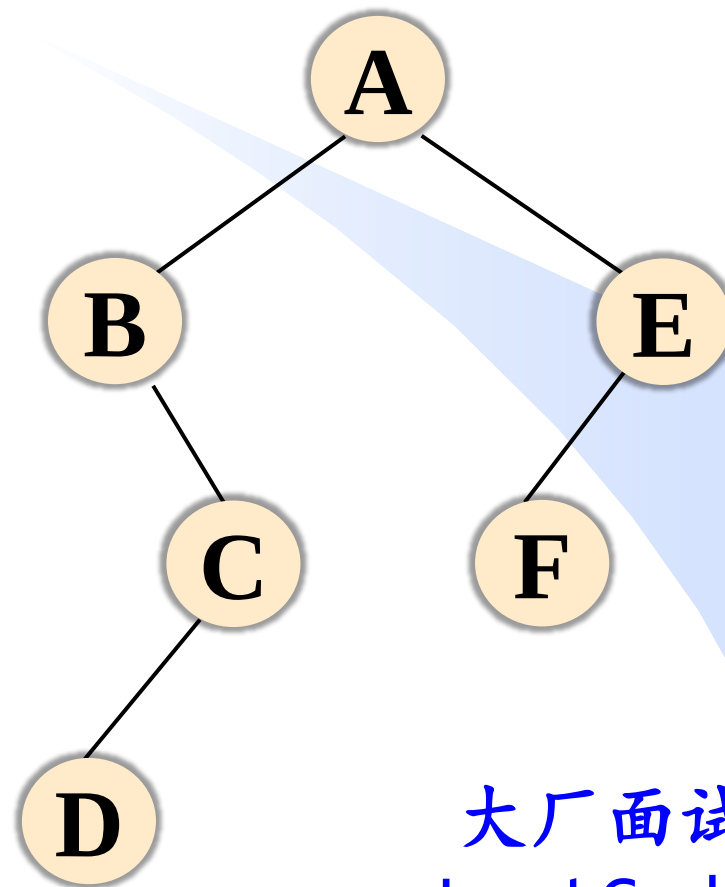
➤ 否则

✓ 中根遍历左子树;

✓ 访问根结点;

✓ 中根遍历右子树。

遍历结果 **B D C A F E**



大厂面试题
[LeetCode94](https://leetcode.com/problems/binary-tree-inorder-traversal/)

二叉树中根遍历的递归算法

```
void Inorder(TreeNode* root){  
    if (root == NULL) return;  
    Inorder(root->left);  
    visit(root->data);  
    Inorder(root->right);  
}
```

时间复杂度 $O(n)$
空间复杂度 $O(h)$
 n 为二叉树结点数
 h 为二叉树高度

```
void visit (int data){  
    printf("%d ", data);  
}
```

后根遍历 (Postorder Traversal, 后序遍历)

后根遍历二叉树算法的框架:

➤ 若二叉树为空, 则空操作;

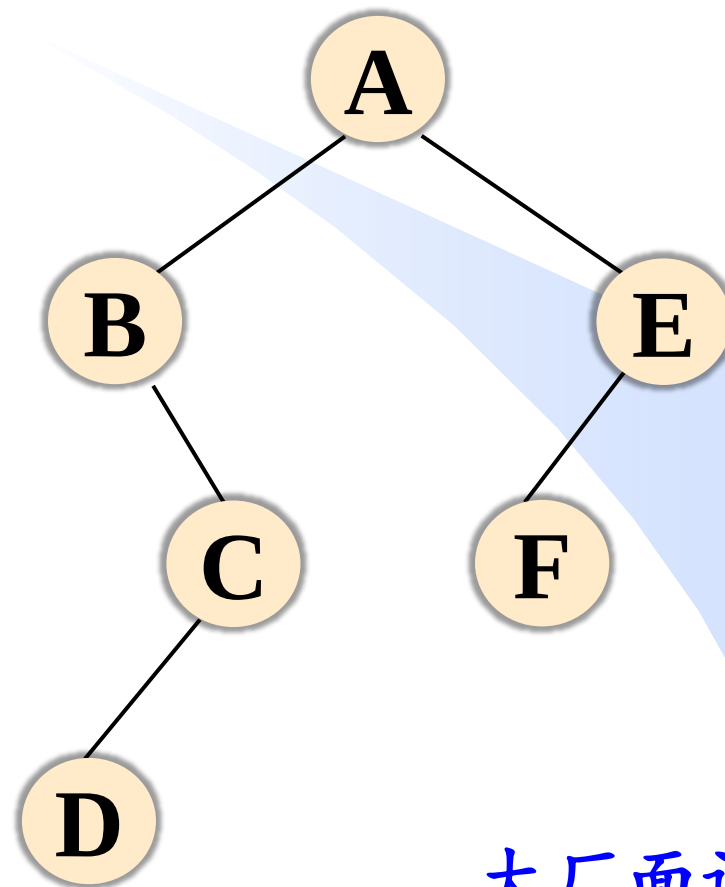
➤ 否则

✓ 后根遍历左子树;

✓ 后根遍历右子树;

✓ 访问根结点。

遍历结果 **D C B F E A**



大厂面试题
[LeetCode145](https://leetcode.com/problems/postorder-traversal/)

二叉树后根遍历的递归算法

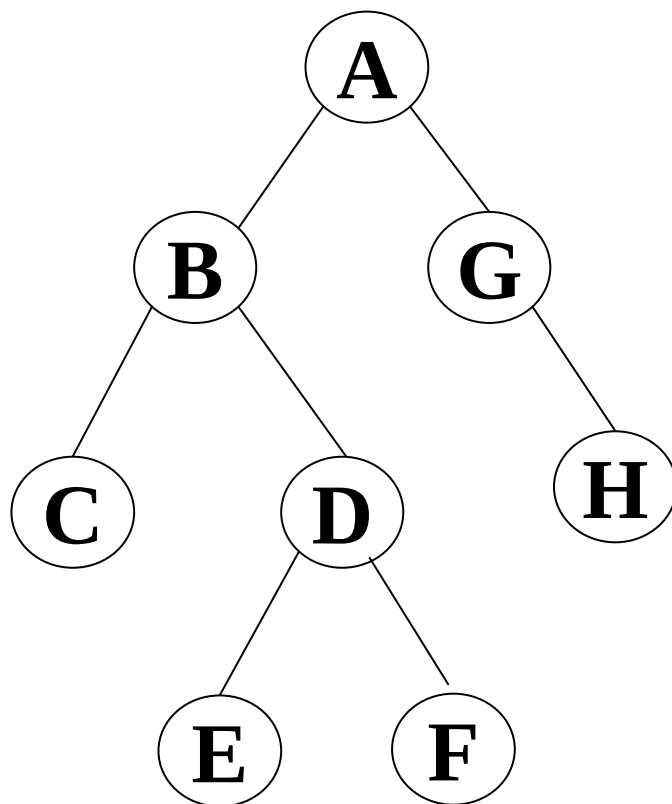
```
void Postorder(TreeNode* root){  
    if (root == NULL) return;  
    Postorder(root->left);  
    Postorder(root->right);  
    visit(root->data);  
}
```

时间复杂度 $O(n)$
空间复杂度 $O(h)$
 n 为二叉树结点数
 h 为二叉树高度

```
void visit (int data){  
    printf("%d ", data);  
}
```

课下练习

下面二叉树的先根序列为_____，中根序列为_____
_____, 后根序列为_____。



练习

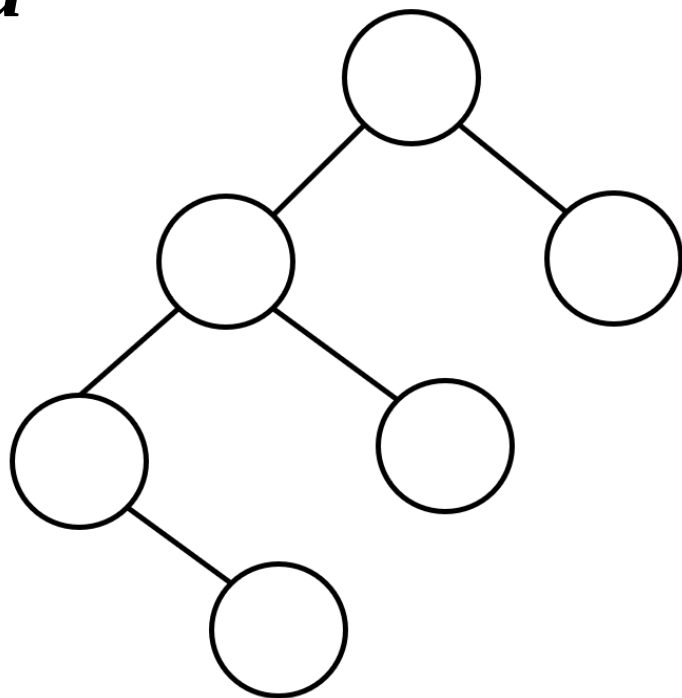
已知一棵二叉树的树形如下，若其后根序列为 f, d, b, e, c, a ，则其先根序列为 A。【2023年考研题全国卷】

A. $a \ e \ d \ f \ b \ c$

B. $a \ c \ e \ b \ d \ f$

C. $c \ a \ b \ e \ f \ d$

D. $d \ f \ e \ b \ a \ c$



练习

要使一棵非空二叉树的**先根序列**与**中根序列**相同，其所有非叶结点须满足的条件是(**B**) 【2017年考研题全国卷】

A.只有左子树

B.只有右子树

C.结点的度均为1

D.结点的度均为2

先根序列：根 左 右

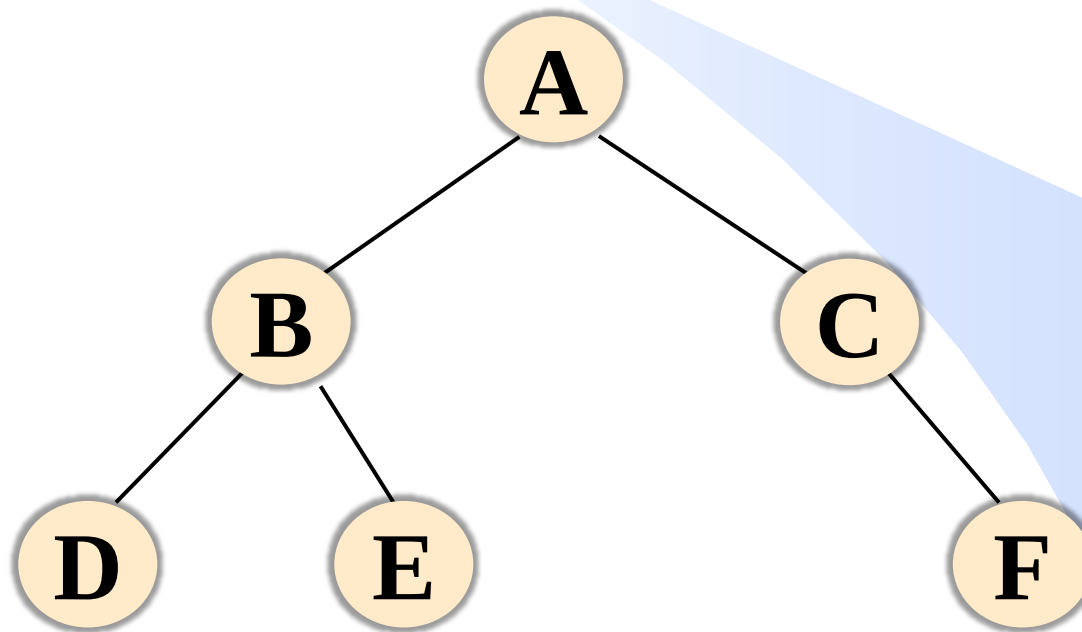
中根序列：左 根 右

层次遍历——定义

按层数由小到大，同层由左向右的次序访问结点。

遍历结果：

A B C D E F



层次遍历——实现

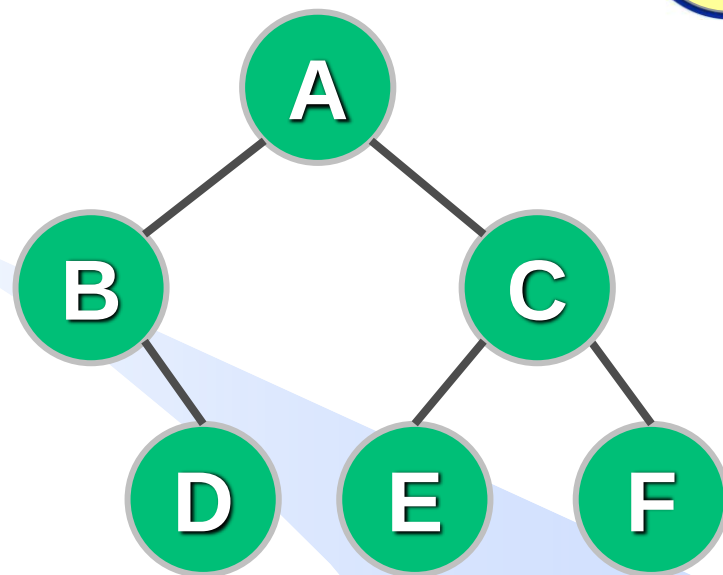
- 通过观察发现，在第 i 层上若结点 x 在结点 y 的左边，则 x 一定在 y 之前被访问。
- 并且，在第 $i+1$ 层上， x 的子结点一定在 y 的子结点之前被访问。
- 用一个队列来实现。

层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。

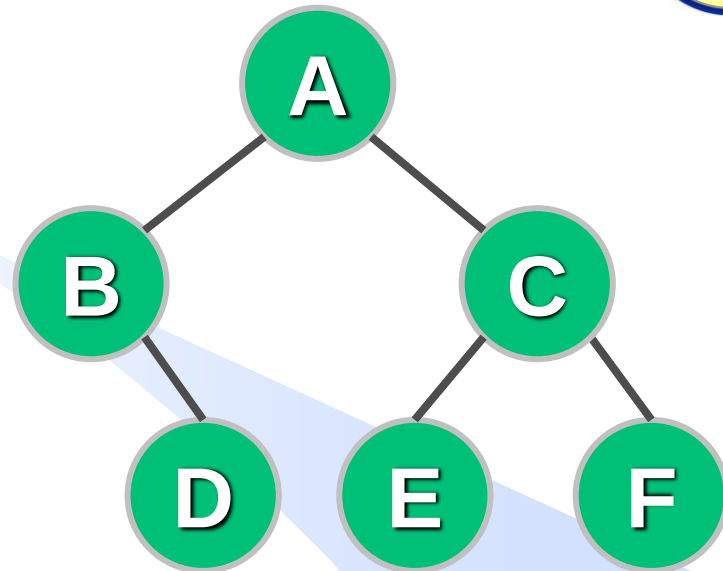


A入队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。



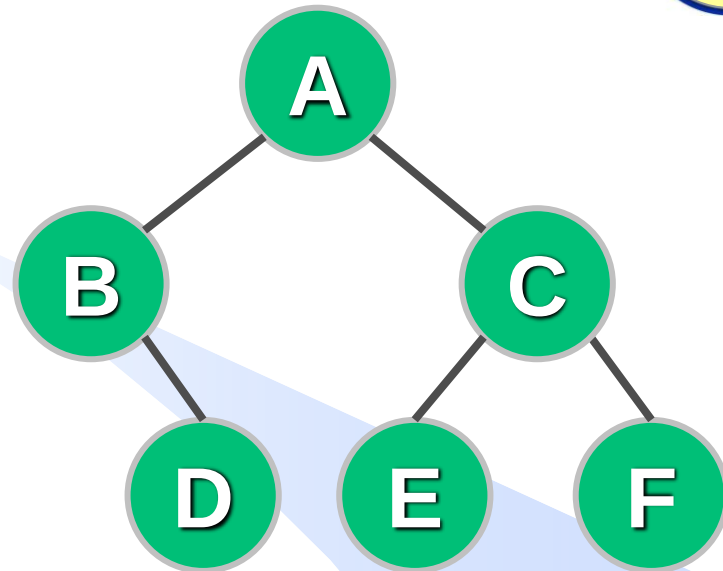
A出队

B、C入队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。

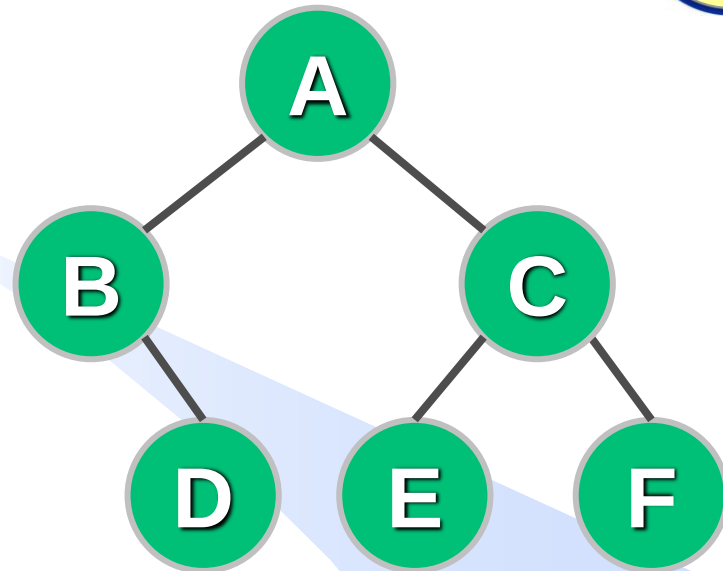


B出队
D入队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。



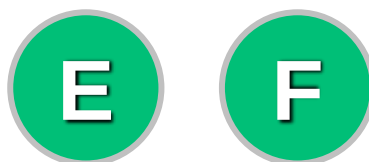
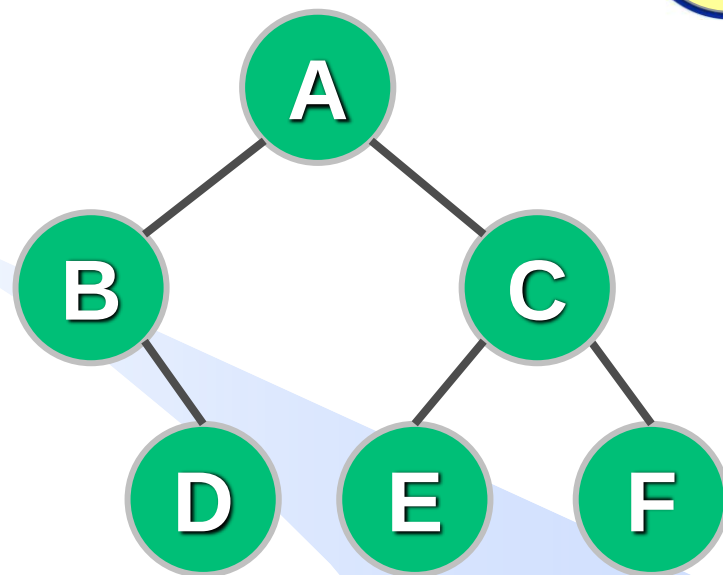
C出队

E、F入队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。

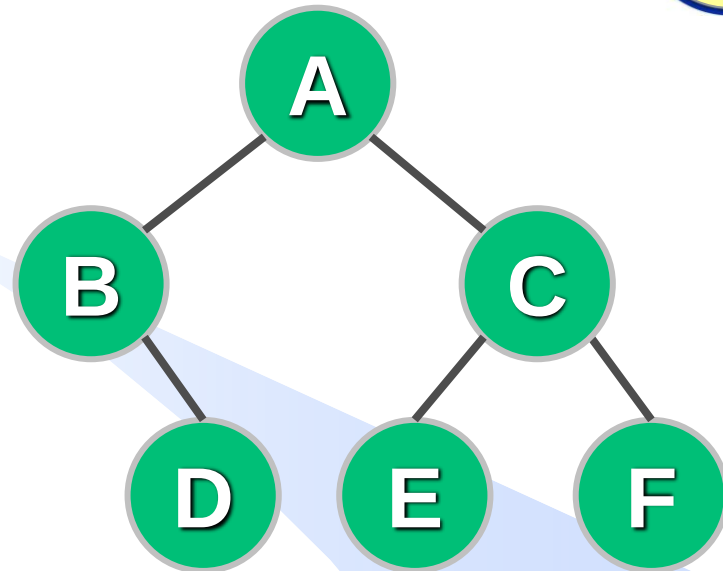


D出队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。

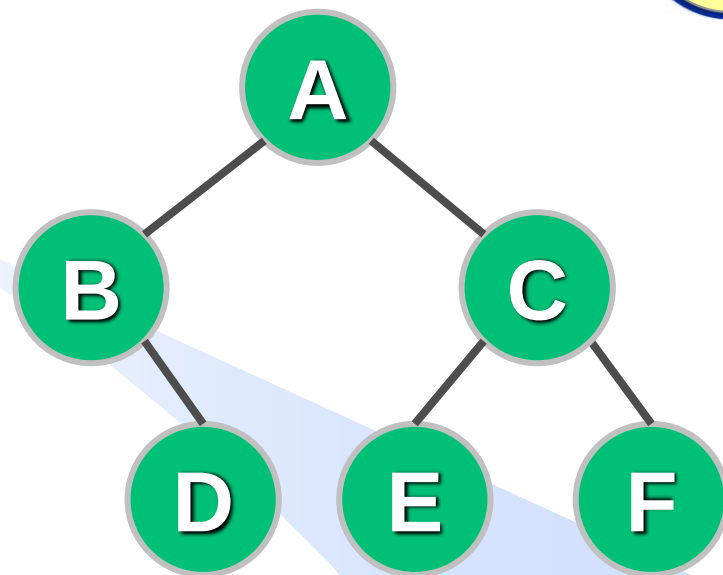


E出队



层次遍历——实现

- 根结点入队。
- 重复下列步骤直至队列为空：
 - ✓ 出队一个结点并访问；
 - ✓ 若其有左孩子，将左孩子入队；
 - ✓ 若其有右孩子，将右孩子入队。



F出队



二叉树层次遍历——实现

```
void LevelOrder(TreeNode *root){  
    Queue<TreeNode*> Q;  
    if(root!=NULL) Q.enqueue(root);  
    while(!Q.empty()){  
        TreeNode* p = Q.dequeue();  
        visit(p->data);  
        if(p->left!=NULL) Q.enqueue(p->left);  
        if(p->right!=NULL) Q.enqueue(p->right);  
    }  
}
```

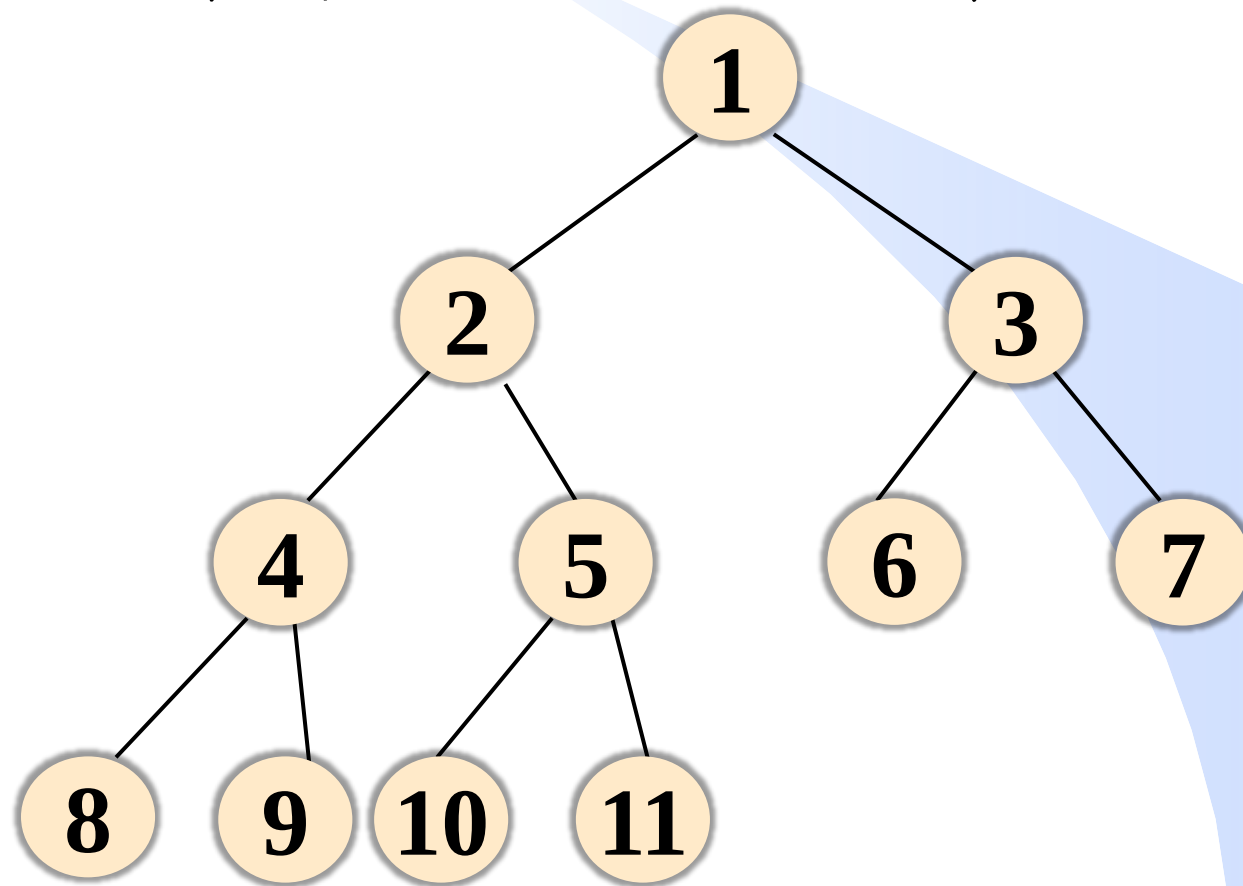
辅助队列的规模

若对由2021个结点构成的**完全二叉树**进行层次遍历，辅助队列的容量至少需要多大（即同时保存在队列中的结点的最大个数）。【清华大学、吉林学期末考试题】

辅助队列的规模

若对由 n 个结点构成的**完全二叉树**进行层次遍历，辅助队列的容量至少需要多大（即同时保存在队列中的结点的最大个数）。

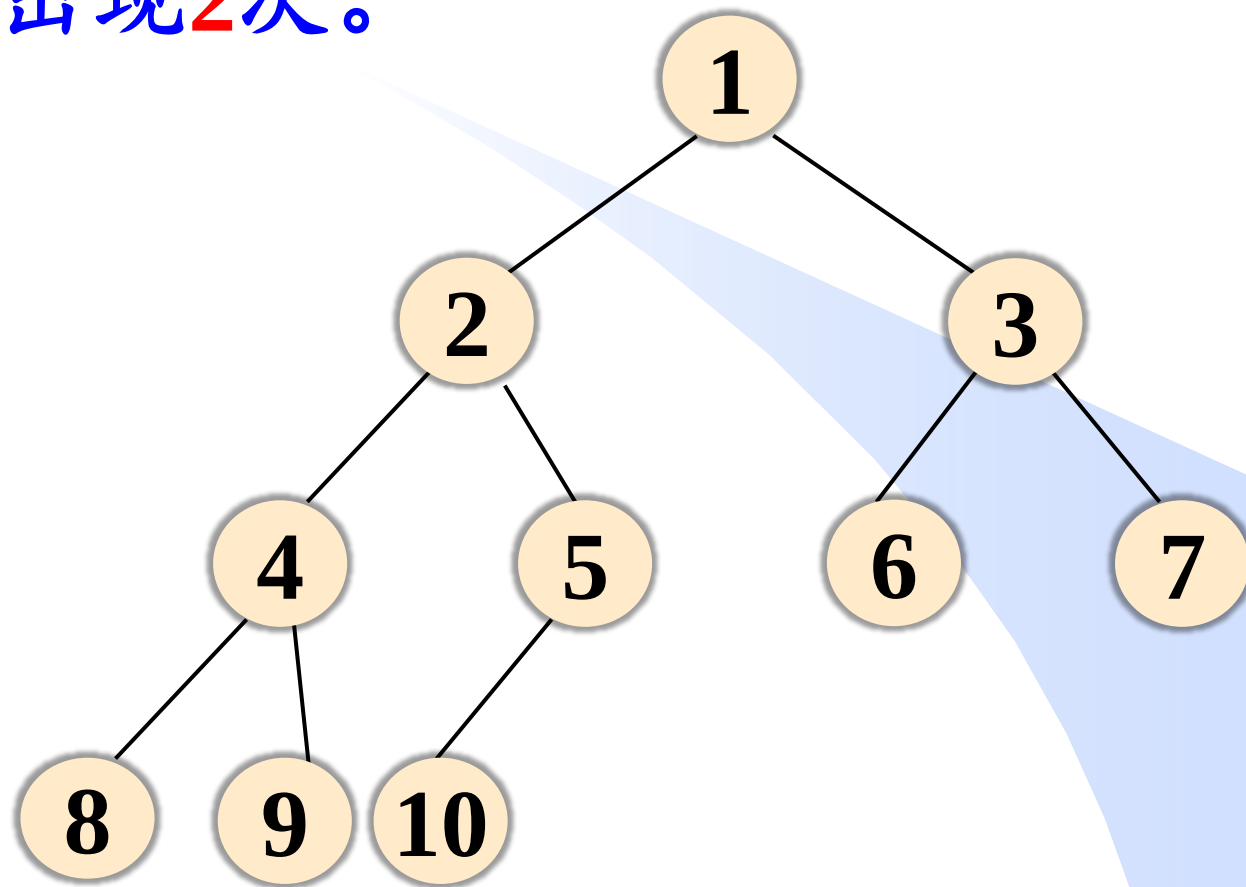
最大规模： $\lceil n/2 \rceil$



辅助队列的规模

课下思考：最大规模可能出现**2**次。

5	6	7	8	9
6	7	8	9	10



课下思考

判断题：

对叶结点数量为2018的二叉树进行层次遍历，辅助队列的最大规模（同时保存在队列中的结点个数）绝对不超过2018. 【清华大学考研题】

总结

先根遍历

中根遍历

后根遍历

层次遍历

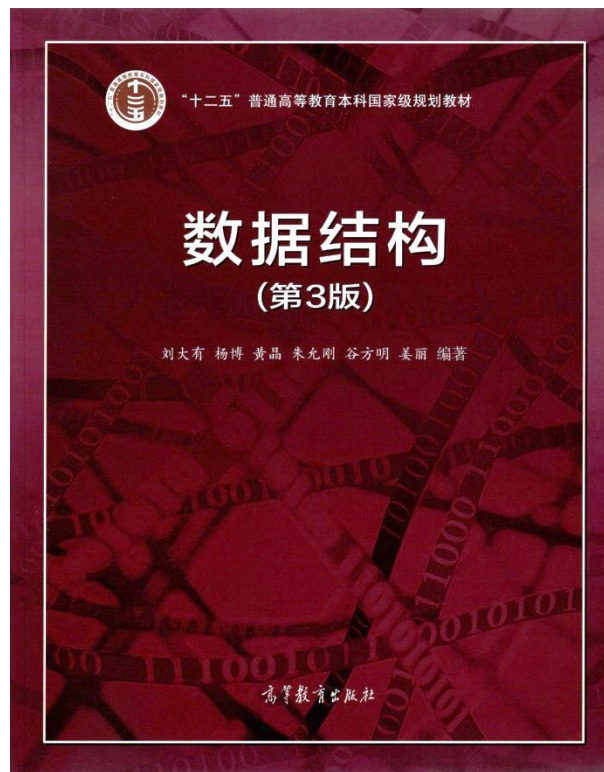
深度优先搜索

广度优先搜索



二叉树的存储和操作

- 二叉树的存储结构
- 二叉树的遍历
- **二叉树的其他基本操作**
- 二叉树的序列化/反序列化
- 深度遍历的非递归算法

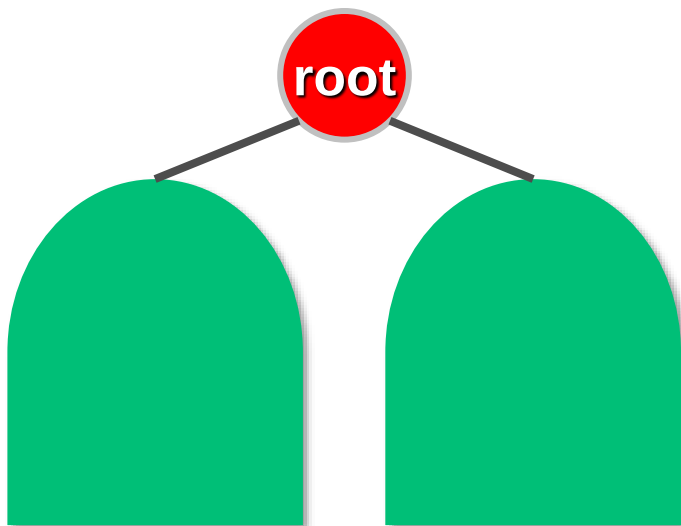


数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

搜索二叉树中符合数据域条件的结点

```
TreeNode* Search(TreeNode *root, int K){  
    //在以root为根的二叉树中找数据域为K的结点，返回指针  
    if(root == NULL) return NULL;  
    if(root->data == K) return root;           //root即为所求  
    TreeNode* ans=Search(root->left, K);      //在左子树中查找  
    if(ans!=NULL) return ans;  
    return Search(root->right, K);            //在右子树中查找  
}
```



分治法

遍历

回溯

深度优先搜索

在二叉树中搜索给定结点的父结点

A

```
TreeNode* FindParent(TreeNode *root, TreeNode *p){  
    //在以root为根的二叉树中找p的父结点，返回指针  
    if(root==NULL || p==root) return NULL;    //根空或p为根  
    if(root->left==p || root->right==p)        //root即p的父结点  
        return root;  
    TreeNode *ans= FindParent(root->left,p);    //在左子树找p父结点  
    if(ans!=NULL) return ans;  
    return FindParent(root->right, p);    //在右子树找p父结点  
}
```

解决二叉树问题的一般框架

算法 $f(\text{root})$

可在此处处理根结点

递归处理左子树 $f(\text{root} \rightarrow \text{left})$.

可在此处处理根结点

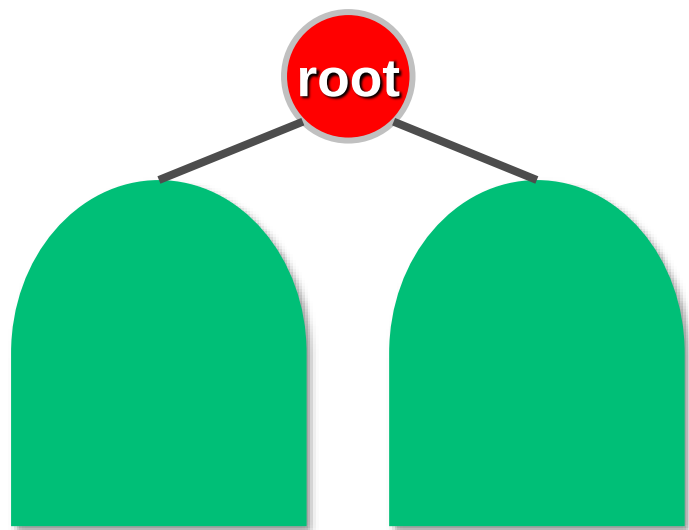
递归处理右子树 $f(\text{root} \rightarrow \text{right})$.

可在此处处理根结点

RETURN. ■

计算二叉树结点个数

```
int Count(TreeNode* root){  
    if(root==NULL) return 0;  
    return Count(root->left)+Count(root->right)+1;  
}
```

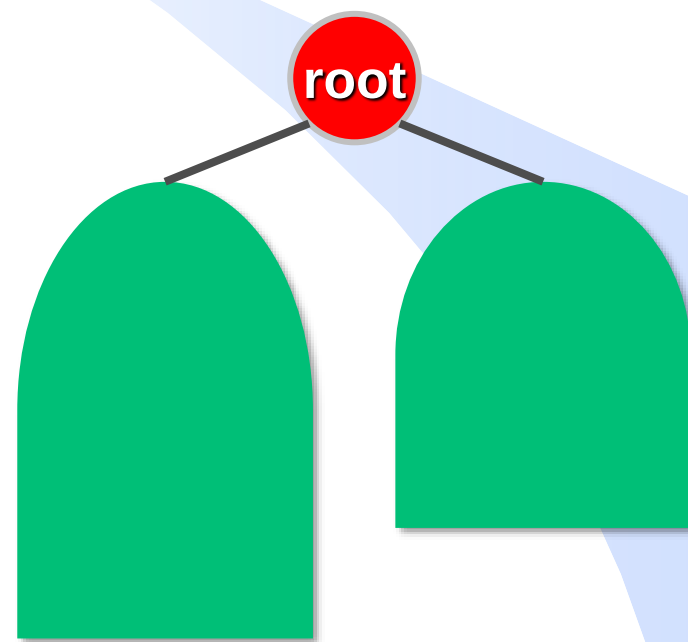


二叉树中结点总数 = 左子树结点总数 + 右子树结点总数 + 1 (根结点)

计算二叉树高度

$$\text{depth}(t) = \begin{cases} -1 & \text{若 } t = \text{NULL} \\ \max\{\text{depth}(t \rightarrow \text{left}), \text{depth}(t \rightarrow \text{right})\} + 1 & \text{若 } t \neq \text{NULL} \end{cases}$$

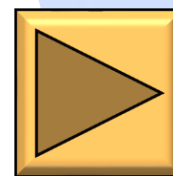
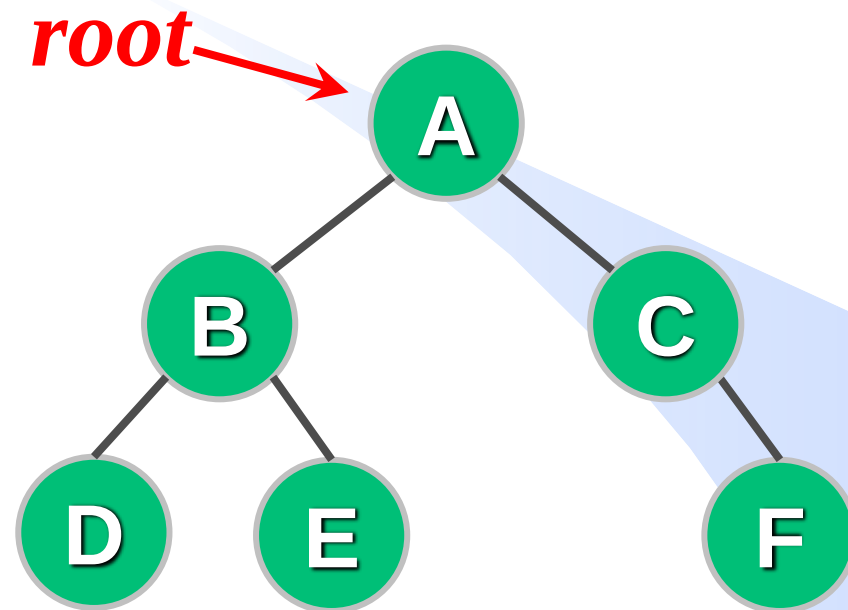
```
int depth(TreeNode* root){  
    if(root==NULL) return -1;  
    int d1 = depth(root->left);  
    int d2 = depth(root->right);  
    return(d1>d2)? d1+1:d2+1;  
}
```



删除二叉树

```
void Del(TreeNode* &root){ //删除以root为根的二叉树
    if(root==NULL) return;
    Del(root->left);
    Del(root->right);
    delete root;
    root = NULL;
}
```

不同于查询操作，**插入**、**删除**操作将使二叉树的形态发生变化，在函数执行过程中二叉树的**根指针**可能被修改，在函数退出后，这种修改应得以继续保留。可将根指针作为函数的“引用参数”进行传递。



类比

设计一个函数对变量a的值加1。

错误方案:

```
void increase(int x){  
    x++;  
}
```

主函数:

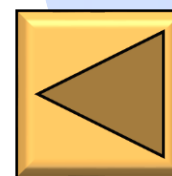
```
int a=5;  
increase(a);
```

正确方案:

```
void increase(int &x){  
    x++;  
}
```

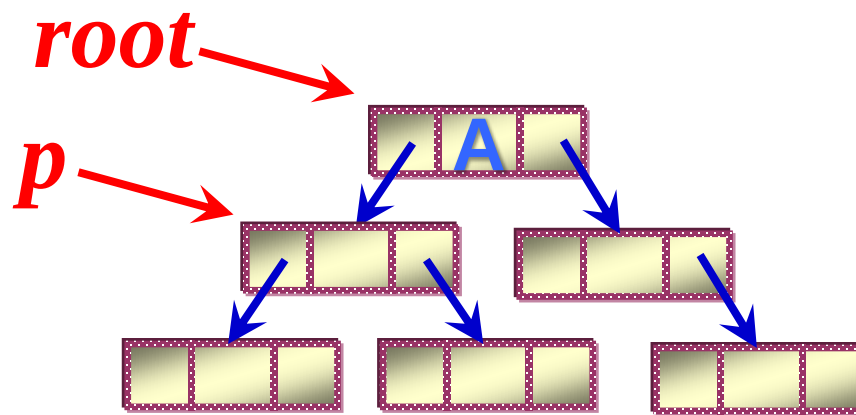
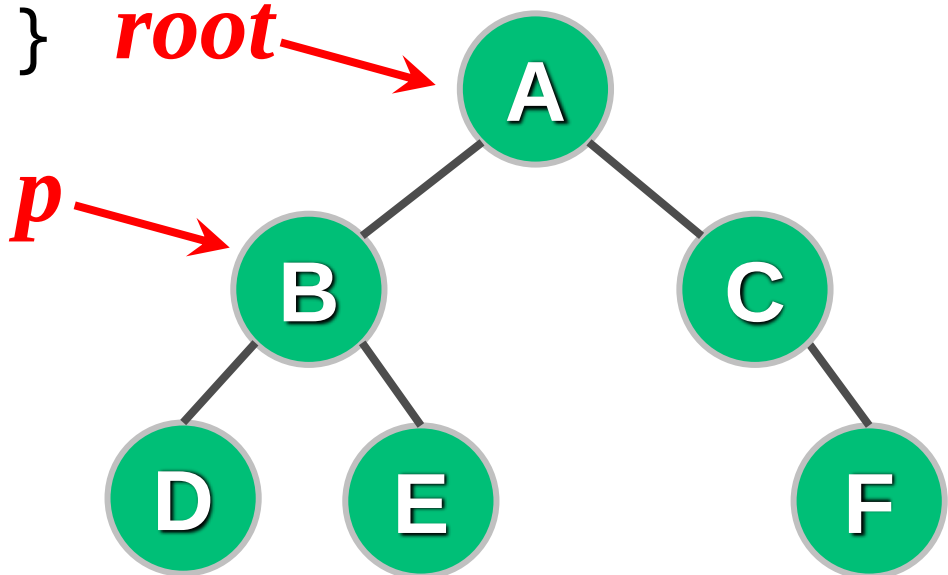
主函数:

```
int a=5;  
increase(a);
```



在二叉树中删除子树

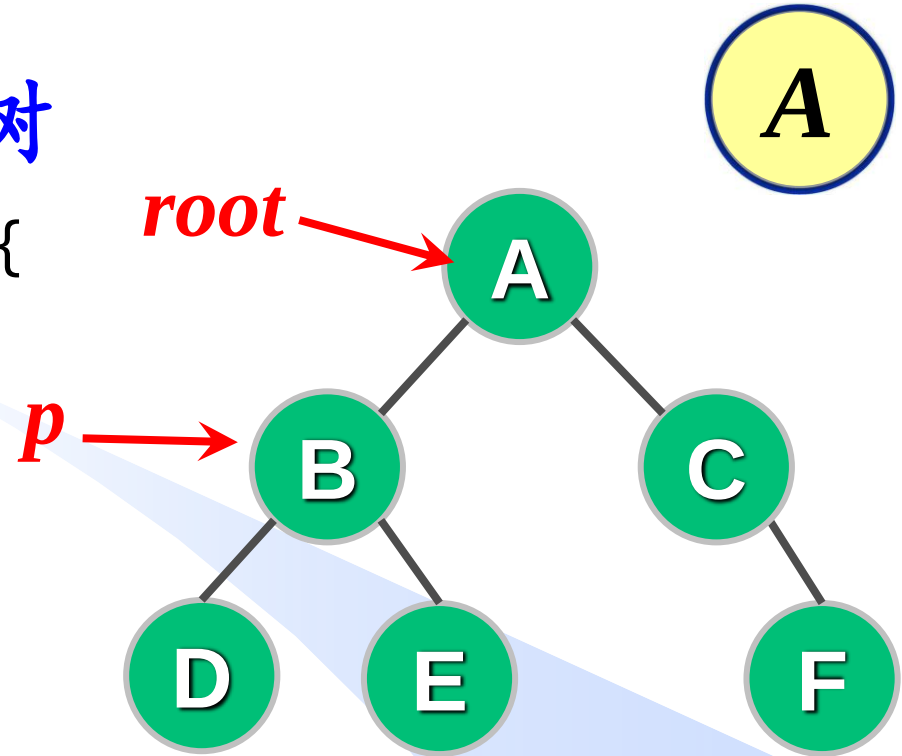
```
bool DelSubTree(TreeNode *&root, TreeNode *p){
    //在以root为根的二叉树中删除p指向的子树,删除成功返回真,否则返回假
    if(root==NULL || p==NULL) return false;
    if(p==root){ Del(root); return true;} //p是根
    if(DelSubTree(root->left,p)) return true; //在左子树中删p子树
    return DelSubTree(root->right, p); //在右子树中删p子树
} root
```



上述函数删除A的左子树后，会把A的左指针置为空么？

在二叉树中删除子树

```
bool DelSubTree(TreeNode *&root, TreeNode *p){  
    if(root==NULL || p==NULL) return false;  
    if(p==root){ Del(root); return true;}  
    if(DelSubTree(root->left, p)) return true;  
    return DelSubTree(root->right, p);  
}
```



在二叉树中删除子树

A

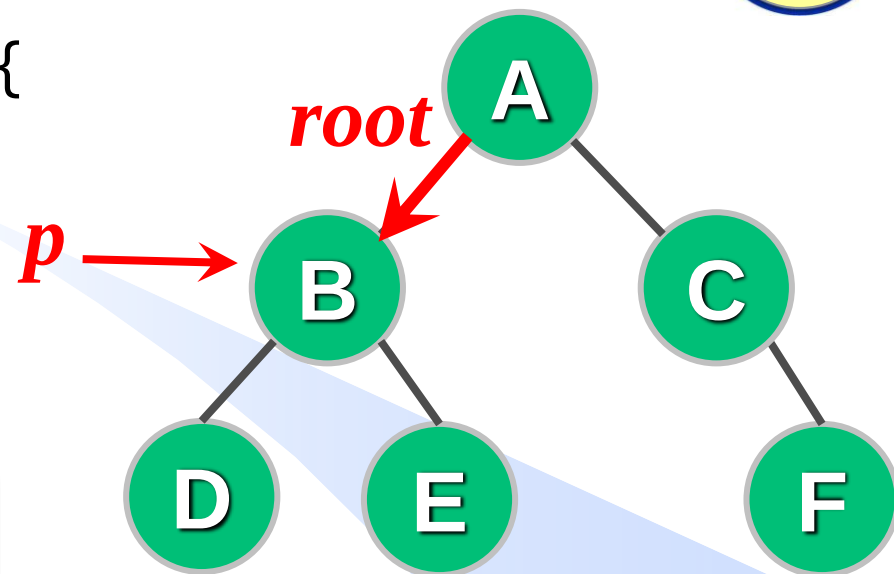
```
bool DelSubTree(TreeNode *&root, TreeNode *p){  
    if(root==NULL || p==NULL) return false;  
    if(p==root){ Del(root); return true;}  
    if(DelSubTree(root->left, p)) return true;  
    return DelSubTree(root->right, p);  
}
```

进入下
层递归



此处的root是上一层
root->left的引用

```
bool DelSubTree(TreeNode *&root, TreeNode *p){  
    if(root==NULL || p==NULL) return false;  
    if(p==root){ Del(root); return true;}  
    if(DelSubTree(root->left, p)) return true;  
    return DelSubTree(root->right, p);  
}
```



```
void Del(TreeNode* &root  
    if(root==NULL) return;  
    Del(root->left);  
    Del(root->right);  
    delete root;  
    root = NULL;  
}
```

创建二叉树

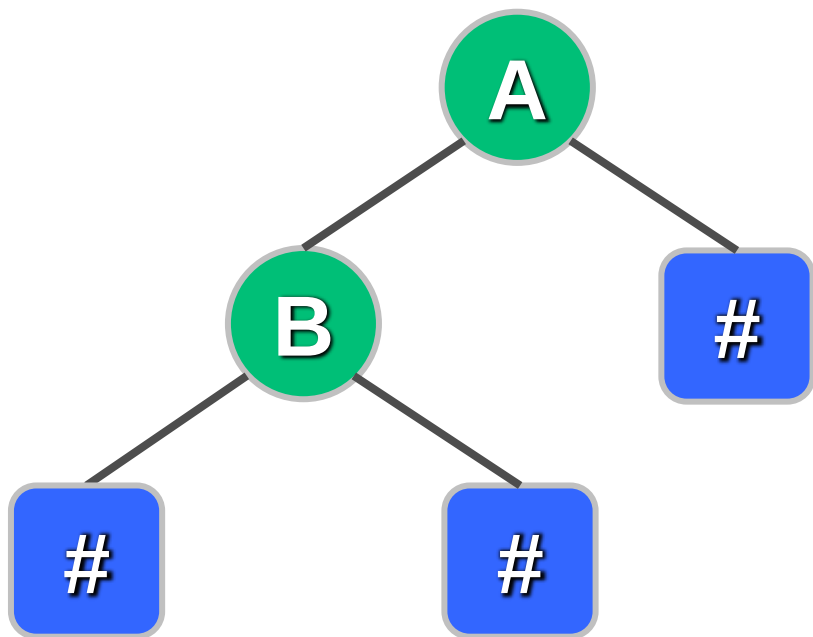
- 通过一种遍历序列：先根序列？
- 先根序列不能唯一确定二叉树。因为在二叉树中，**有的结点之左/右指针可能为空，这在先根序列中不能被体现**，导致两棵不同的二叉树却可能有相同的先根序列。
- 在先根序列中加入特殊符号以表示空指针位置，不妨用#表示空指针位置。



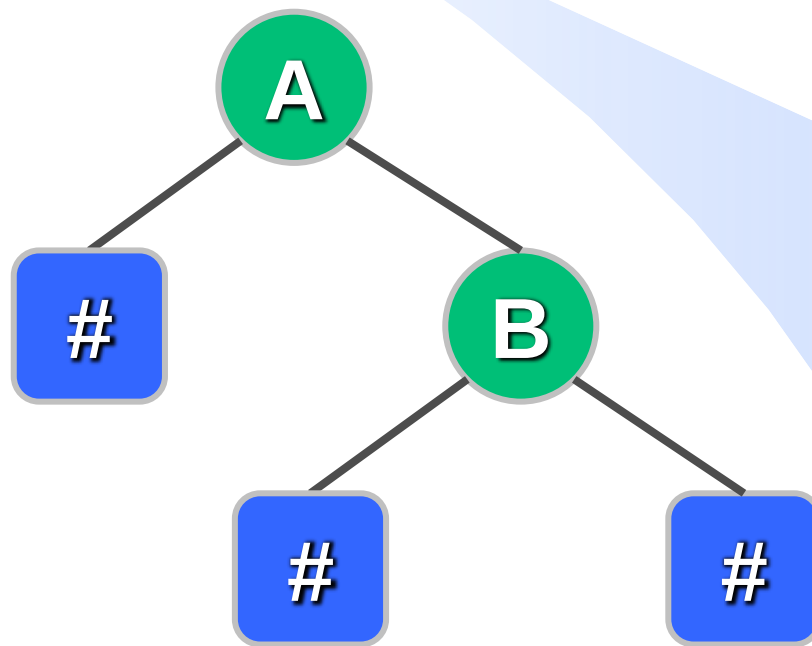
带空指针信息的先根序列——增强先根序列

A

下面两棵不同的二叉树有相同的先根序列AB。在先根序列中加入特殊符号#以表示空指针位置后，先根序列则不同。



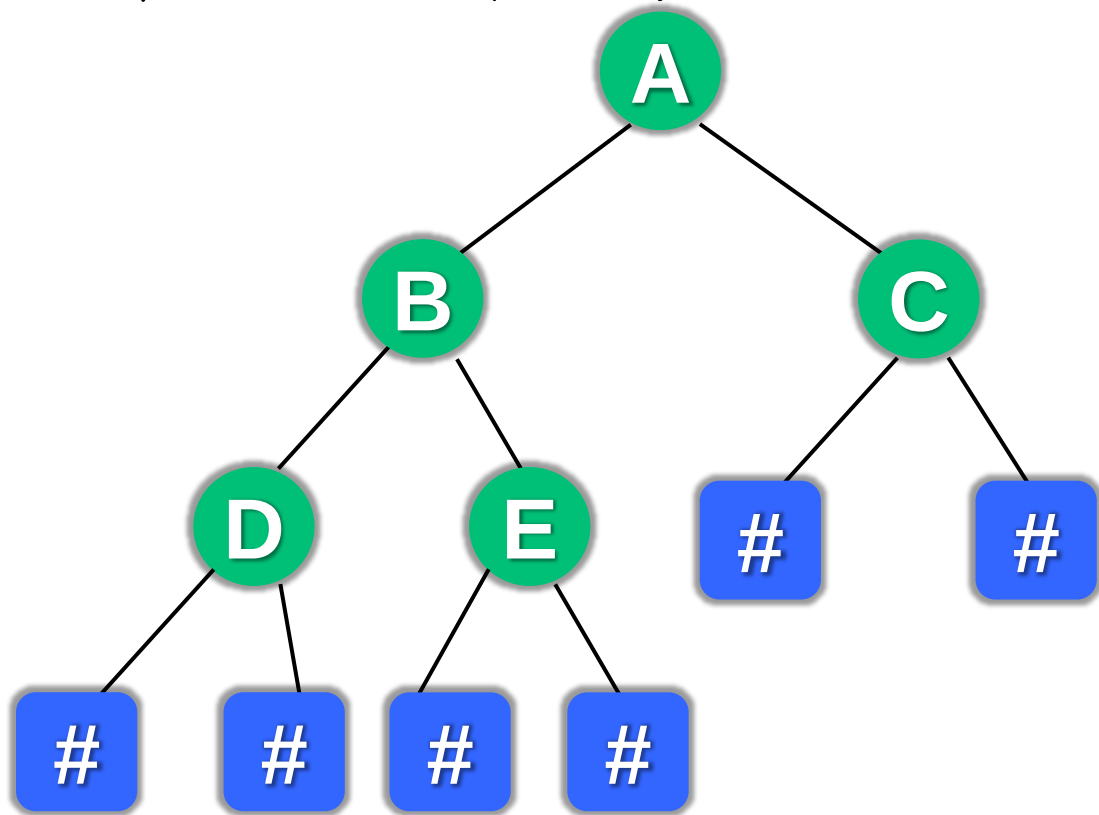
AB###



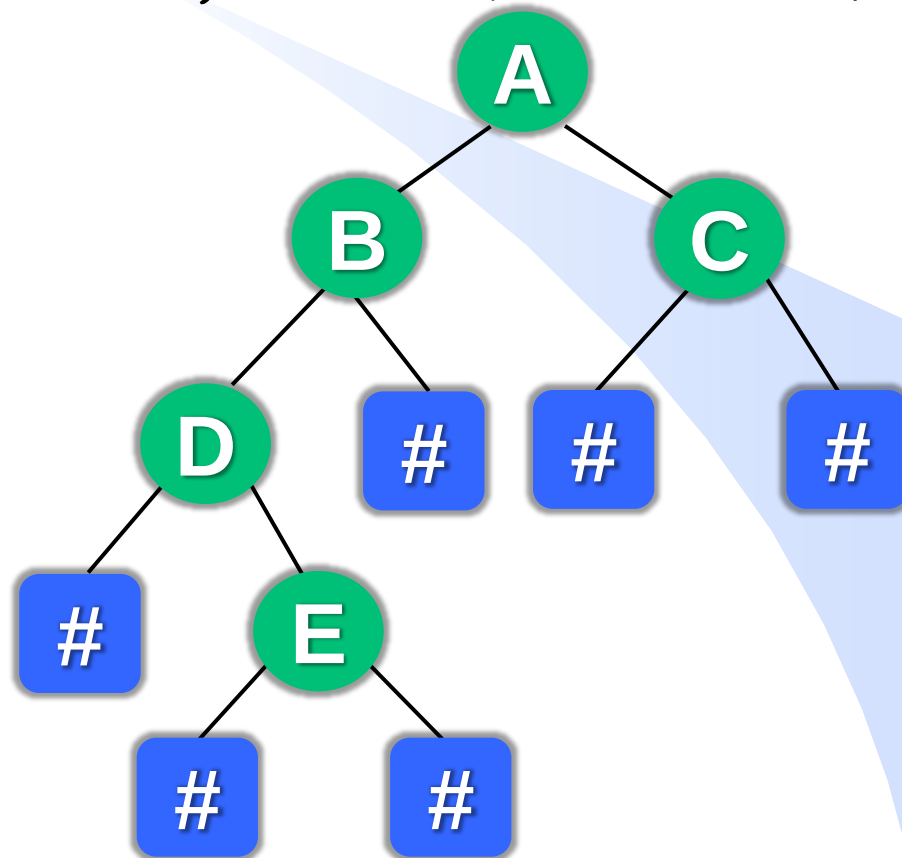
A#B##

带空指针信息的先根序列——增强先根序列

下面两棵不同的二叉树有相同的先根序列 $ABDEC$ 。在先根序列中加入特殊符号 $\#$ 以表示空指针位置后，先根序列则不同。



$ABD\#\#E\#\#C\#\#$



$ABD\#E\#\#\#C\#\#$

根据增强先根序列创建二叉树

根据带空指针信息的先根序列创建二叉树，其中空指针信息用#表示，如ABD##E##C## 【清华大学考研复试机试】

```
int k=0;
```

```
TreeNode* CreateBinTree(char preorder[]){
    char ch=preorder[k++];
    if(ch=='#') return NULL;
    TreeNode *root = new TreeNode;
    root->data=ch;
    root->left=CreateBinTree(preorder);
    root->right=CreateBinTree(preorder);
    return root;
```

```
} char s[N]; scanf("%s",s);
  TreeNode *root=CreateBinTree(s);
```

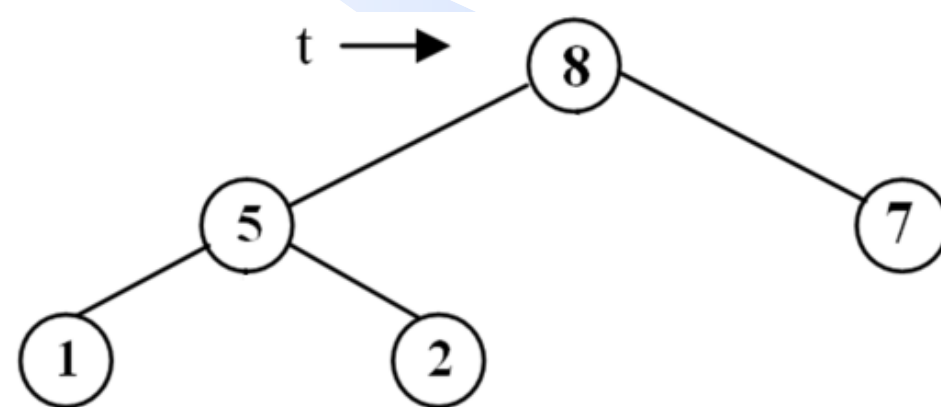
```
struct TreeNode{
    char data;
    TreeNode *left;
    TreeNode *right;
};
```

当读入#时，将其初始化为一个空指针，否则生成一个新结点

上机实验常见创建二叉树形式

已知一棵非空二叉树结点的数据域为不等于0的整数，输入为一组用空格间隔的整数，表示带空指针信息的二叉树先根序列，其中空指针信息用0表示，如8 5 1 0 0 2 0 0 7 0 0表示如下二叉树。

```
TreeNode* CreateBinTree(){  
    int k;  
    scanf("%d", &k);  
    if(k==0) return NULL;  
    TreeNode *root = new TreeNode;  
    root->data = k;  
    root->left = CreateBinTree();  
    root->right = CreateBinTree();  
    return root;  
}
```

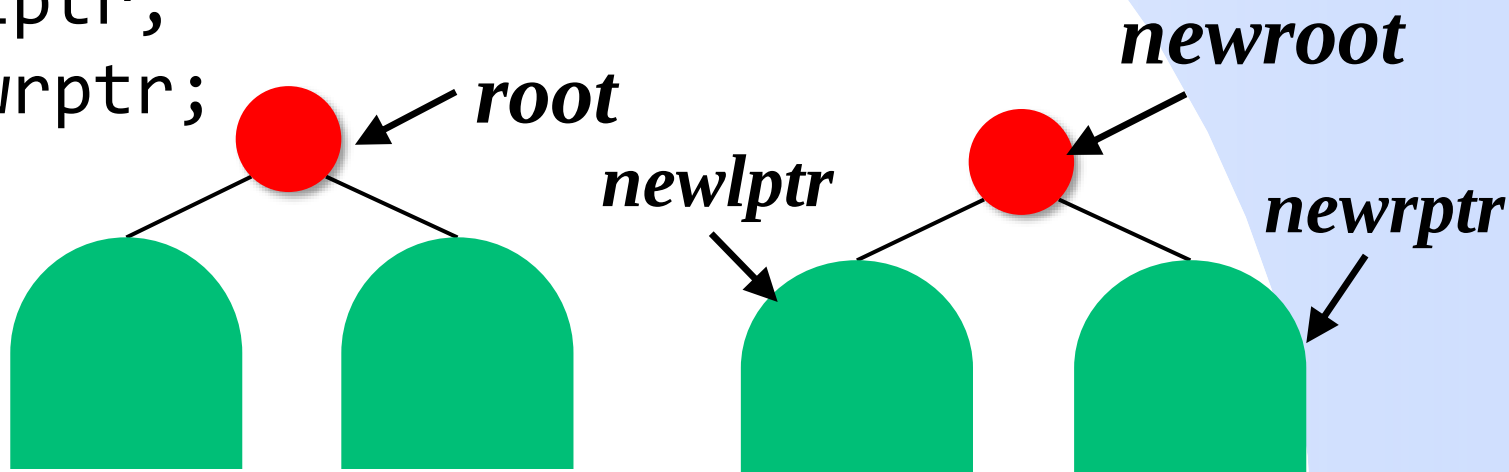


```
struct TreeNode{  
    int data;  
    TreeNode *left;  
    TreeNode *right;  
};
```


复制二叉树

先复制子树，再复制根结点，将父结点与子结点连接起来。

```
TreeNode* CopyTree(TreeNode* root){ //复制以root为根的二叉树
    if(root==NULL) return NULL;
    TreeNode* newlptr=CopyTree(root->left); //复制左子树
    TreeNode* newrptr=CopyTree(root->right); //复制右子树
    TreeNode* newroot=new TreeNode; //生成根结点
    newroot->data = root->data;
    newroot->left = newlptr;
    newroot->right = newrptr;
    return newroot;
}
```



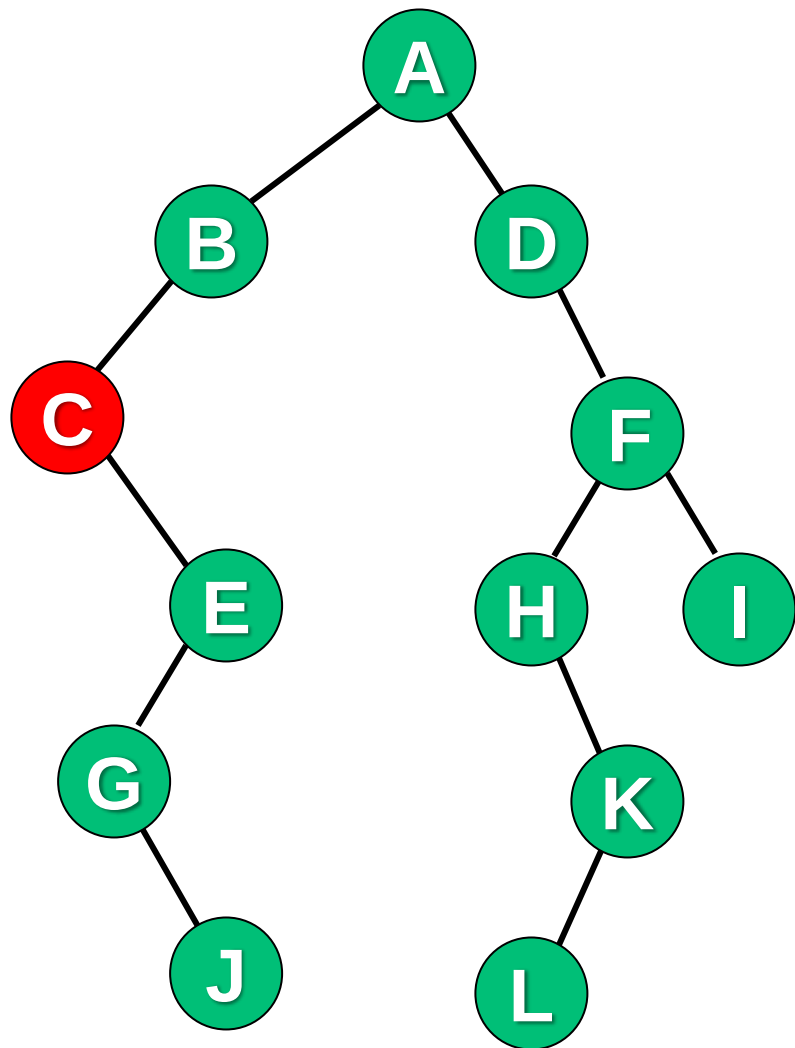
二叉树中、先、后根序列的首末结点

- 编写算法，找出二叉树**中根序列**的**第一个结点**，要求不使用递归、不使用栈。
- 编写算法，找出二叉树**先根序列**的**最后一个结点**，要求不使用递归、不使用栈。【上海交通大学、吉林大学考研题】
- 编写算法，找出二叉树**后根序列**的**第一个结点**，要求不使用递归、不使用栈。【哈尔滨工业大学期末考试题】

二叉树中、先、后根序列的首末结点

	中根序列	先根序列	后根序列
第一个结点			
最后一个结点			

二叉树中根序列的第一个结点



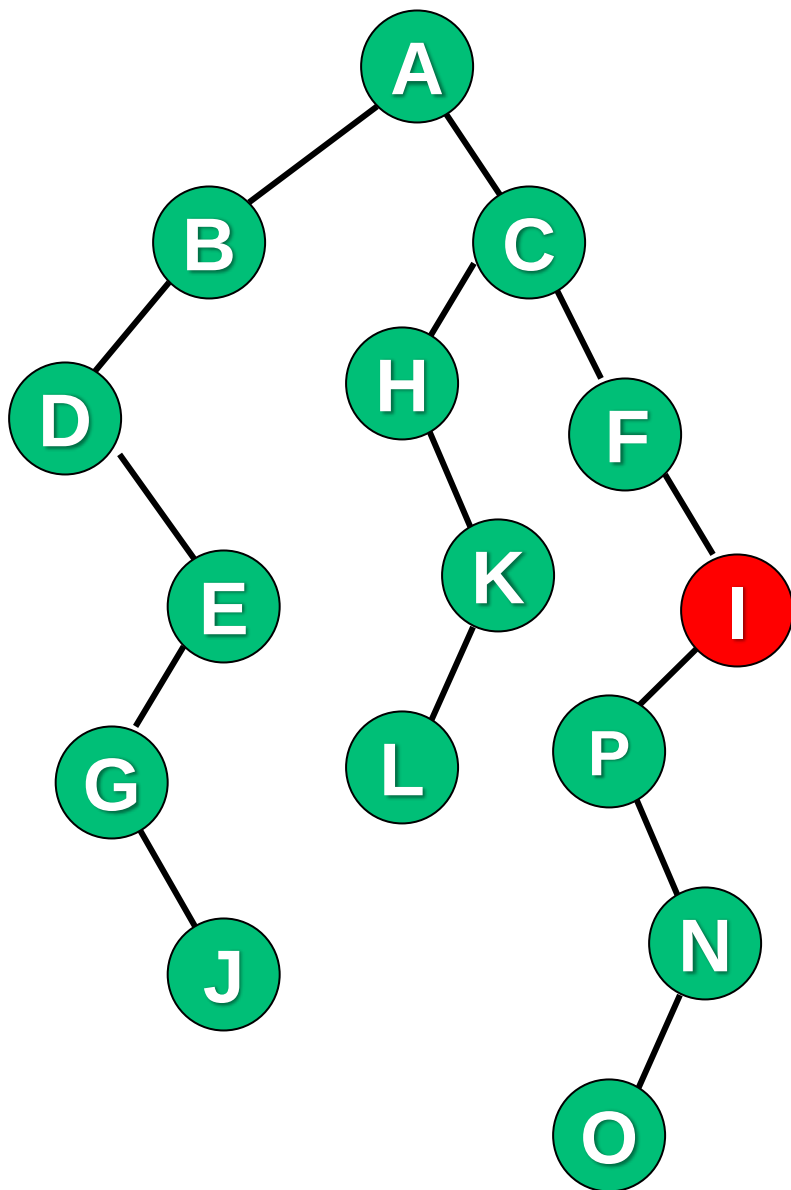
```
if(root==NULL) return NULL;  
TreeNode* p=root;  
while(p->left!=NULL)  
    p=p->left;  
return p;
```

时间复杂度 $O(h)$
 h 为二叉树高度

二叉树中、先、后根序列的首末结点

	中根序列	先根序列	后根序列
第一个结点	<pre>if(root==NULL)return NULL; TreeNode* p=root; while(p->left!=NULL) p=p->left; return p;</pre>		
最后一个结点			

二叉树中根序列的最后一个结点



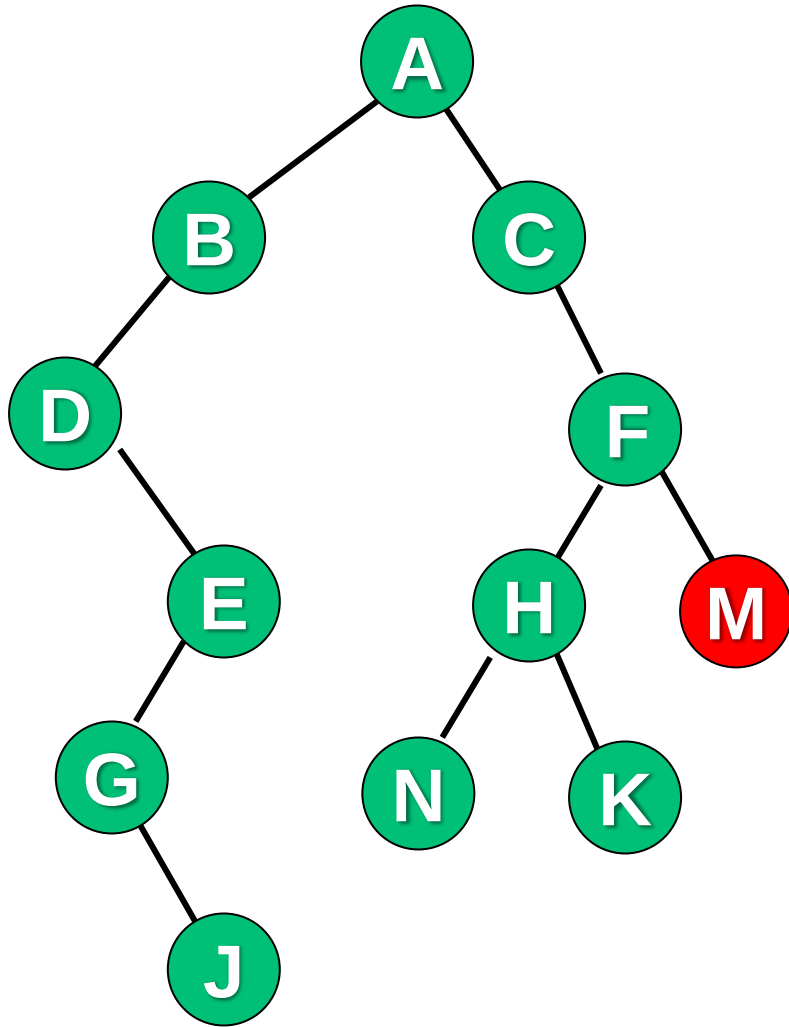
```
if(root==NULL) return NULL;  
TreeNode* p=root;  
while(p->right!=NULL)  
    p=p->right;  
return p;
```

时间复杂度 $O(h)$
 h 为二叉树高度

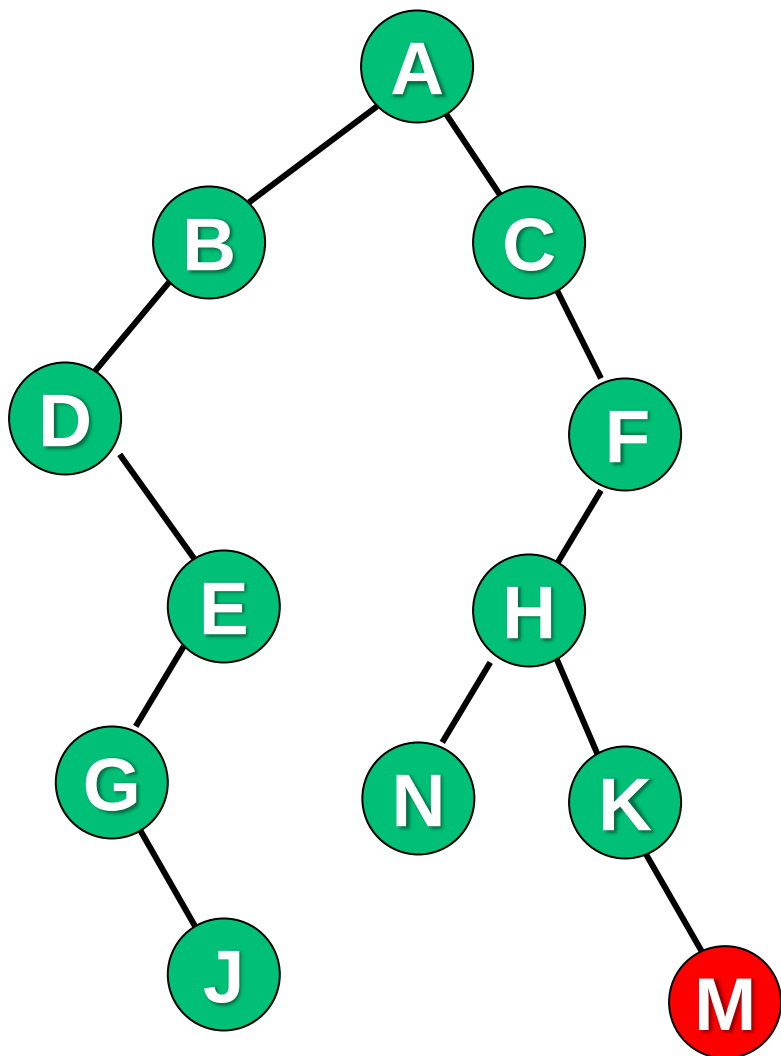
二叉树中、先、后根序列的首末结点

	中根序列	先根序列	后根序列
第一个结点	<pre>if(root==NULL)return NULL; TreeNode* p=root; while(p->left!=NULL) p=p->left; return p;</pre>	<pre>return root;</pre>	
最后一个结点	<pre>if(root==NULL)return NULL; TreeNode* p=root; while(p->right!=NULL) p=p->right; return p;</pre>		

二叉树先根序列的最后一个结点

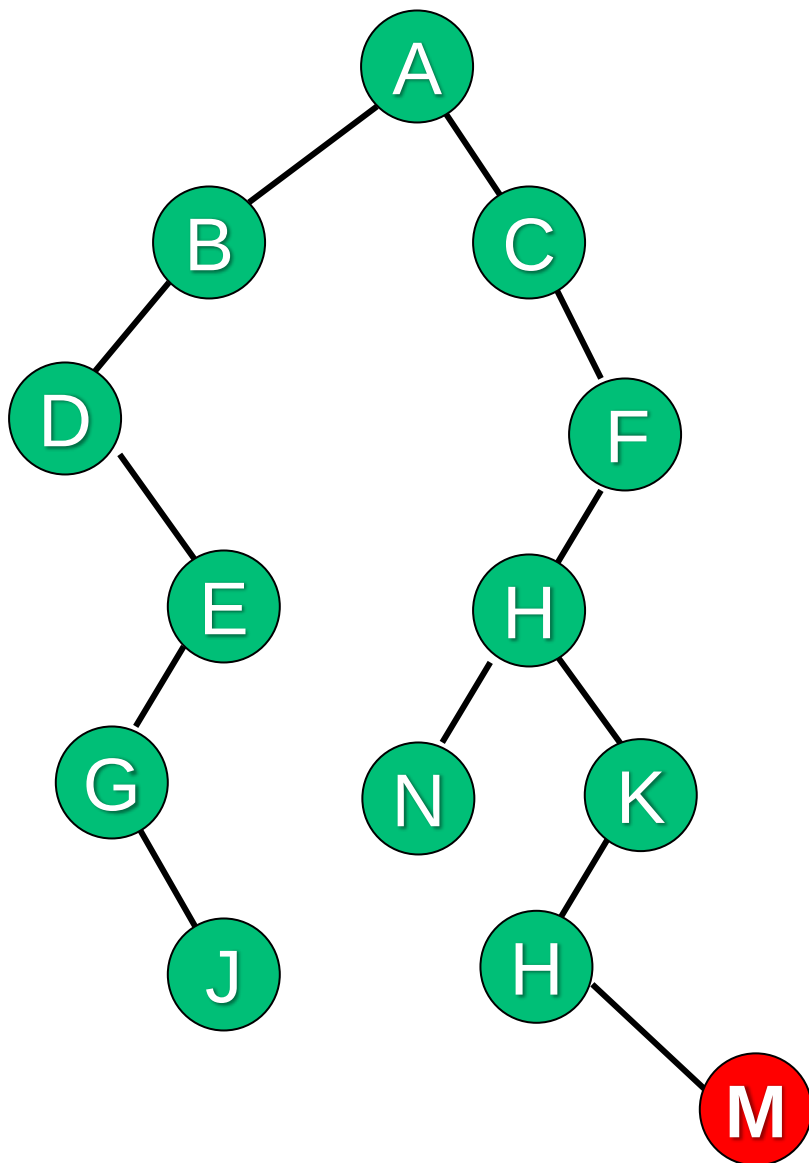


二叉树先根序列的最后一个结点



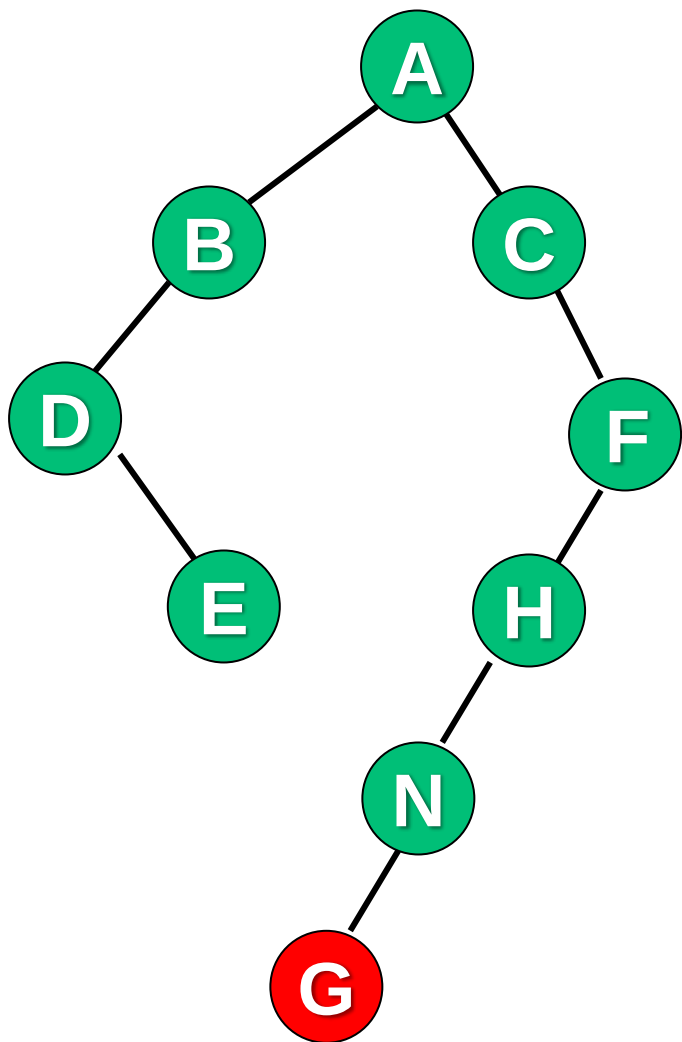
从根结点开始，沿右分支找第一个叶结点，若找不到则在最右边的结点的左子树沿右分支找叶结点，以此类推。

二叉树先根序列的最后一个结点



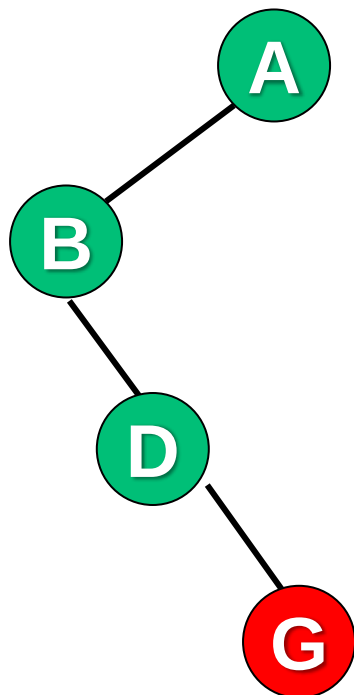
从根结点开始，沿右分支找第一个叶结点，若找不到则在最右边的结点的左子树沿右分支找叶结点，以此类推。

二叉树先根序列的最后一个结点



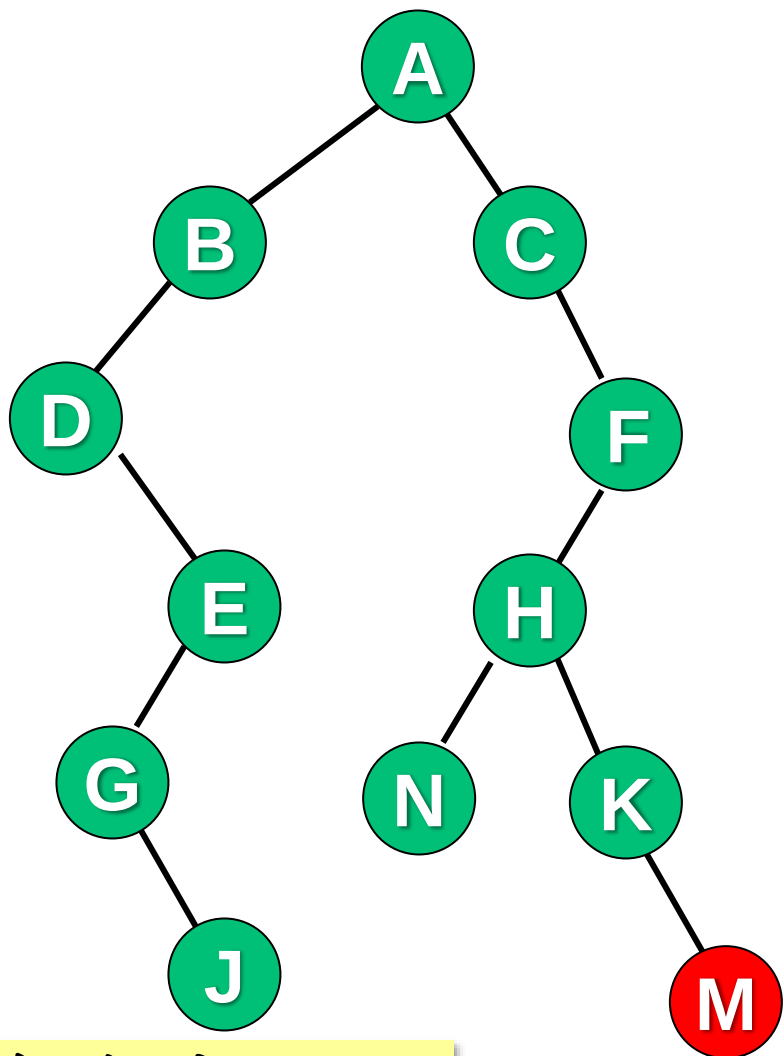
从根结点开始，沿右分支找第一个叶结点，若找不到则在最右边的结点的左子树沿右分支找叶结点，以此类推。

二叉树先根序列的最后一个结点



从根结点开始，沿右分支找第一个叶结点，若找不到则在最右边的结点的左子树沿右分支找叶结点，以此类推。

二叉树先根序列的最后一个结点



从根结点开始，沿右分支找第一个叶结点，若找不到则在最右边的结点的左子树沿右分支找叶结点，以此类推。

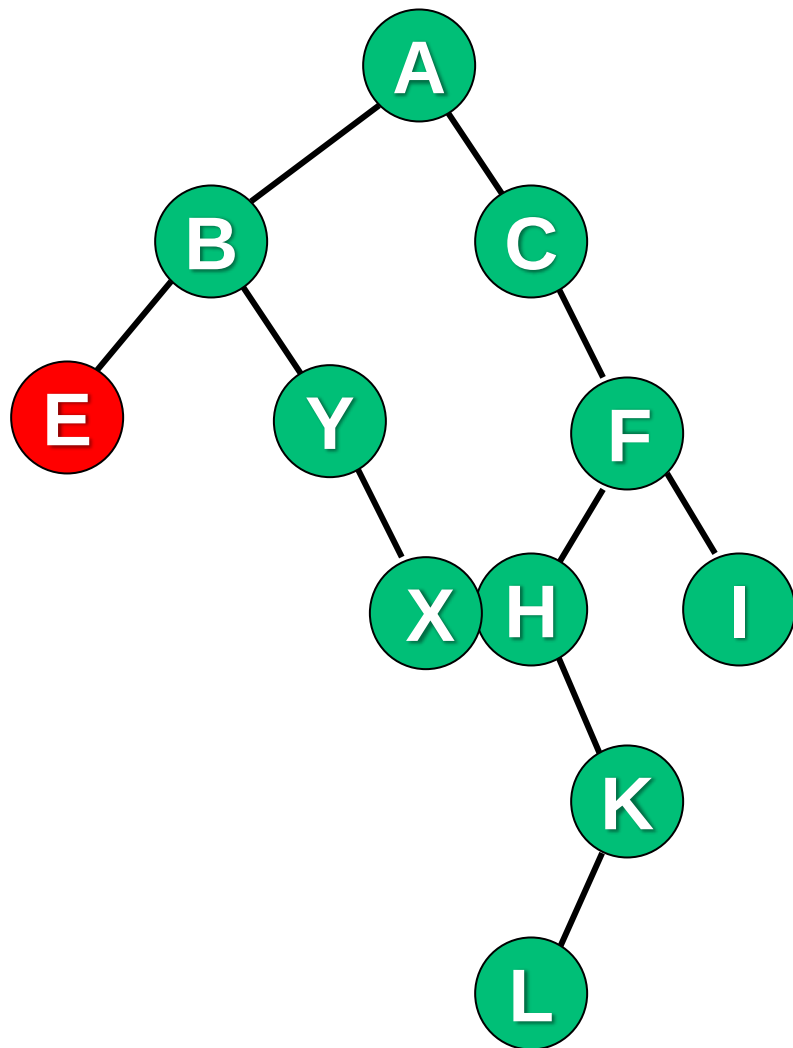
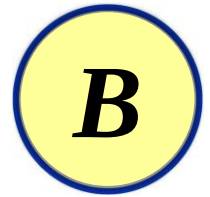
```
if(root==NULL) return NULL;
TreeNode* p=root;
while(p!=NULL){
    if(p->right!=NULL)
        p=p->right;
    else if(p->left!=NULL)
        p=p->left;
    else return p;
}
```

时间复杂度 $O(h)$
 h 为二叉树高度

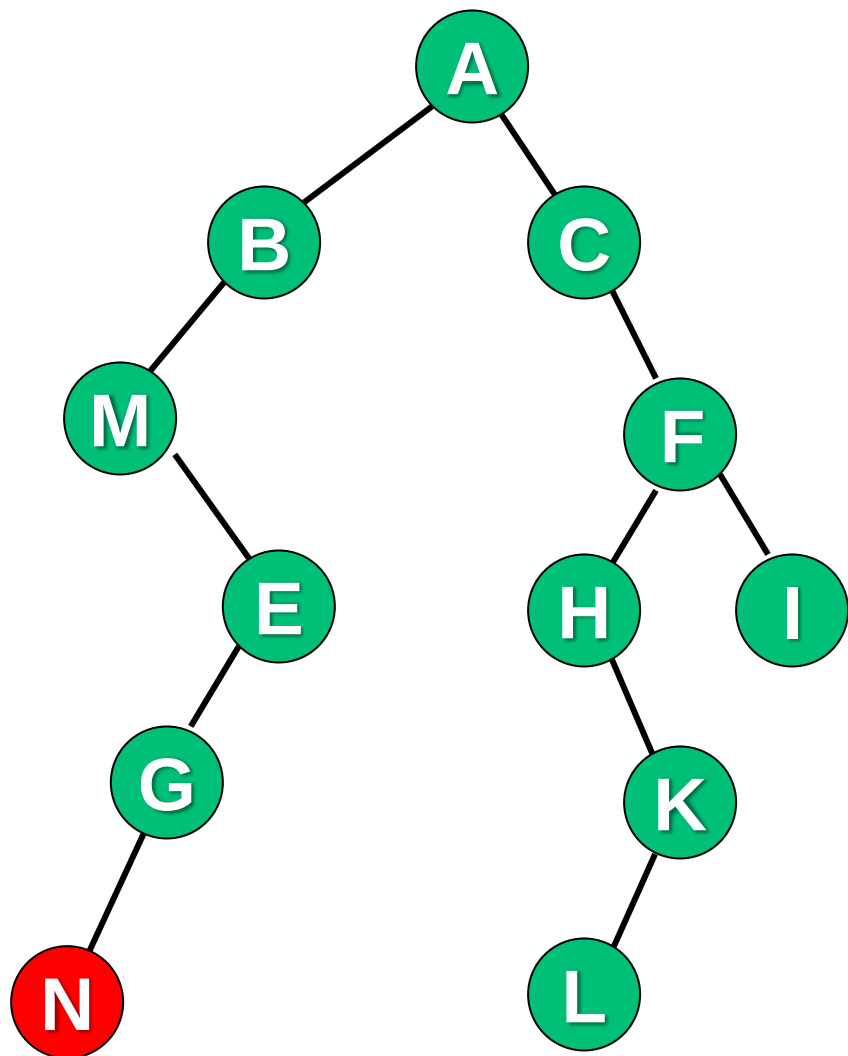
二叉树中、先、后根序列的首末结点

	中根序列	先根序列	后根序列
第一个结点	<pre> if(root==NULL)return NULL; TreeNode* p=root; while(p->left!=NULL) p=p->left; return p; </pre>	<pre> return root; </pre>	
最后一个结点	<pre> if(root==NULL)return NULL; TreeNode* p=root; while(p->right!=NULL) p=p->right; return p; </pre>	<pre> if(root==NULL) return NULL; TreeNode* p=root; while(p!=NULL){ if(p->right!=NULL) p=p->right; else if(p->left!=NULL) p=p->left; else return p; } </pre>	

二叉树后根序列的第一个结点

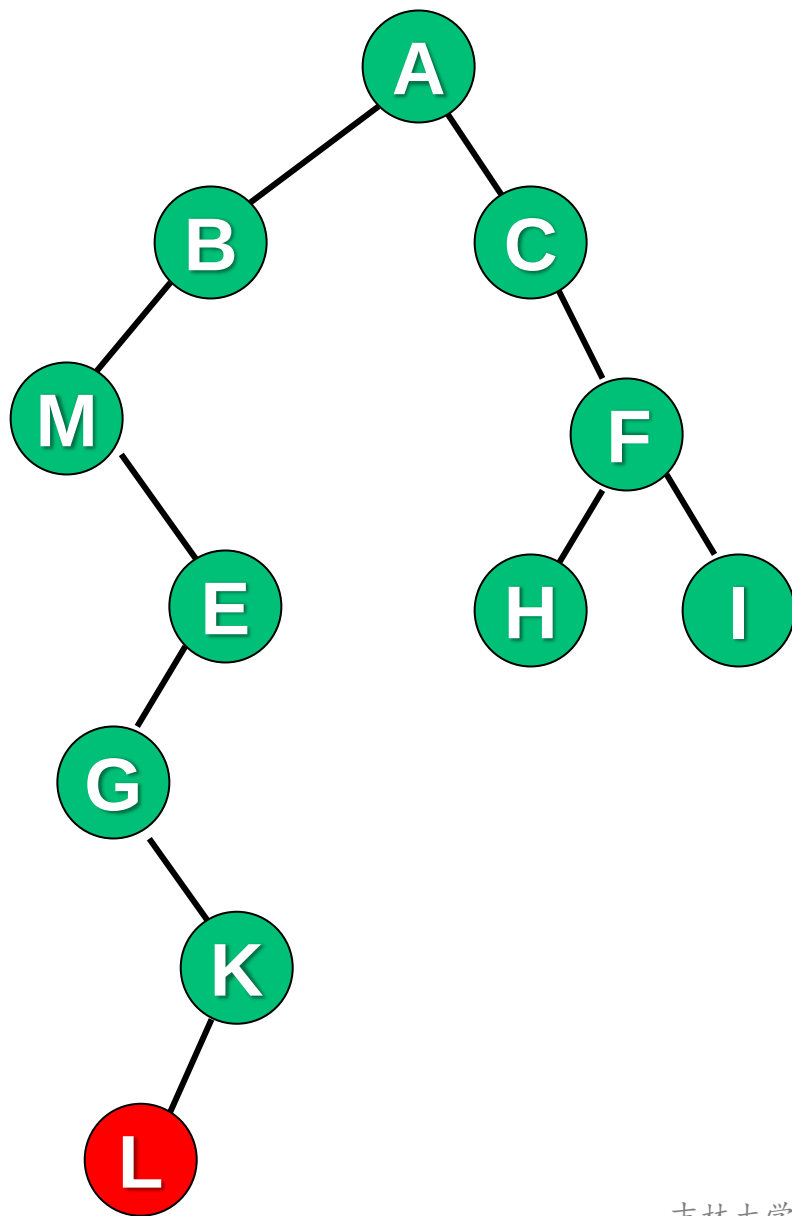


二叉树后根序列的第一个结点



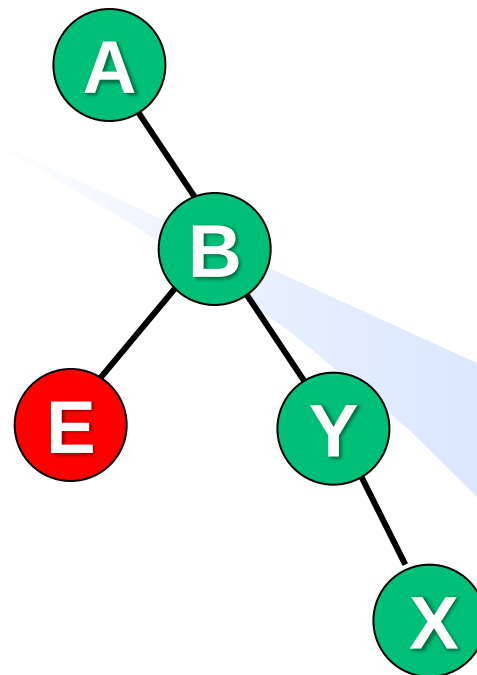
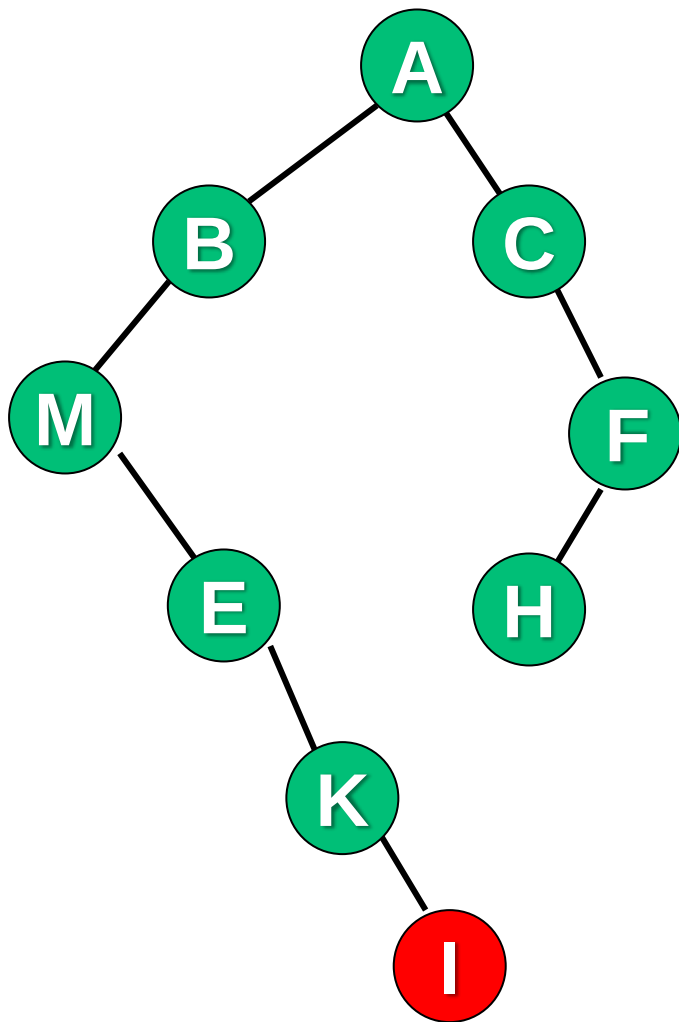
从根结点开始，沿左分支找第一个叶结点，若找不到则在最左边的结点的右子树沿左分支找叶结点，以此类推。

二叉树后根序列的第一个结点



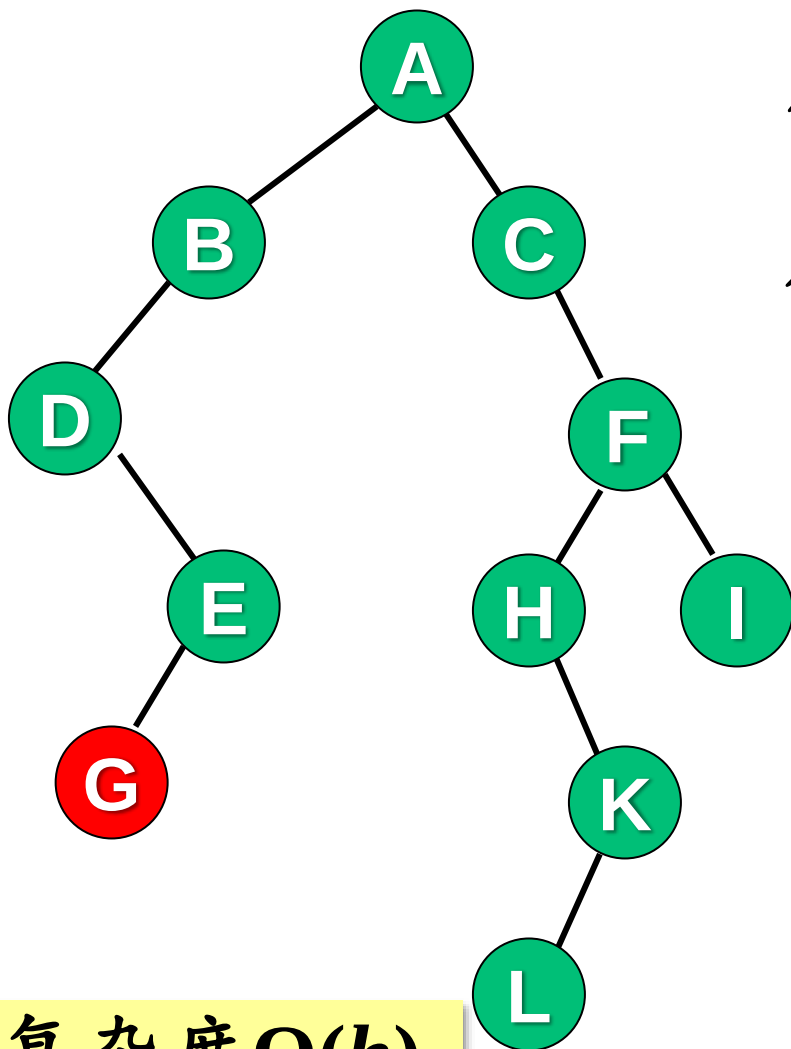
从根结点开始，沿左分支找第一个叶结点，若找不到则在最左边的结点的右子树沿左分支找叶结点，以此类推。

二叉树后根序列的第一个结点



从根结点开始，沿左分支找第一个叶结点，若找不到则在最左边的结点的右子树沿左分支找叶结点，以此类推。

二叉树后根序列的第一个结点



从根结点开始，沿左分支找第一个叶结点，若找不到则在最左边的结点的右子树沿左分支找叶结点，以此类推。

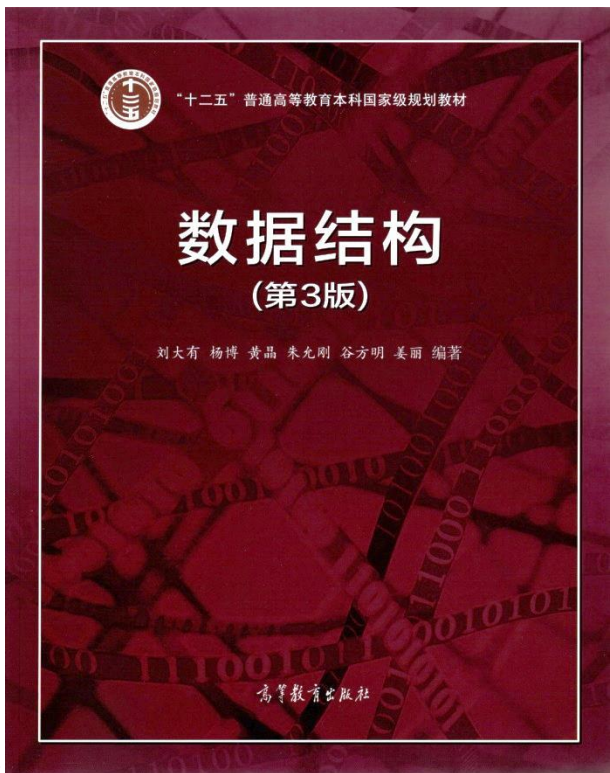
```
if(root==NULL) return NULL;
TreeNode* p=root;
while(p!=NULL){
    if(p->left!=NULL)
        p=p->left;
    else if(p->right!=NULL)
        p=p->right;
    else return p;
}
```

时间复杂度 $O(h)$
 h 为二叉树高度

二叉树中、先、后根序列的首末结点

B

	中根序列	先根序列	后根序列
第一个结点	<pre>if(root==NULL)return NULL; TreeNode* p=root; while(p->left!=NULL) p=p->left; return p;</pre>	<pre>return root;</pre>	<pre>if(root==NULL) return NULL; TreeNode* p=root; while(p!=NULL){ if(p->left!=NULL) p=p->left; else if(p->right!=NULL) p=p->right; else return p; }</pre>
最后一个结点	<pre>if(root==NULL)return NULL; TreeNode* p=root; while(p->right!=NULL) p=p->right; return p;</pre>	<pre>if(root==NULL) return NULL; TreeNode* p=root; while(p!=NULL){ if(p->right!=NULL) p=p->right; else if(p->left!=NULL) p=p->left; else return p; }</pre>	<pre>return root;</pre>



二叉树的存储和操作

- 二叉树的存储结构
- 二叉树的遍历
- 二叉树的其他基本操作
- **二叉树的序列化/反序列化**
- 深度遍历的非递归算法

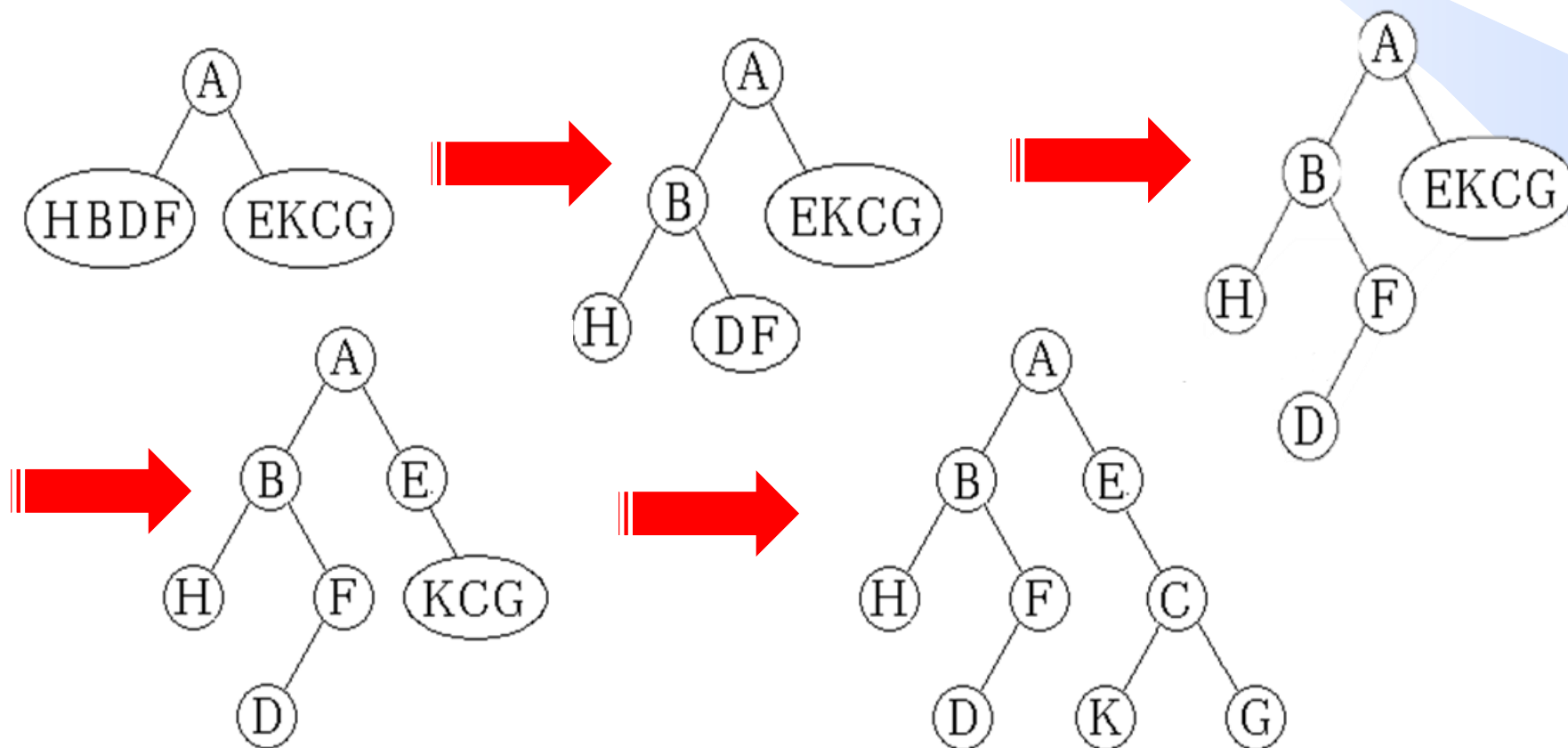
数据之法
结构之美
算法之道

二叉树的重建

由先根序列和中根序列可否唯一确定一棵二叉树？

[例] 先根序列 **A B H F D E C K G**
中根序列 **H B D F A E K C G**

通过先根序列确定子树的根
通过中根序列确定左右子树

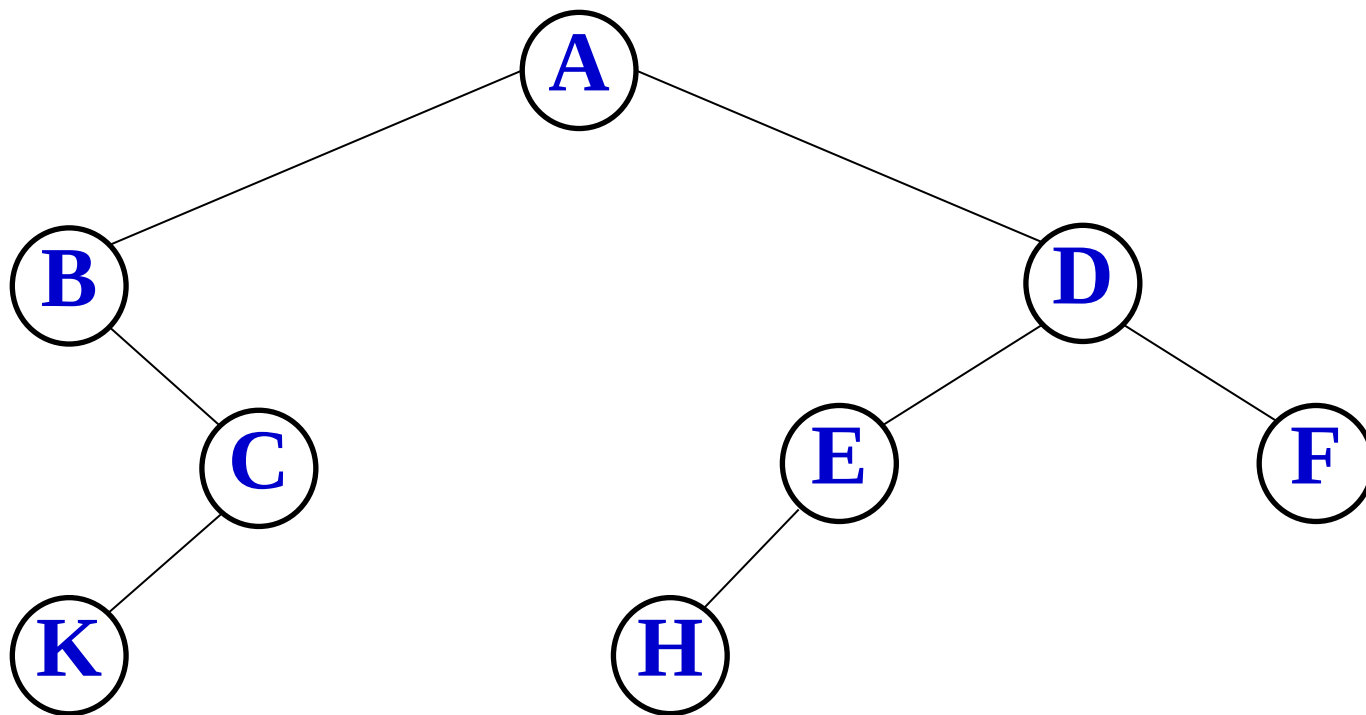


课下练习

由以下**先根序列**和**中根序列**确定一棵二叉树

先根序列 **A** B C K D E H F

中根序列 B K C **A** H E D F



课下练习

已知一棵二叉树的先序和中序遍历序列如下：

先序序列：A B C D E F G H I J

中序序列：C B A E F D I H J G

则其后序遍历序列为_____【阿里笔试题】

- A. C B D E A G I H J F
- B. C B D A E G I H J F
- C. C E D B I J H G F A
- D. C E D B I H J G F A
- E. C B F E I J H G D A
- F. C B F E I H J G D A

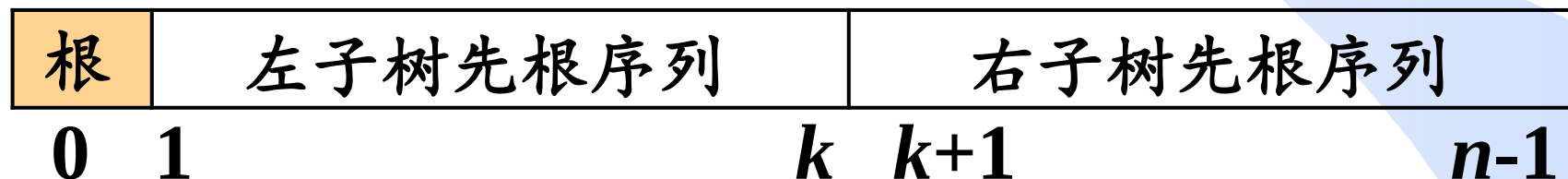
二叉树的重建——编程实现

给定二叉树的先根序列和中根序列，编写程序构建二叉树。

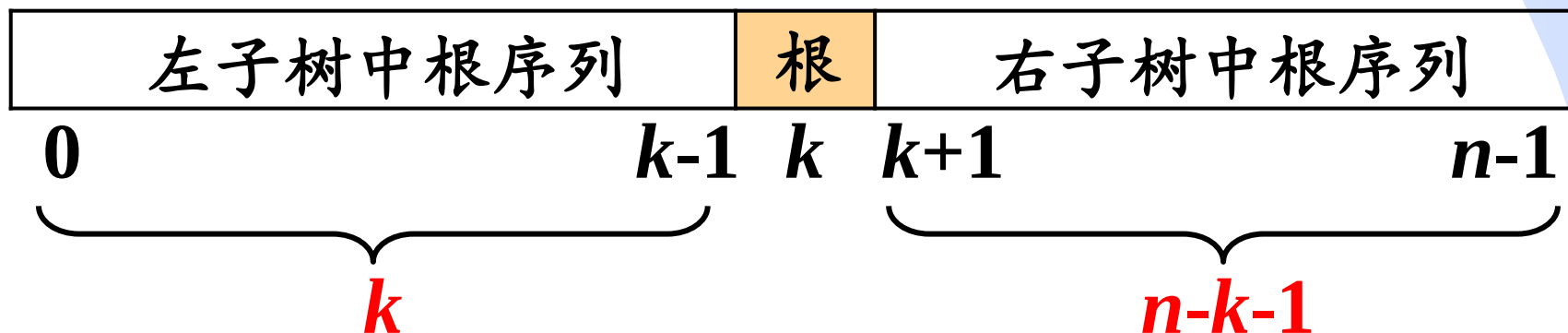
【大厂面试题、华中科技大学考研复试机试题、吉林大学保研复试机试题 [OpenJudgeP0570](#)、[LeetCode105](#)】

```
TreeNode* buildTree(char *preorder, char *inorder, int n)
```

先根序列



中根序列



二叉树的重建——编程实现

```
TreeNode* buildTree(char *preorder, char *inorder, int n) {  
    if (n <= 0) return NULL;  
    TreeNode *root = new TreeNode;  
    root->data = preorder[0]; //先根序列的根  
    int k = findroot(inorder, n, root->data); //在中根序列找根
```

```
int findroot(char *inorder, int size, char val) {  
    for(int i = 0; i<size; i++)  
        if(inorder[i] == val) return i;  
    return -1;  
}
```

二叉树的重建——编程实现

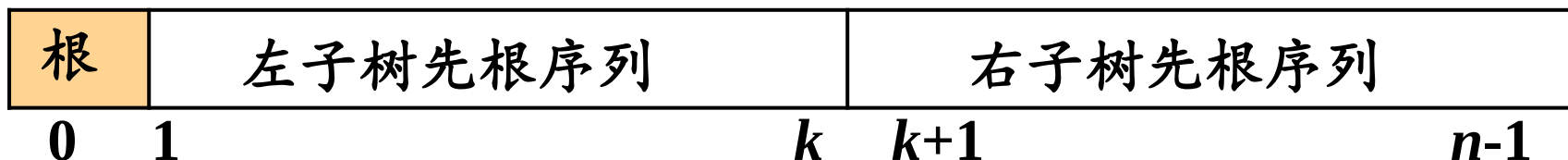
```

TreeNode* buildTree(char *preorder, char *inorder, int n) {
    if (n <= 0) return NULL;
    TreeNode *root = new TreeNode;
    root->data = preorder[0]; //先根序列的根
    int k = findroot(inorder, n, root->data); //在中根序列找根
    root->left=buildTree(&preorder[1], &inorder[0], k);
    root->right=buildTree(&preorder[k+1], &inorder[k+1], n-k-1);
    return root;
}

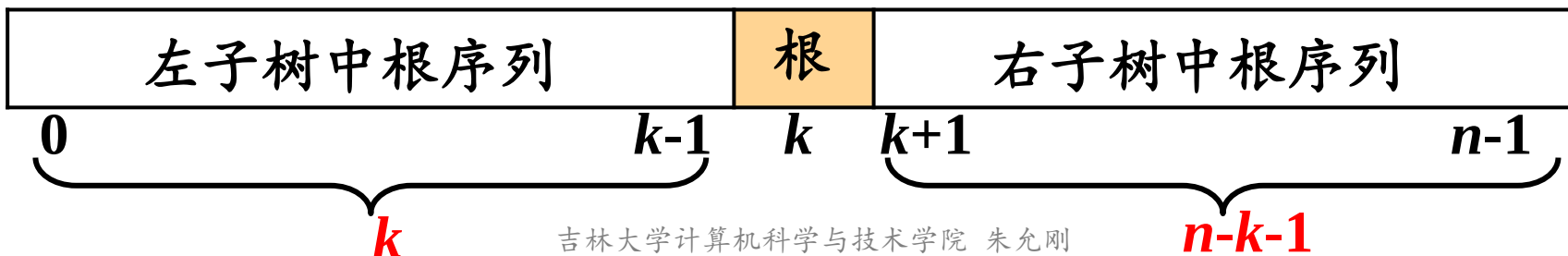
```

最好/平均时间复杂度 $O(n\log n)$
最坏时间复杂度 $O(n^2)$

先根序列



中根序列

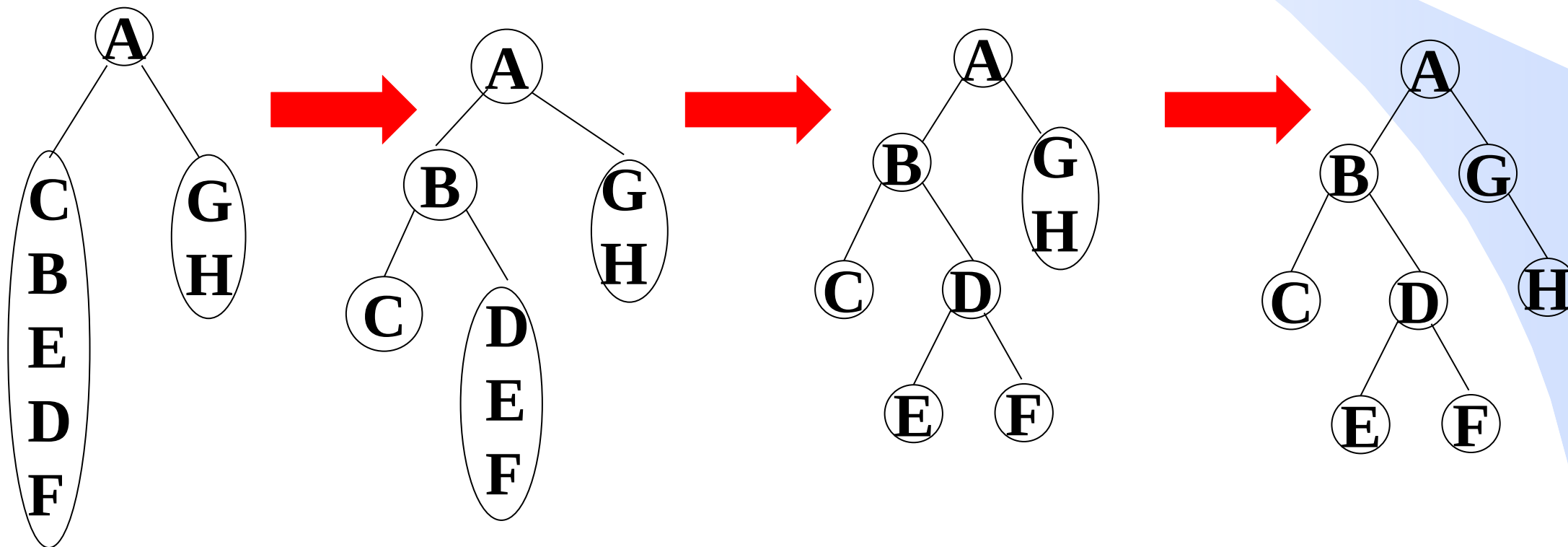


二叉树的重建

由后根序列和中根序列是否可以唯一地确定一棵二叉树？

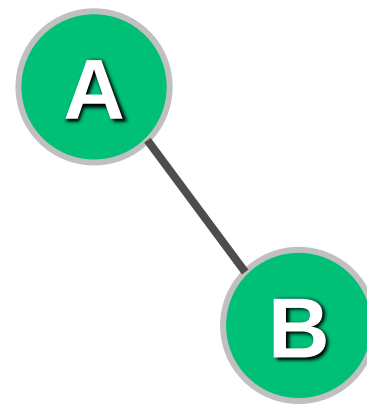
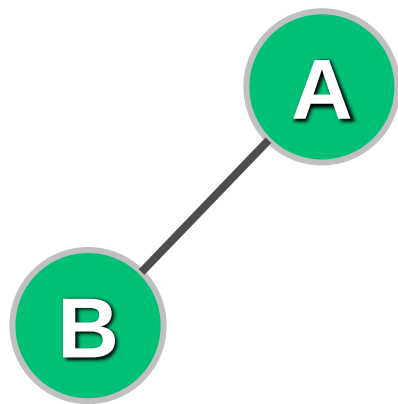
后根序列 C E F D B H G A

中根序列 C B E D F A G H



➤ 由先根序列和后根序列是否可以唯一地确定一棵二叉树？

➤ 先根序列：A B
后根序列：B A



➤ 由层次遍历序列可否唯一确定一棵二叉树？

➤ 层次序列：AB



由先根序列和层次遍历序列可否唯一确定一棵二叉树？

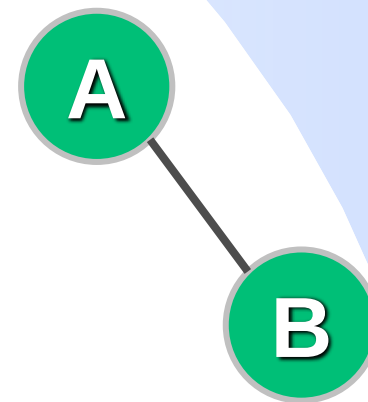
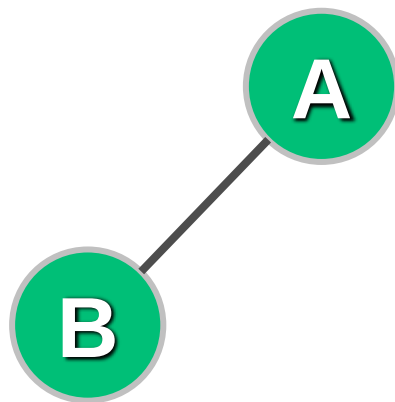
➤ 先根序列：AB

➤ 层次序列：AB

由后根序列和层次遍历序列可否唯一确定一棵二叉树？

➤ 后根序列：BA

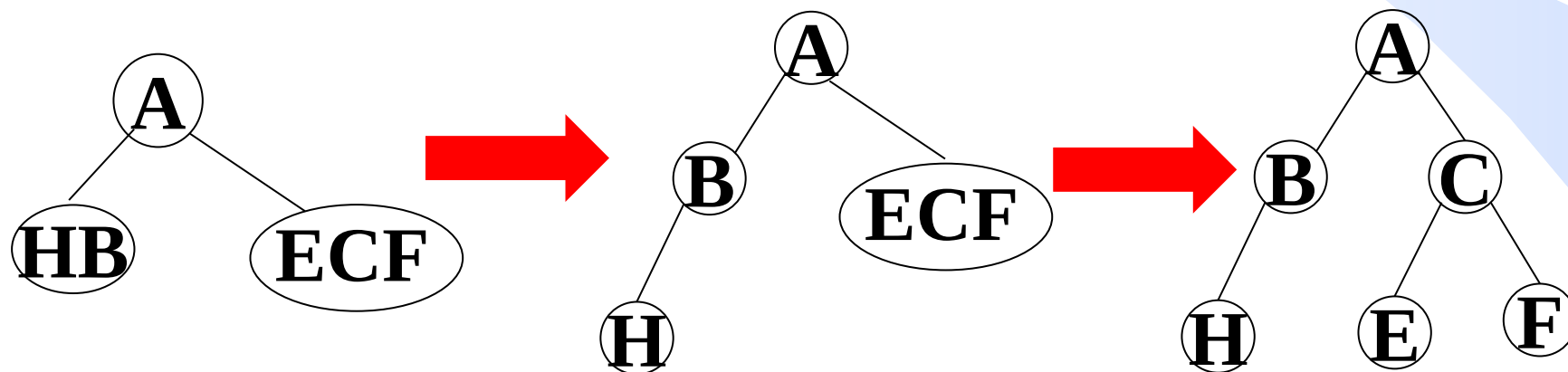
➤ 层次序列：AB



由中根序列和层次遍历序列可否唯一确定一棵二叉树？【清华大学考研题、吉林大学18级期末考试题】

中根序列 H B A E C F

层次序列 A B C H E F

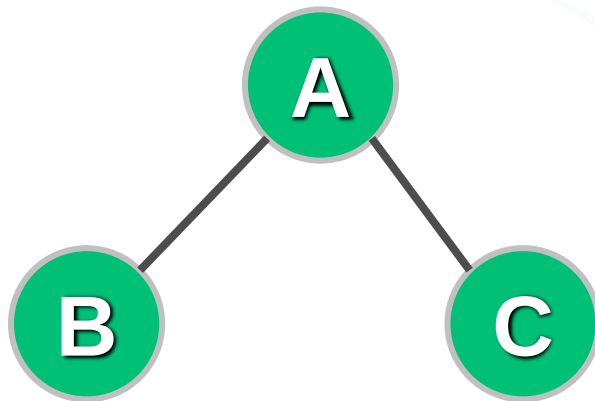


中根序列和任意一种遍历序列都可以唯一地确定一棵二叉树

由先根序列和后根序列可否唯一确定层次遍历序列？【清华大学考研题】

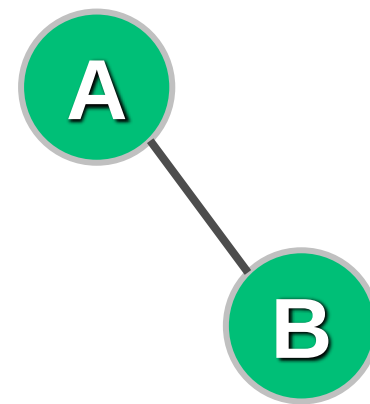
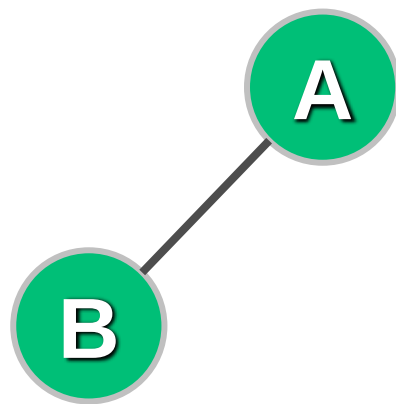
先根序列：A B C

后根序列：B C A



先根序列：A B

后根序列：B A

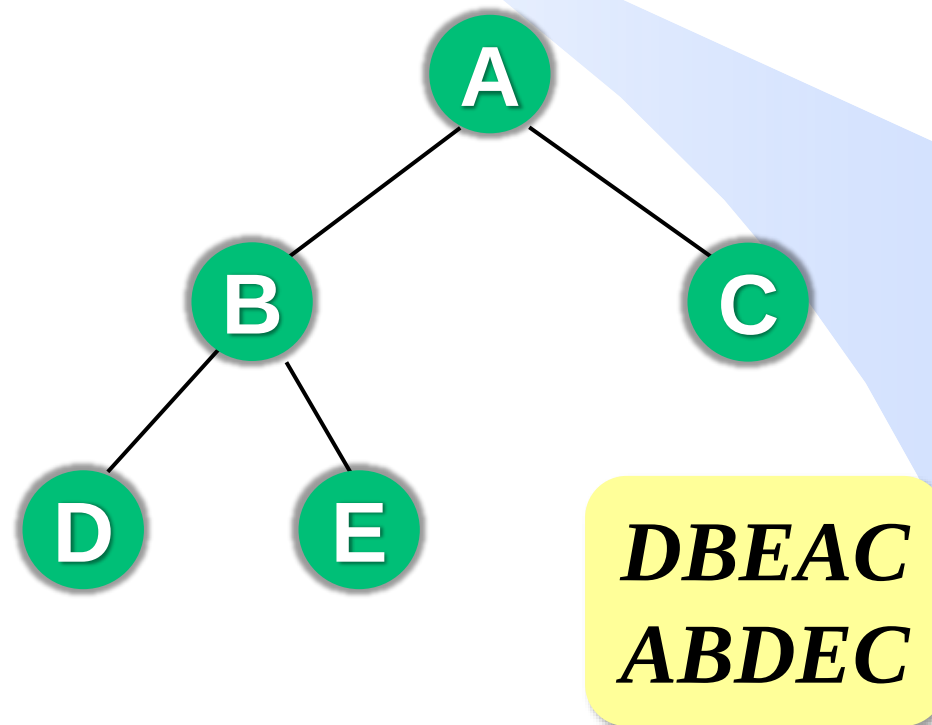


课下思考

- 利用完全二叉树的**先根**序列能否唯一确定一棵完全二叉树？
- 利用完全二叉树的**中根**序列能否唯一确定一棵完全二叉树？
- 利用完全二叉树的**后根**序列能否唯一确定一棵完全二叉树？
- 利用完全二叉树的**层次**序列能否唯一确定一棵完全二叉树？

二叉树的序列化与反序列化

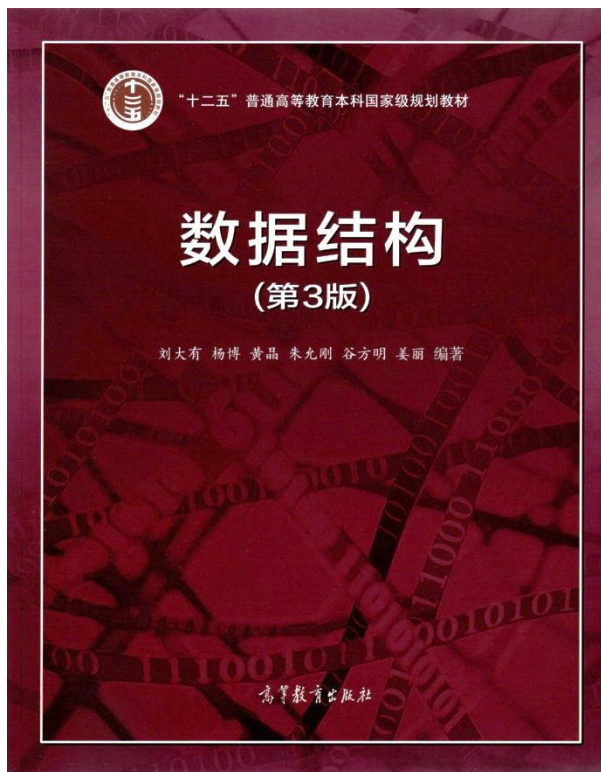
- 序列化：将数据结构转换为一个连续的序列，以便于存储和传输。
- 反序列化：将序列化的数据恢复、重建为原始的数据结构。
- 二叉树的序列化方法：
 - ✓ 中根序列+先根序列
 - ✓ 中根序列+后根序列
 - ✓ 中根序列+层次序列
 - ✓ 增强的先根序列
 - ✓





二叉树的存储和操作

- 二叉树的存储结构
- 二叉树的遍历
- 二叉树的其他基本操作
- 二叉树的重建
- **深度遍历的非递归算法**



数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

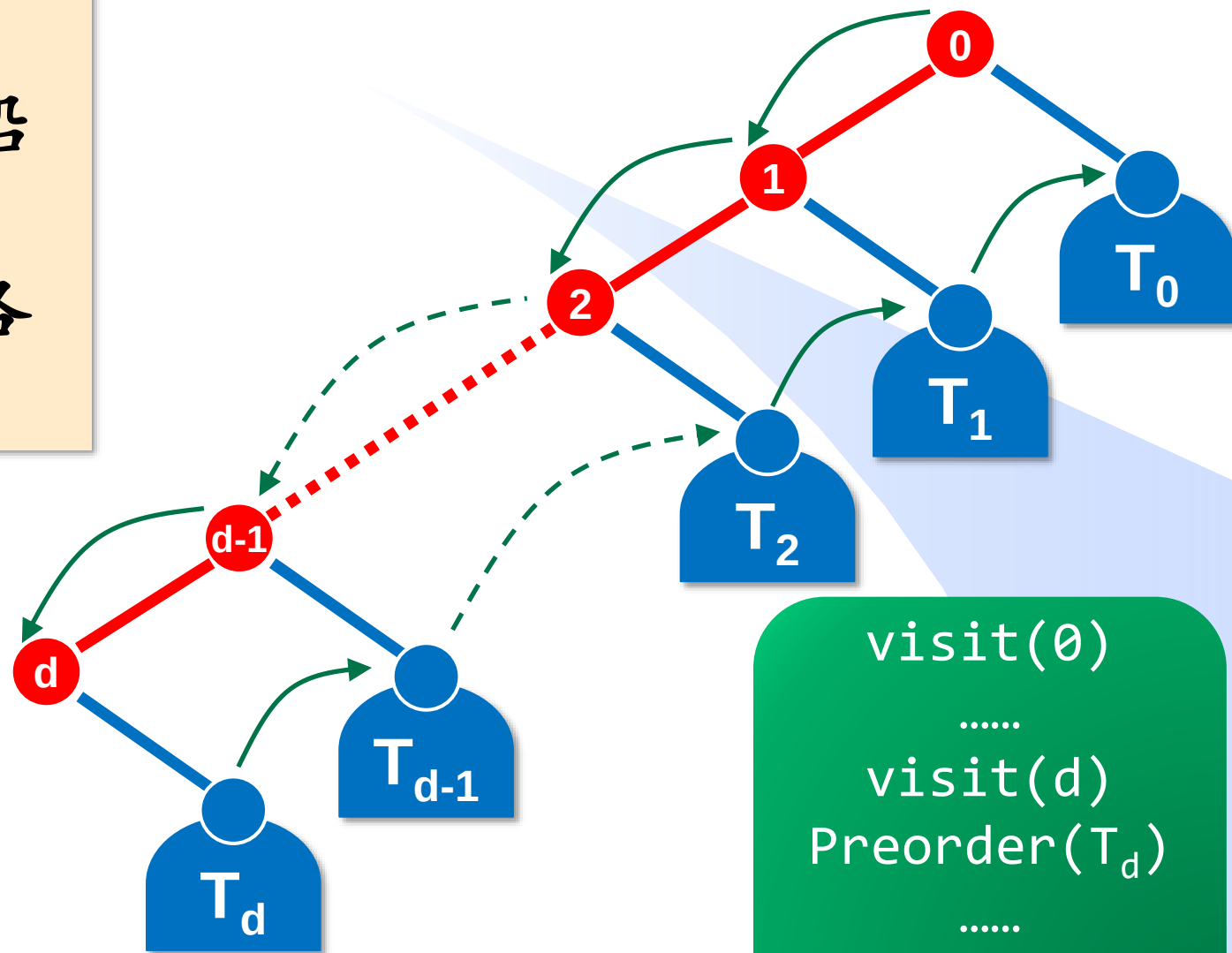
为什么研究二叉树遍历的非递归算法？

- 当二叉树高度很高时，递归算法可能因递归深度过深导致系统栈溢出
- 有利于更深刻理解二叉树遍历的过程

先根遍历的非递归算法

先根遍历过程：

- ① 从根结点开始 **自上而下** 沿着 **左侧分支** 访问结点。
- ② 自下而上依次访问沿途各结点的右子树。



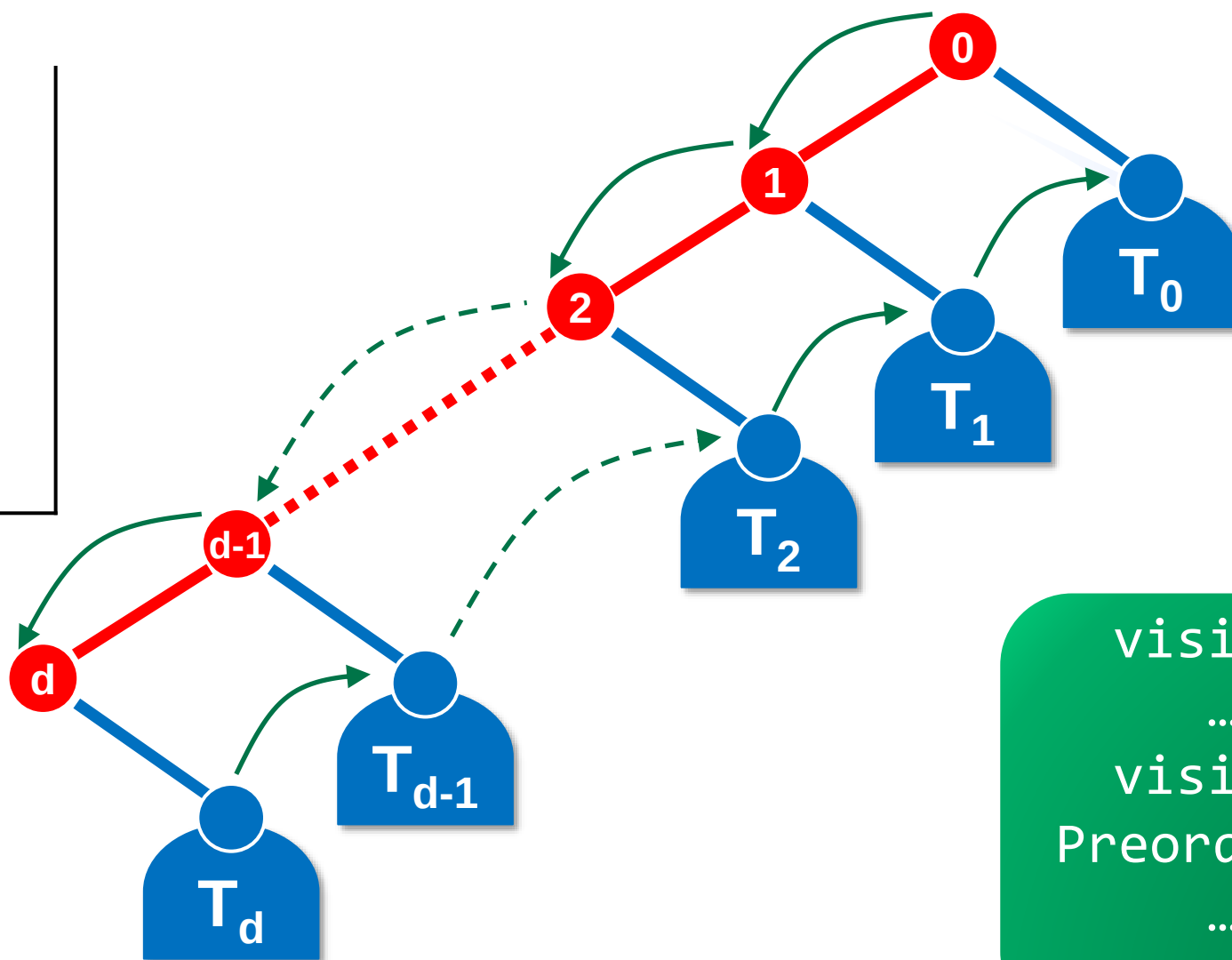
不同右子树的遍历：

- 相互独立
- 自成一个子任务

```
visit(0)
.....
visit(d)
Preorder( $T_d$ )
.....
Preorder( $T_0$ )
```


先根遍历的非递归算法

策略：从根结点开始自上而下沿着左分支访问结点，并把该结点压栈。之后再自下而上弹栈，访问沿途结点的右子树。

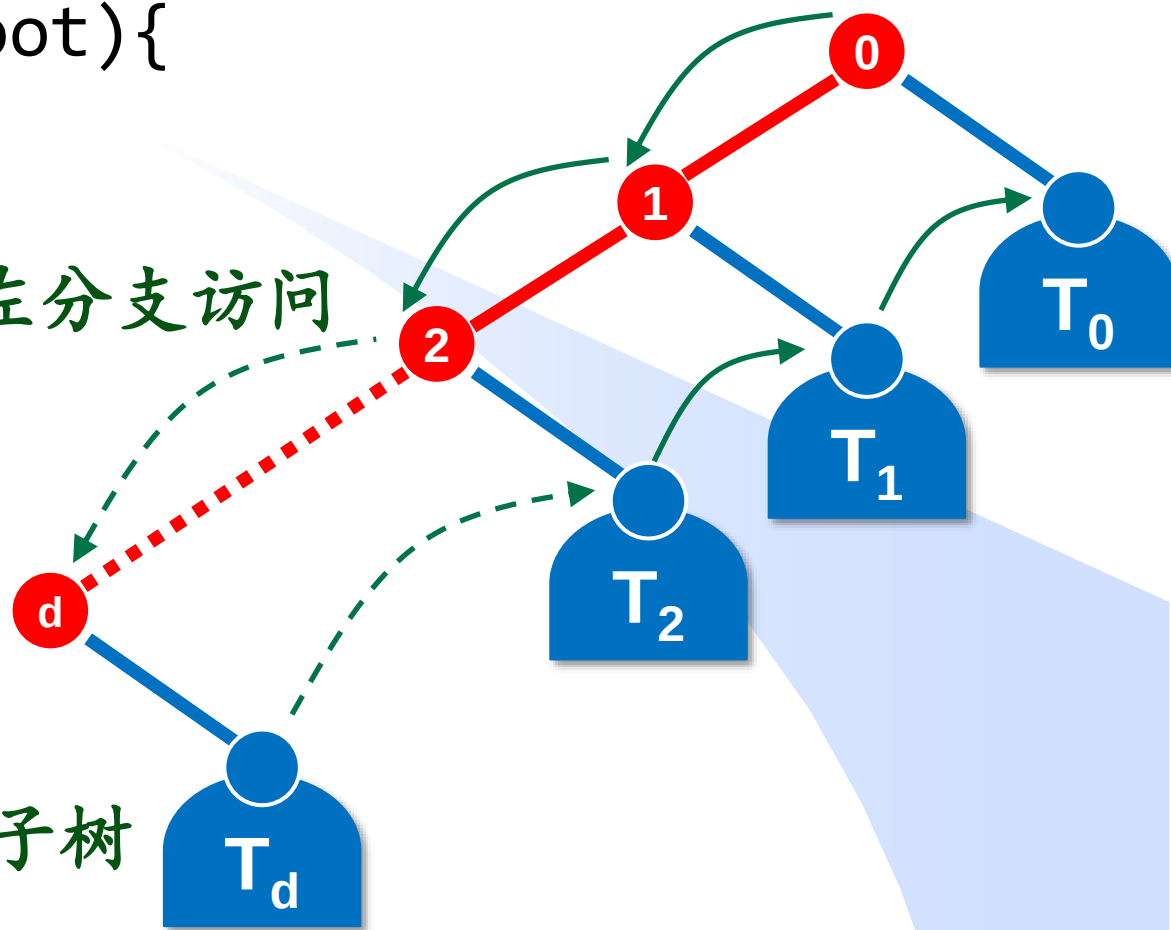


```
visit(0)
.....
visit(d)
Preorder( $T_d$ )
.....
Preorder( $T_0$ )
```


先根遍历的非递归算法

B

```
void PreOrder_NonRec(TreeNode* root){  
    Stack S; TreeNode* p = root;  
    while(true) {  
        while(p!=NULL){ //自上而下沿左分支访问  
            visit(p->data);  
            S.PUSH(p);  
            p=p->left;  
        }  
        if(S.IsEmpty()) return;  
        p=S.POP(); //自下而上访问各右子树  
        p=p->right;  
    }  
}
```



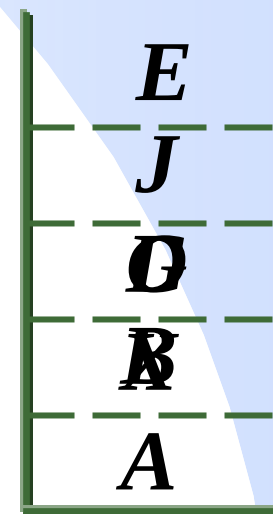
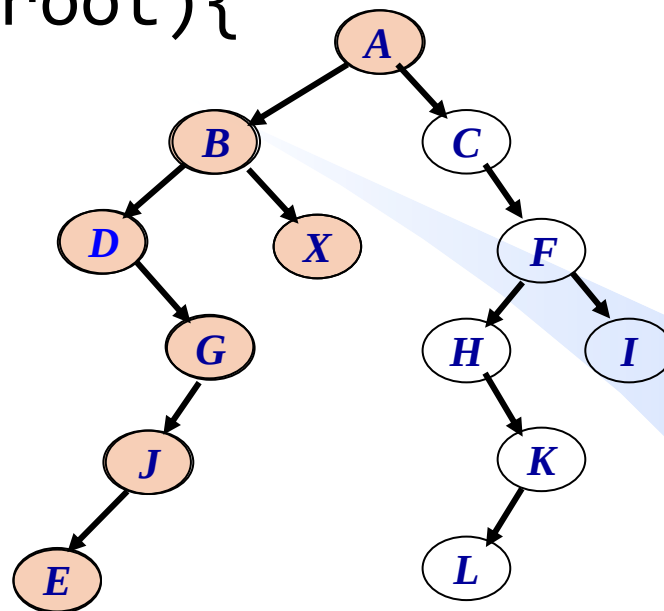
- ✓ 压栈的同时访问
- ✓ 压栈的点若有左孩子则继续把左孩子压栈，若无左孩子则弹栈
- ✓ 弹出的点若有右孩子则把右孩子压栈，若无右孩子则弹栈

先根遍历的非递归算法——运行实例

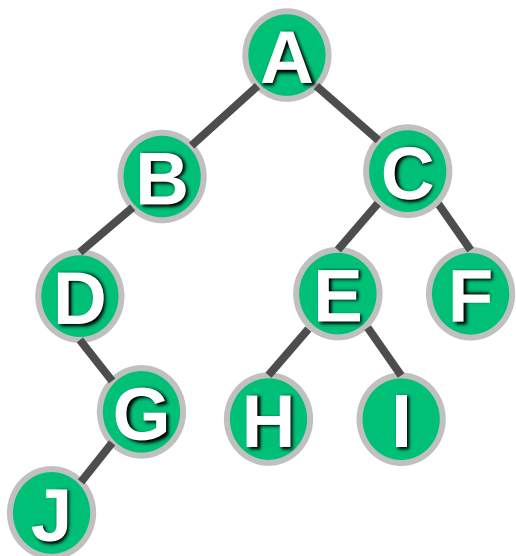
```

void PreOrder_NonRec(TreeNode *root){
    Stack S; TreeNode *p = root;
    while(true) {
        while(p != NULL){
            visit(p->data);
            S.PUSH(p);
            p=p->left;
        }
        if(S.IsEmpty()) return;
        p = S.POP(); //自下而上访问各右子树
        p = p->right;
    }
}

```



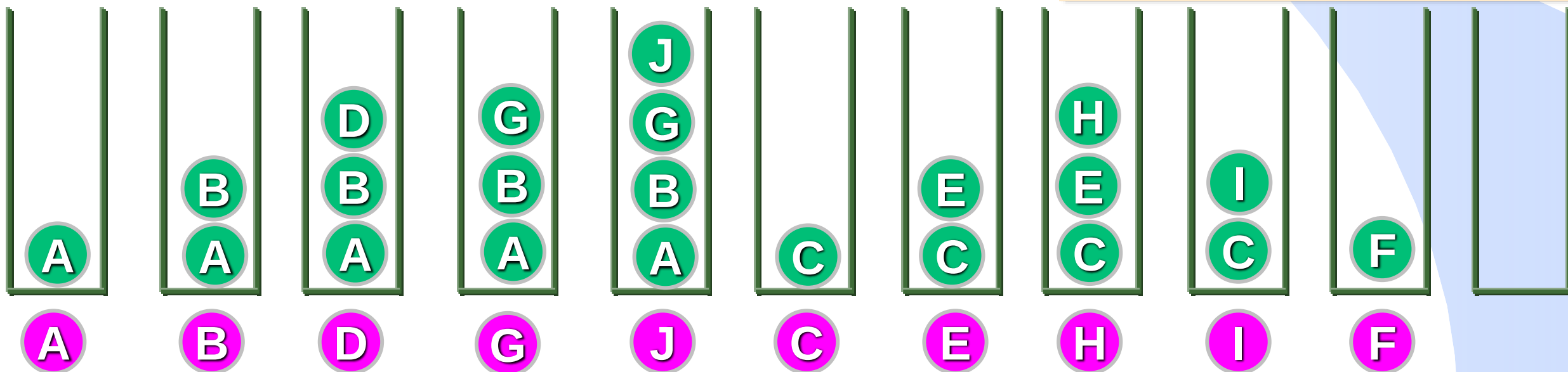
先根遍历的非递归算法——运行实例



```

void PreOrder_NonRec(TreeNode *root){
    Stack<TreeNode*> S; TreeNode *p = root;
    while (true) {
        while(p!=NULL){visit(p->data); S.PUSH(p); p=p->left;}
        if(S.IsEmpty()) return;
        p=S.POP();    p=p->right;
    }
}
  
```

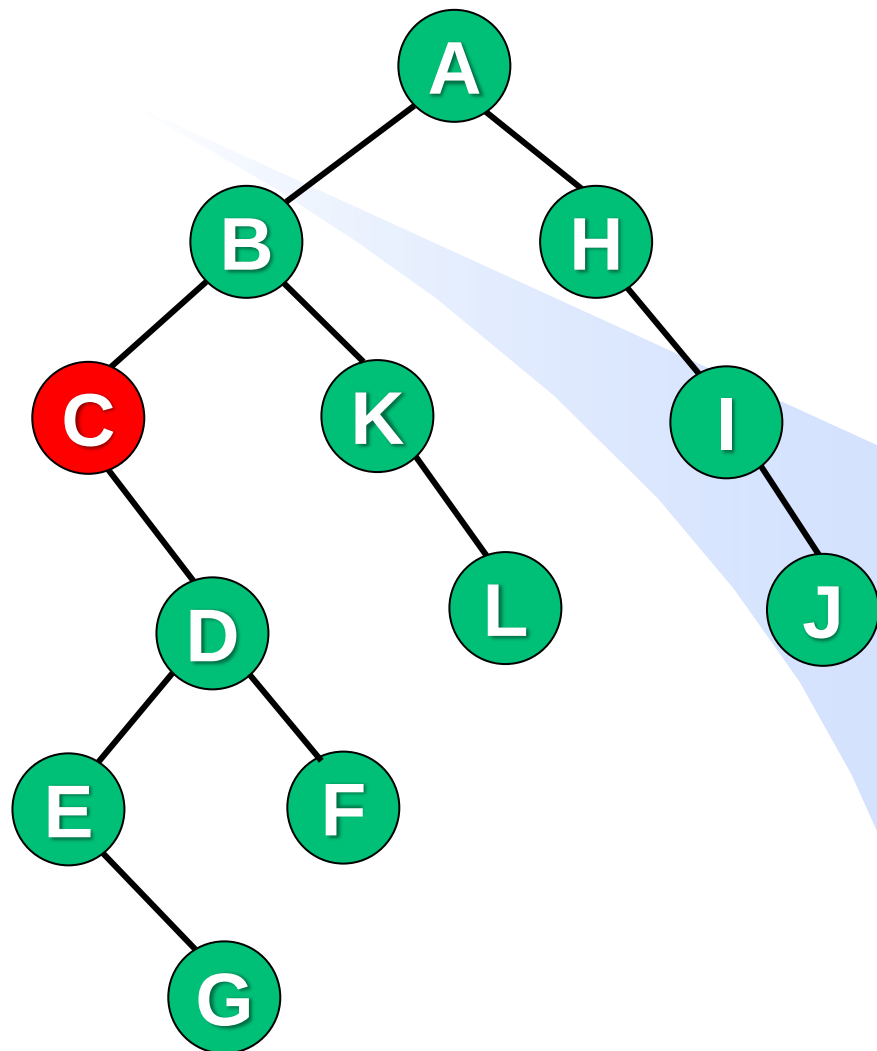
- ✓ 压栈的同时访问
- ✓ 压栈的点若有左孩子则继续把左孩子压栈，若无左孩子则弹栈
- ✓ 弹出的点若有右孩子则把右孩子压栈，若无右孩子则弹栈



中根遍历的非递归算法

中根序列第一个结点

从根结点出发，沿左分支下行，直到最深的结点（没有左孩子的结点），该结点是中根序列第一个结点



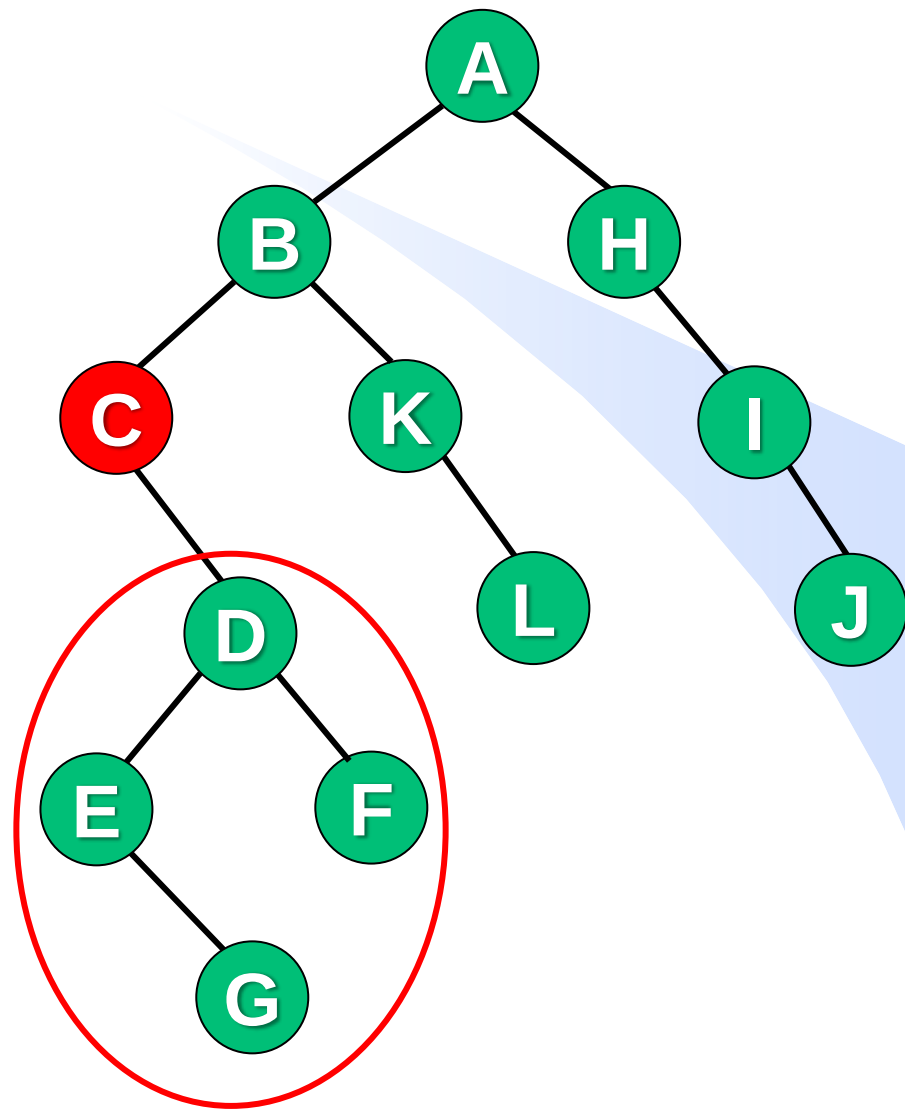
中根遍历的非递归算法

中根遍历过程：

- ① 从根结点出发**沿左分支**下行，直到最深的结点（无左孩子）。
- ② 沿着左侧通道，**自下而上**依次访问沿途各结点及其右子树。

不同右子树的遍历：

- 相互独立
- 自成一个子任务



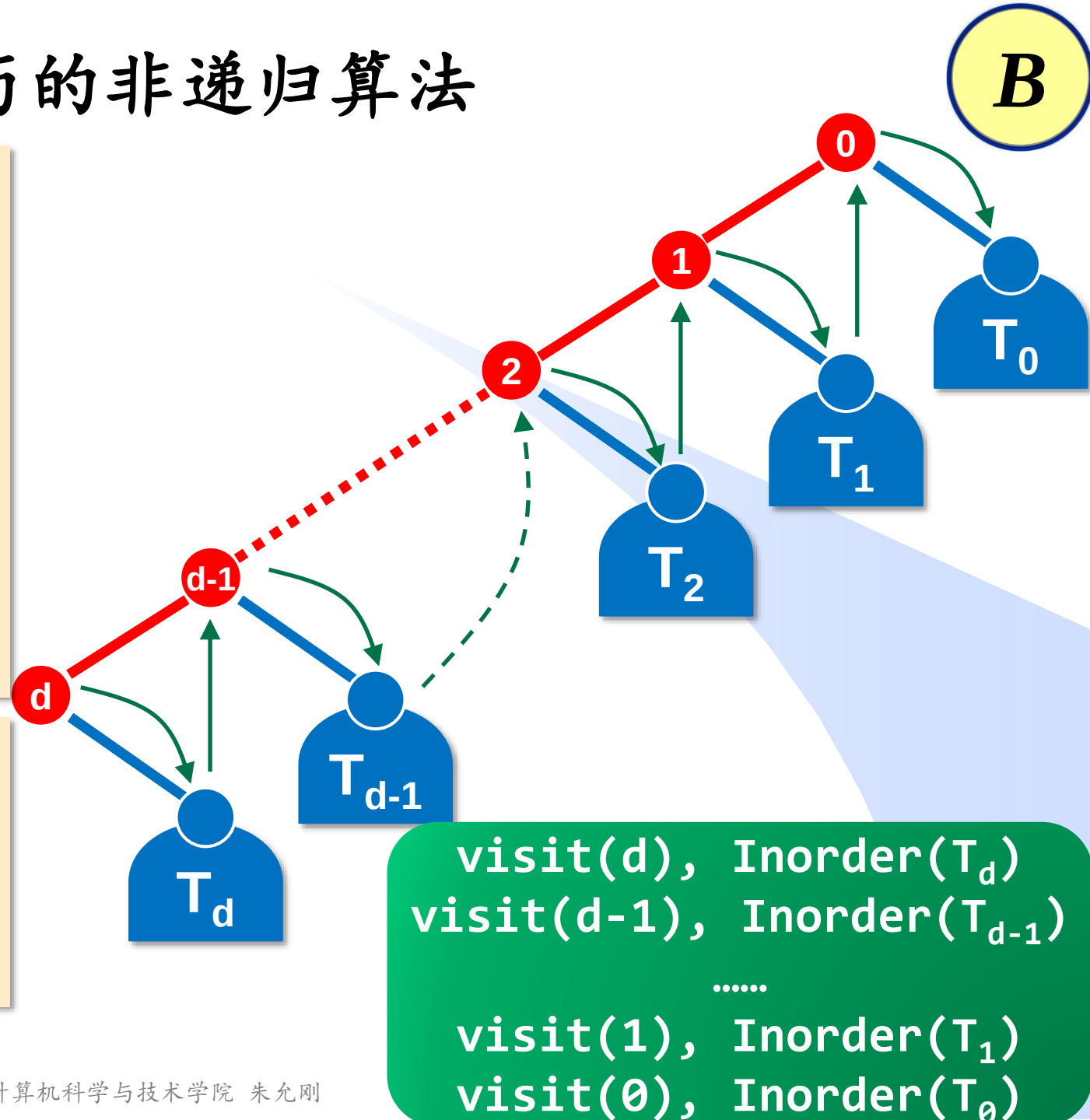
中根遍历的非递归算法

中根遍历过程:

- ① 从根结点出发**沿左分支**下行，直到最深的结点（无左孩子）。
- ② 沿着左侧通道，**自下而上**依次访问沿途各结点及其右子树。

不同右子树的遍历:

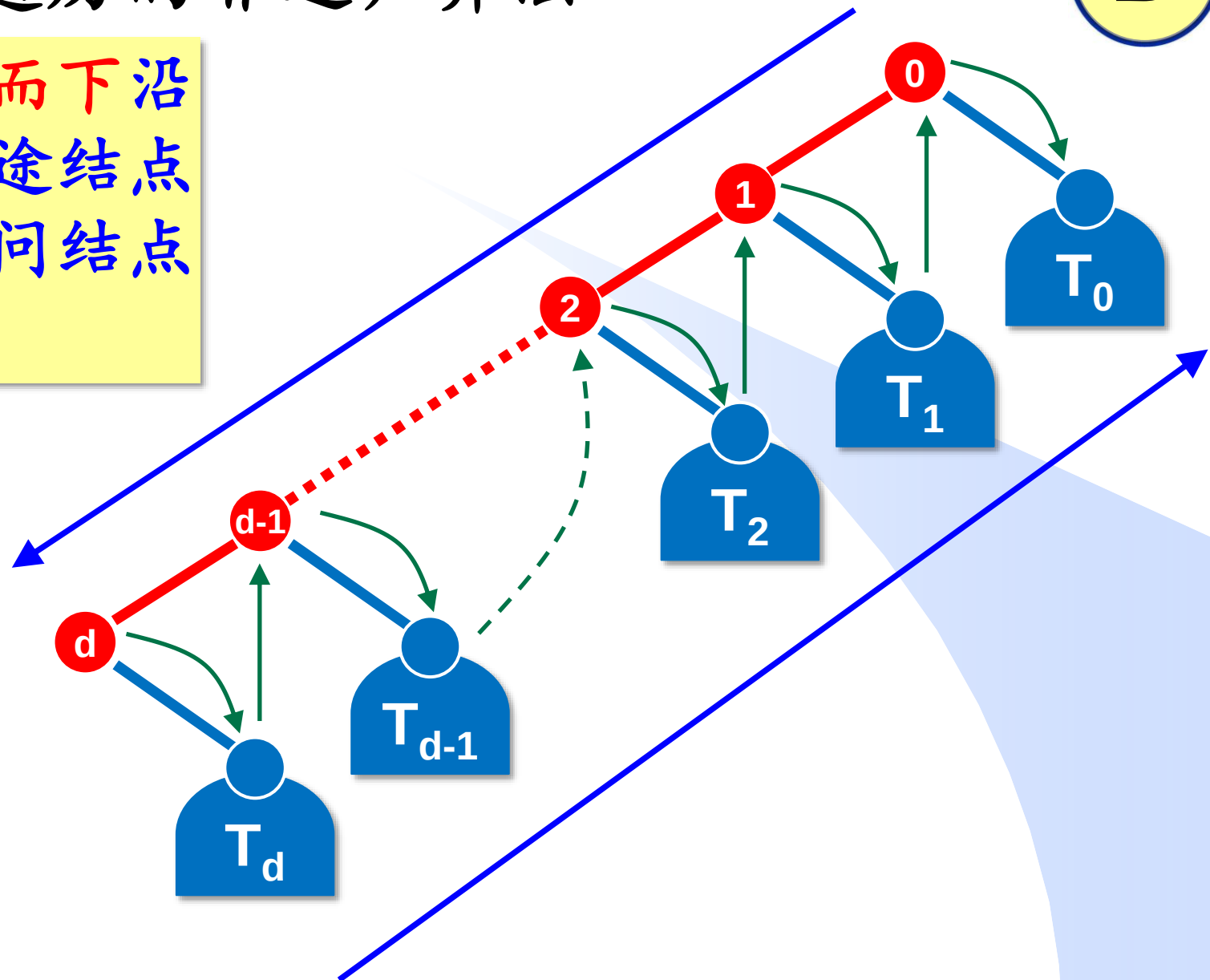
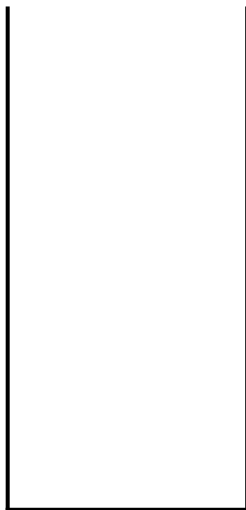
- 相互独立
- 自成一个子任务



中根遍历的非递归算法

B

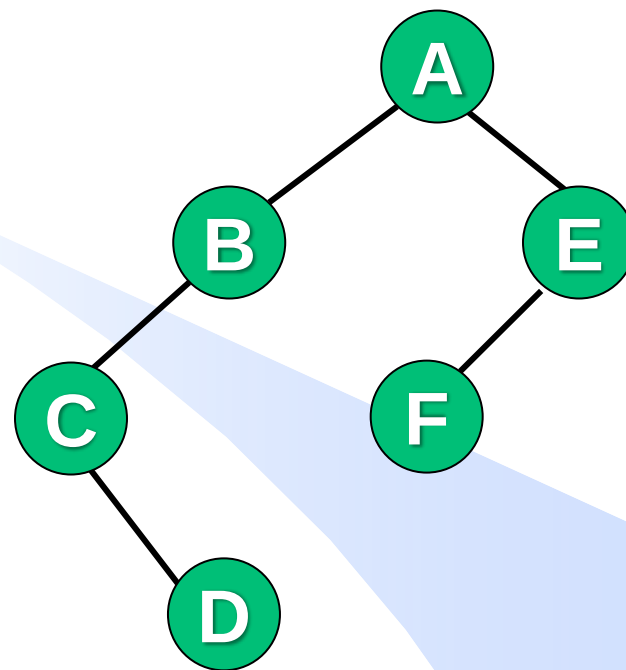
策略：从根结点开始自上而下沿着左侧分支下行，并把沿途结点压栈。自下而上弹栈，访问结点，访问其右子树



用栈存放沿途遇到的结点

中根遍历的非递归算法

```
void InOrder_NonRec(TreeNode *root){  
    Stack<TreeNode*> S; TreeNode *p = root;  
    while (true) {  
        while (p != NULL){ //沿左分支下行  
            S.PUSH(p);  
            p=p->left;  
        }  
        if (S.IsEmpty()) return;  
        p=S.POP(); //自下而上访问结点及右子树  
        visit(p->data);  
        p=p->right;  
    }  
}
```



中根遍历的非递归算法——正确性证明

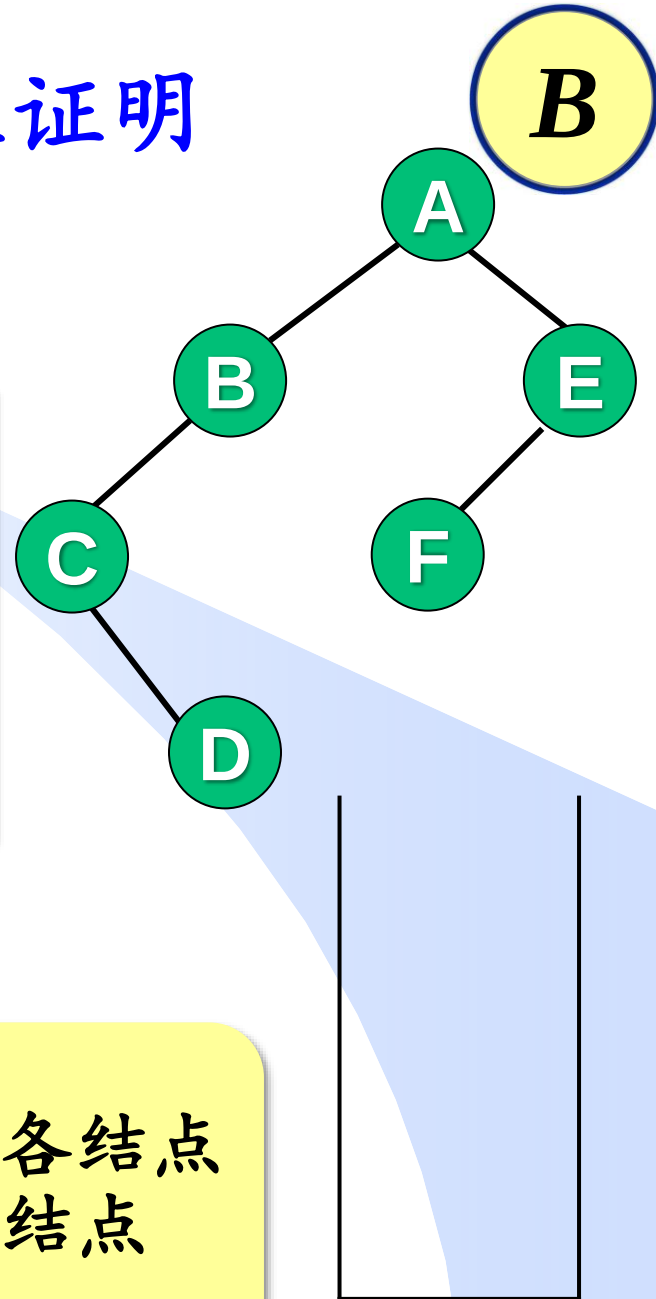
```
void InOrder_NonRec(TreeNode *root){  
    Stack<TreeNode*> S; TreeNode *p = root;  
    while (true) {  
        while (p != NULL) {  
            S.PUSH(p);  
            p=p->left;  
        }  
        if (S.IsEmpty()) return;  
        p=S.POP();  
        visit(p->data);  
        p=p->right;  
    }  
}
```

归纳法：设 n 为 p 指向的二叉树的结点个数。

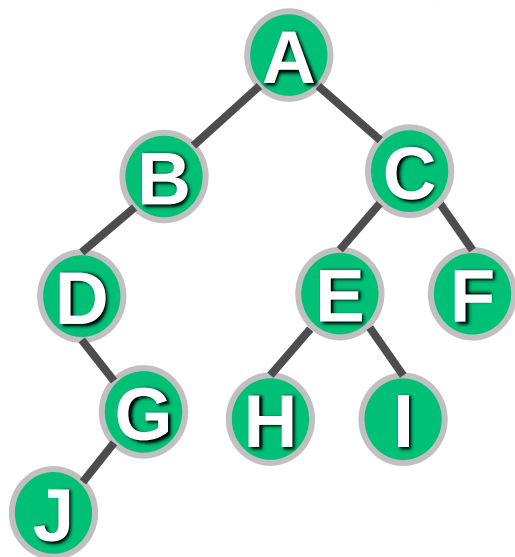
- ① $n=0$ 时成立
- ② 假设 $<n$ 时算法正确
- ③ 往证等于 n 时亦正确

算法正确：

- ✓能够按中根遍历的顺序访问各结点
- ✓遍历结束后站内不留存树中结点
- ✓遍历结束后算法执行到此处



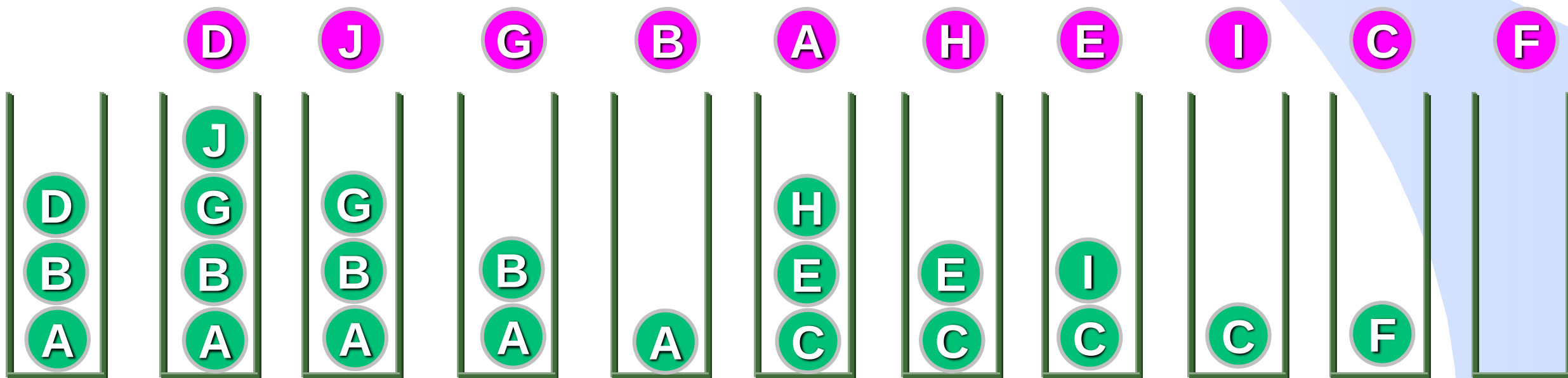
中根遍历的非递归算法——运行实例



```

void InOrder_NonRec(TreeNode *root){
    Stack<TreeNode*> S; TreeNode *p = root;
    while (true) {
        while(p!=NULL){S.PUSH(p); p=p->left;}
        if(S.IsEmpty()) return;
        p=S.POP(); visit(p->data);
        p=p->right;
    }
}
  
```

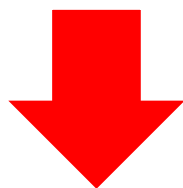
- ✓ 弹栈的同时访问
- ✓ 压栈的点若有左孩子则继续把左孩子压栈, 否则弹栈
- ✓ 弹出的点若有右孩子则把右孩子压栈, 否则弹栈



二叉树的计数

n 个结点的二叉树有多少种的形态？

- 对二叉树的 n 个结点进行编号，不妨按先根序列进行编号 $1\dots n$ 。
- 因中根序列和先根序列能唯一确定一棵二叉树，故二叉树有多少个可能得中根序列，就能确定多少棵不同的二叉树。



二叉树先根序列为 $1\dots n$ 时，有多少种可能的中根序列

非递归先根遍历

```
void PreOrder_NonRec(TreeNode* root){
    Stack S; TreeNode *p = root;
    while(true) {
        while(p != NULL){
            visit(p->data);
            S.PUSH(p);
            p=p->left;
        }
        if(S.IsEmpty()) return;
        p=S.POP();
        p=p->right;
    }
}
```

非递归中根遍历

```
void InOrder_NonRec(TreeNode *root){
    Stack S; TreeNode *p = root ;
    while(true) {
        while(p != NULL) {
            S.PUSH(p);
            p=p->left;
        }
        if(S.IsEmpty()) return;
        p=S.POP();
        visit(p->data);
        p=p->right;
    }
}
```

先根遍历的**进栈**序列=中根遍历的**进栈**序列

先根遍历的**出栈**序列=中根遍历的**出栈**序列

非递归先根遍历

```
void PreOrder_NonRec(TreeNode* root){
    Stack S; TreeNode *p = root;
    while(true) {
        while(p != NULL){
            visit(p->data);
            S.PUSH(p);
            p=p->left;
        }
        if(S.IsEmpty()) return;
        p=S.POP();
        p=p->right;
    }
}
```

非递归中根遍历

```
void InOrder_NonRec(TreeNode *root){
    Stack S; TreeNode *p = root ;
    while(true) {
        while(p != NULL) {
            S.PUSH(p);
            p=p->left;
        }
        if(S.IsEmpty()) return;
        p=S.POP();
        visit(p->data);
        p=p->right;
    }
}
```

先根遍历：结点进栈顺序就是先根访问的顺序，即进栈序列=先根序列

中根遍历：结点出栈顺序就是中根访问的顺序，即出栈序列=中根序列

二叉树先根序列为 $1...n$ 时，有多少种可能的中根序列

n 个结点的二叉树有多少种的形态？



二叉树先根序列为 $1\dots n$ 时，有多少种可能的中根序列



对于进栈序列 $1\dots n$ ，有多少种可能的合法出栈序列



$$\text{Catalan}(n) = \frac{1}{n+1} C_{2n}^n$$

练习

先根序列为 a, b, c, d 的不同二叉树的个数是 (**B**)

【考研题全国卷】

A. 13

B. 14

C. 15

D. 16

$$\text{Catalan}(n) = \frac{1}{n+1} C_{2n}^n$$

练习

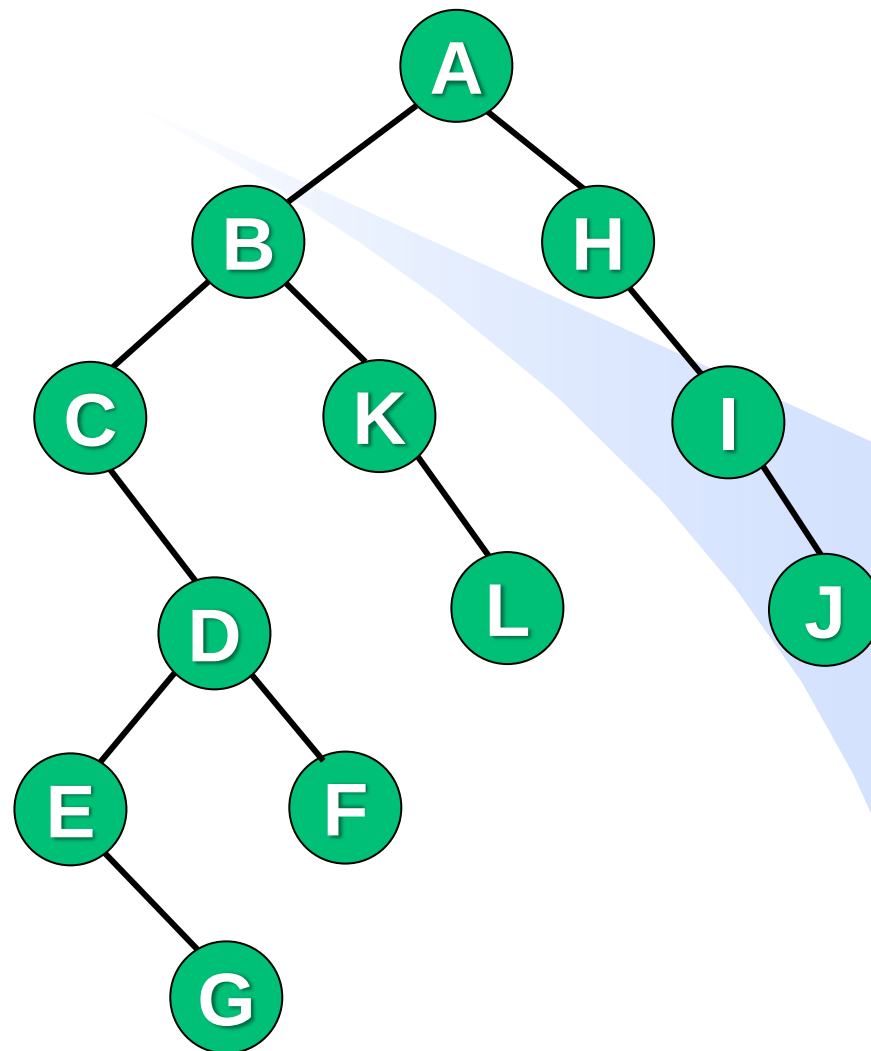
对于一个栈，若其入栈序列为 $1, 2, 3, \dots, n$ ，其合法的出栈序列个数正好等于包含 n 个结点的二叉树的个数，且与不同形态的二叉树一一对应。请简要叙述一种从“入栈序列（固定为 $1, 2, 3, \dots, n$ ）/出栈序列”对应一种“二叉树形态”的方法，并举例。【浙江大学考研题】

入栈序列 = 先根序列

出栈序列 = 中根序列

后根遍历的非递归算法

回顾中根遍历：从根结点出发，沿左分支下行，直到最深的结点

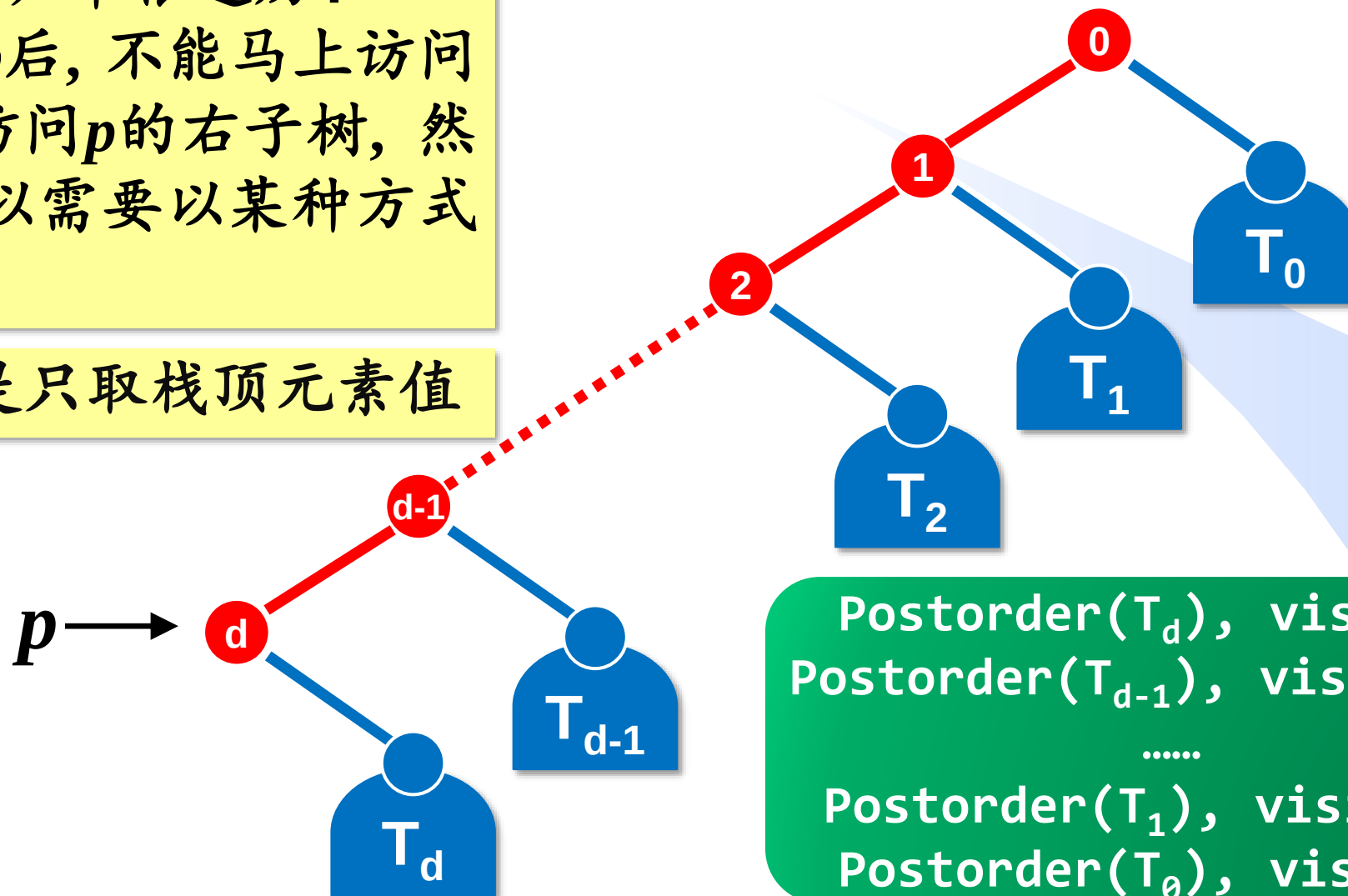


后根遍历的非递归算法

可否仿造非递归中根遍历？

问题1： 弹栈 p 后，不能马上访问 p ，而是先要访问 p 的右子树，然后访问 p 。所以需要以某种方式保存 p 。

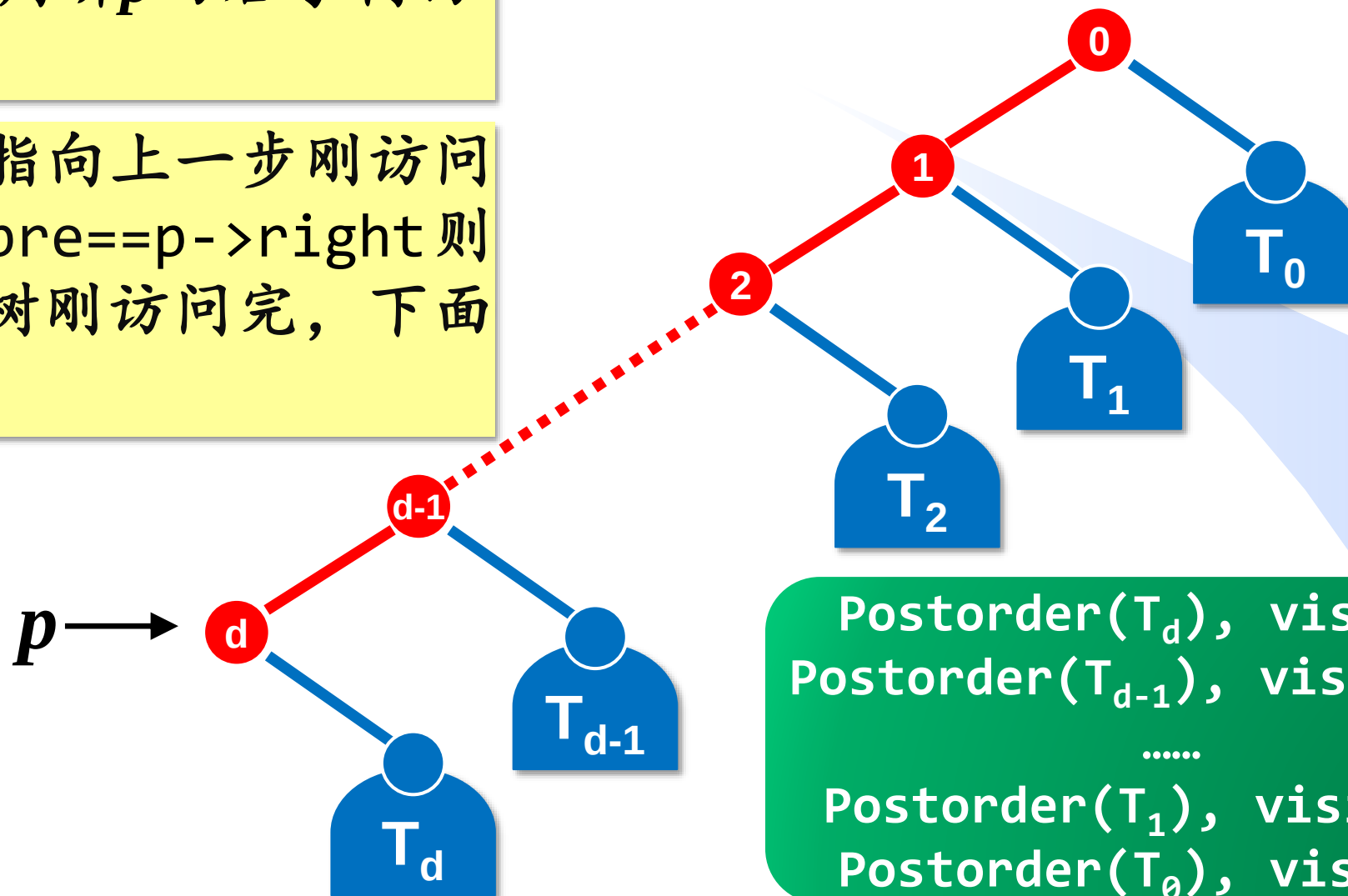
不弹栈 p ，而是只取栈顶元素值



后根遍历的非递归算法

问题2: 如何判断 p 的右子树访问完了?

设置指针 pre 指向上一步刚访问完的结点,若 $pre == p \rightarrow right$ 则表示 p 的右子树刚访问完,下面该访问 p 了



Postorder(T_d), visit(d)
 Postorder(T_{d-1}), visit($d-1$)

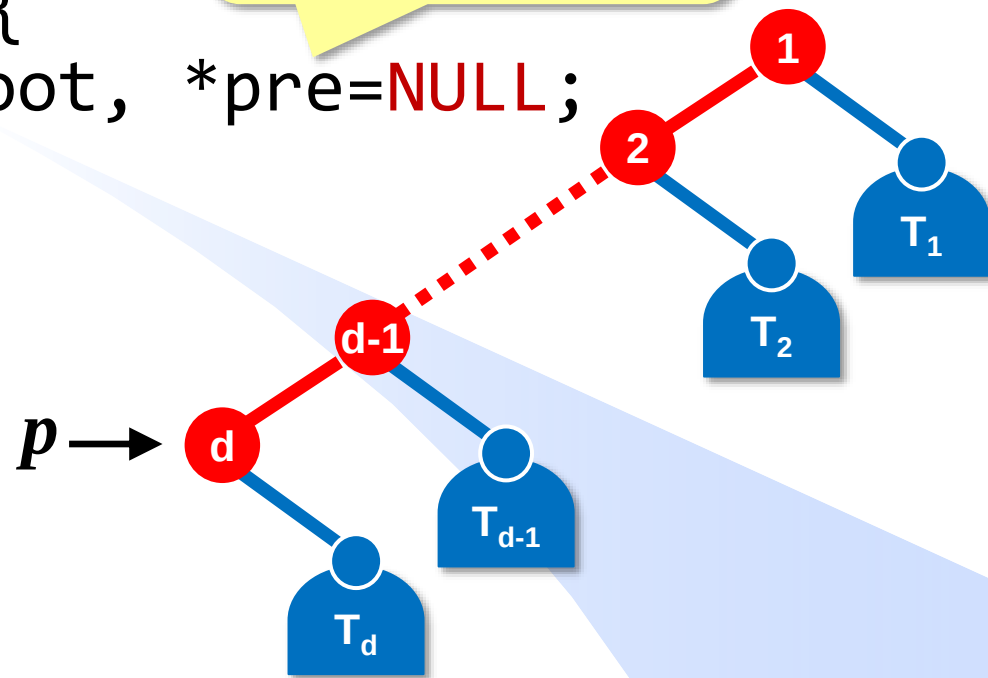
 Postorder(T_1), visit(1),
 Postorder(T_0), visit(0)

后根遍历的非递归算法

*pre*是上一步刚访问完的结点

B

```
void PostOrder_NonRec(TreeNode* root){  
    Stack<TreeNode*> S; TreeNode *p=root, *pre=NULL;  
    while (true){  
        while(p!=NULL){ //沿左分支下行  
            S.Push(p);  
            p=p->left;  
        }  
        if(S.IsEmpty()) return;  
        p=S.Peek();  
        if(p->right!=NULL && p->right!=pre) //p的右子树没访问完  
            p=p->right;  
        else{ //p无右子树或p的右子树已访问完, 此时应访问p  
            p=S.Pop(); visit(p->data); pre=p; p=NULL;  
        }  
    }  
}
```





自愿性质OJ练习题

- ✓ [LeetCode 101](#) (二叉树基础)
- ✓ [LeetCode 654](#) (二叉树基础)
- ✓ [POJ 2499](#) (二叉树基础)
- ✓ [LeetCode 236](#) (二叉树最近公共祖先)
- ✓ [LeetCode 872](#) (二叉树基础)
- ✓ [LeetCode 543](#) (二叉树基础)
- ✓ [LeetCode 863](#) (二叉树基础)
- ✓ [HDU 1622](#) (二叉树层次遍历)